EINDHOVEN UNIVERSITY OF TECHNOLOGY

Department of Mechanical Engineering
Control Systems Technology

AI-Enabled Systems Engineering: A Case Study in Failure Mode and Effects Analysis

Master's Thesis

S.S. Moosavi Lar

Supervisor:
Dr. Pascal Etman

Draft version

Eindhoven, May 2026

Abstract

Digital Engineering aims to move engineering practice from document-centred information exchange toward connected, model-based engineering environments. In this context, the usefulness of engineering models depends strongly on their integrity, including completeness, structure, traceability, and reviewability.

This thesis proposes a framework for improving model integrity in Digital Engineering environments. The framework combines SysML-oriented model parsing, ontology-based reasoning guidance, traceability support, and AI-assisted analysis. Artificial intelligence is not treated as the main research goal, but as one enabling method within a broader model integrity framework.

The framework is demonstrated using Failure Mode and Effects Analysis (FMEA) as a downstream case study task. Two SysML-style models are used: a simple flashlight model and a more complex quadcopter model. The parser extracts model elements such as parts, requirements, actions, ports, connections, and allocations. These outputs are then combined with a domain-agnostic FMEA ontology to support failure mode generation, review, scoring, explanation, and revision.

The results show that structured model information and semantic guidance can improve the traceability and reviewability of downstream engineering analysis. The work also highlights limitations, including dependence on model completeness and the continued need for human engineering judgement.

Preface

This thesis was inspired by a central question in modern engineering practice: how can engineers trust increasingly complex digital models when decisions, analyses, and risks depend on them? As engineering organisations move from document-based work toward model-based and digitally connected environments, the model is no longer only a representation of the system. It also becomes part of the decision-making environment itself. This observation motivated the present research on model integrity in Digital Engineering environments.

The project also emerged from a broader interest in systems thinking. Engineering work often involves connecting different worlds: abstract requirements and physical behaviour, human judgement and automated reasoning, established practice and emerging technology. Digital Engineering promises to connect these worlds through linked models, simulations, data, and analyses. However, that promise can only be realised if the links are trustworthy. This thesis therefore approaches artificial intelligence not as a replacement for systems engineering judgement, but as one possible assistant within a disciplined structure of evidence, semantics, and traceability.

I would like to express my sincere gratitude to my supervisor, Dr. Pascal Etman, for his guidance, feedback, and support throughout the development of this research. His supervision helped shape the project into a clearer and more academically rigorous investigation. I am also grateful to the Department of Electrical Engineering and the Artificial Intelligence and Engineering Systems programme at Eindhoven University of Technology for providing the academic environment in which this work could be developed.

This research journey involved several shifts in focus. It began with a broad interest in Digital Engineering, Model-Based Systems Engineering, and the connection between descriptive and analytical models. Over time, the work became more focused on model integrity, semantic grounding, traceability, and the responsible use of AI-supported reasoning for engineering analysis. The final thesis reflects that journey by combining systems engineering foundations, semantic technologies, traceability principles, and AI-assisted reasoning into a framework for examining engineering models and supporting traceable Failure Mode and Effects Analysis outputs.

Writing this thesis has reinforced the importance of disciplined engineering thinking in the age of AI. The central lesson is simple but important: powerful AI tools become useful in engineering only when they are connected to structure, evidence, and accountability. Without those elements, automation may produce fluent text but weak engineering knowledge. With them, AI can become a meaningful assistant in maintaining and improving the integrity of digital engineering models.

Contents

List of Figures

List of Tables

Listings

Chapter 1

Introduction

1.1 Background and Motivation

Modern engineering systems are increasingly complex. They often combine mechanical, electrical, software, control, safety, operational, and human factors considerations within a single system. As a result, engineering decisions are rarely based on isolated pieces of information. Instead, they depend on many connected information artefacts that describe what the system should do, how it is structured, how it behaves, how it is analysed, and how it is verified.

These information artefacts include requirements, functions, components, parameters, assumptions, simulations, verification evidence, and operational constraints. Each artefact may represent only one part of the engineering problem, but decisions usually require understanding how these parts relate to each other. For example, a design parameter may affect a simulation result, a simulation result may support a verification claim, and a verification claim may be linked to one or more system requirements. In this sense, the value of engineering information depends not only on the information itself, but also on the quality of the relationships between pieces of information.

Traditional document-centric engineering practices have difficulty managing these relationships. Engineering information is often distributed across reports, spreadsheets, diagrams, models, emails, databases, and tool-specific files. Although these artefacts may contain useful information, their relationships are often difficult to maintain manually. As projects grow in scale and complexity, it becomes increasingly challenging to keep information consistent, reusable, and available for decision-making.

Digital Engineering has recently emerged as a response to this challenge. It aims to support more connected, model-based, and lifecycle-oriented engineering practice. Instead of treating information artefacts as separate documents, Digital Engineering seeks to create digital environments in which models, data, analyses, and evidence can be connected and reused throughout the system lifecycle [38, 36]. The purpose is not only to digitise existing documents, but to improve how engineering information is represented, related, updated, and used.

Model-Based Systems Engineering (MBSE) provides an important foundation for the transition to digital engineering. MBSE shifts systems engineering from document-centred communication toward model-centred representation and reasoning. System models can describe information in a more descriptive way than traditional documents [26] and This makes

MBSE a key enabler for Digital Engineering, because it provides structured model-based artefacts that can support lifecycle decision-making.

MBSE mainly concerns the use of system models to represent the System of Interest (SoI). Digital Engineering is broader. It refers to a connected engineering environment in which not just models but also data, analyses and simulations are linked and reused across the system lifecycle. In this sense, MBSE provides the model-based foundation, while Digital Engineering extends this foundation into a wider digital ecosystem.

MBSE developments create the background for studying how engineering information can be managed in Digital Engineering environments. As engineering practice becomes more model-based, the quality of the model ecosystem becomes increasingly important. The central concern is not only whether models exist, but whether the information within and between those models can reliably support engineering analysis and decision-making.

1.2 Problem Statement

Although Digital Engineering and MBSE aim to improve the connection of engineering information, the existence of models does not guarantee that the information is usable for analysis. A model-based environment may contain many useful models and datasets, but the relationships between them may still be outdated or difficult to inspect. This creates a gap between the ambition of Digital Engineering and the practical use of model-based information for engineering reasoning.

In practice, requirements may not be linked clearly to functions. Assumptions may remain implicit rather than being represented as explicit model elements. Analysis results may become stale when the model changes. Parameters used in calculations may not be traceable to design variables. Different tools may use inconsistent terminology for the same concept, or the same term may be used with different meanings in different parts of the engineering environment. These issues make it difficult to determine whether the available information is still valid and whether a conclusion is supported by sufficient evidence.

These weaknesses reduce *model integrity*. Although there is no single definition for model integrity, In this thesis, model integrity is understood in practical terms such as reliability, traceability, currency, consistency, and usability of model information for downstream engineering analysis. A model ecosystem with low integrity may still appear complete at the surface level, but it may not support dependable reasoning. Engineers may have to manually inspect separate artefacts, reconstruct missing links, compare versions, interpret assumptions, and check whether the information used in an analysis is still valid.

Poor model integrity therefore makes engineering analysis more difficult to perform. Analysis tasks become slow because engineers must spend time searching for relevant information and reconstructing the reasoning behind previous decisions. They also become time-dependent because the usefulness of an analysis can decrease when the underlying model information changes. If for instance a requirement or parameter is updated, previous conclusions may become outdated.

This issue affects several downstream engineering tasks, such sa failure analysis, risk assessment, verification planning, and so on. These tasks depend on knowing which requirements, functions and components are connected. When those connections are missing or unreliable, engineers must reconstruct the reasoning manually. This increases effort, slows down decision-making, and makes the analysis harder to update after model changes.

Therefore, the central problem addressed in this thesis is how poor model integrity limits the

quality characteristics of downstream engineering analyses in Digital Engineering environments. The thesis focuses on this problem by studying how existing tools and methods can be organised into an AI supported digital framework that supports higher model integrity.

1.3 Research Goal

The goal of this thesis is to develop an AI supported framework for improving model integrity in Digital Engineering environments and to demonstrate its application through a concrete downstream engineering analysis task. The thesis focuses on how engineering information can remain reliable, connected, traceable, and reusable when models evolve over time.

To achieve this goal, the thesis investigates existing tools and methods that can support higher model integrity. These include descriptive modelling notations, ontologies, and artificial intelligence. Descriptive modelling notations can help represent system elements and relationships explicitly. Ontologies can help clarify meanings that are used in models and support reuse across contexts. Artificial intelligence can leverage both descriptive modelling notations and ontologies, to support selected reasoning and information-processing tasks when used within a controlled engineering workflow.

These tools and methods are not studied as isolated or competing solutions. Instead, they are considered as complementary mechanisms that can address different aspects of model integrity. The proposed framework aims to show how engineering information can be structured, linked, checked, and maintained so that downstream analysis tasks remain trustworthy after model changes.

The goal is not to present one tool or technology as the complete solution. Rather, the thesis aims to explain how AI can be build upon descriptive modeling notations and ontologies within a Digital Engineering workflow to support higher model integrity. This is important because model integrity is not a single-tool problem. It depends on how information is represented, how relationships are maintained, and how changes are reflected in downstream analyses.

Failure Mode and Effects Analysis (FMEA) is used as the demonstration task because it is strongly dependent on model integrity. FMEA is not treated as the main goal of the thesis, but as a case study for examining how model integrity affects downstream engineering analysis. This makes it possible to evaluate the proposed framework in a concrete setting while keeping the broader focus on model integrity.

The expected outcome is a practical framework that helps engineers reason about how to employ AI, ontologies and modeling notations for improving the quality characteristics of model-based engineering information.

1.4 Research Questions

This thesis is guided by two research questions.

RQ1: How can existing tools and methods, including descriptive modelling notations, ontologies, and artificial intelligence, be organised into a practical framework for improving model integrity, with Failure Mode and Effects Analysis used as the demonstration case?

RQ2: To what extent does the proposed framework support the quality of downstream engineering analysis, and how can the framework be improved and evaluated through FMEA

as a case study?

The first research question concerns the development of the framework. It focuses on how existing tools and methods can be selected, related, and organised to support model integrity in Digital Engineering environments. The second research question concerns evaluation. It examines whether the proposed framework supports downstream engineering analysis in a concrete case study and identifies how the framework can be improved.

Together, the research questions move from framework development to case-study evaluation. This structure supports the broader aim of the thesis: to understand how higher model integrity can be achieved in Digital Engineering environments and how this can improve the reliability and maintainability of downstream engineering analyses.

1.5 Scope

This thesis focuses on model integrity in Digital Engineering environments. It studies selected tools and methods that are relevant to improving the quality of model-based engineering information. These tools and methods include descriptive modelling notations, ontologies and artificial intelligence.

The thesis uses a compact engineering case study to demonstrate and evaluate the proposed framework. The case study is suitable because it includes components, functions, parameters, assumptions, analytical relations, and performance outputs. This makes it sufficiently rich to represent common model integrity challenges, while remaining small enough to inspect and explain within the scope of an MSc thesis.

The thesis does not aim to build a complete industrial Digital Engineering platform. It also does not claim to solve all interoperability problems across engineering tools. Instead, the focus is on developing and evaluating a framework that explains how different methods can be combined to support higher model integrity.

The thesis also does not claim to replace domain experts, safety engineers, or engineering judgement. The proposed framework is intended to support engineering reasoning, not automate responsibility. Human judgement remains necessary for interpreting results, validating assumptions, and making final engineering decisions.

FMEA is used as the demonstration task for studying how model integrity affects downstream engineering analysis. However, the model integrity problem addressed in this thesis is broader than FMEA. The same underlying issues also affect other downstream tasks, such as risk assessment, verification planning, design review, trade-off analysis, compliance checking, and impact analysis.

1.6 Contributions

This thesis makes five main contributions.

First, it defines model integrity in practical terms as the reliability, traceability, currency, consistency, and usability of model information for downstream engineering analysis. This definition supports the thesis by focusing on how model information is actually used in engineering workflows, rather than treating model quality as an abstract property.

Second, the thesis reviews and organises tools and methods that can support model in-

tegrity in Digital Engineering. These include descriptive modelling notations, semantic representations, ontologies, graph-based representations, and artificial intelligence. The contribution lies in examining how these methods can support model integrity when considered together.

Third, the thesis proposes a framework for achieving higher model integrity using a combination of modelling, ontologies, and AI-supported reasoning. The framework provides a structured way to reason about how engineering information should be represented, maintained, and used in downstream analysis.

Fourth, the thesis demonstrates how the proposed framework can support downstream engineering analysis through an FMEA case study. The case study is used to examine how model integrity influences the reliability, traceability, and maintainability of analysis outputs.

Fifth, the thesis provides guidance on the responsible use of artificial intelligence within a broader model integrity framework. Rather than positioning AI as the central research goal, the thesis treats it as a supporting method that depends on structured model information from descriptive modelling notations and semantic grounding from ontologies. In this role, AI can assist with selected reasoning and information-processing tasks, but it should not replace engineering judgement. Its outputs must remain reviewable, and subject to human-in-the-loop validation so that engineering accountability is preserved.

1.7 Thesis Structure

The remainder of this thesis is organised as follows.

Chapter 2 reviews the literature on Digital Engineering, MBSE, modeling notations, semantic interoperability, ontologies, traceability, FMEA, and AI-supported engineering. This chapter establishes the theoretical and technical background needed to understand the model integrity problem and the methods considered in the proposed framework.

Chapter 3 presents the methodology. It describes the research design, selected tools and methods, proposed framework, case study, and evaluation approach. The chapter explains how the framework is constructed and how its usefulness is examined through the selected downstream engineering analysis task.

Chapter 4 applies and evaluates the proposed framework using the case study. It presents the results of the framework application and discusses how the framework supports reliability, traceability, and maintainability in downstream engineering analysis.

Chapter 5 concludes the thesis. It summarises the main findings, discusses the implications and limitations of the research, and proposes directions for future work.

Chapter 2

Literature Review

This chapter reviews the literature that motivates the thesis focus on improving model integrity in Digital Engineering environments. It first discusses the transition from document-based systems engineering toward connected digital models and lifecycle information. It then explains the role of Model-Based Systems Engineering (MBSE) and descriptive modelling notations in representing system information. The chapter subsequently develops the concept of *model integrity*, which is used in this thesis to describe whether model information remains reliable, traceable, current, consistent, and usable for downstream engineering analysis. It then reviews semantic representations, ontologies, knowledge graphs, and artificial intelligence as complementary methods that may support this goal. Finally, the chapter identifies the research gap addressed by the proposed framework and positions Failure Mode and Effects Analysis (FMEA) as the downstream demonstration task that is treated in more detail in the methodology chapter.

2.1 History of Digital Engineering

Systems engineering has traditionally relied on document artefacts distributed across teams and tools. Examples include specifications, interface control documents, review packs, spreadsheets, and test records. These artefacts remain valuable because they support communication, contractual exchange, certification, and review. However, when engineering information is mainly managed through documents, the relations between requirements, architectures, assumptions, analyses, and verification evidence are often difficult to maintain [10, 26, 6]. The problem is therefore not only that engineering projects produce many documents. The deeper problem is that the meaning, status, and justification of engineering information can become fragmented across time.

This fragmentation becomes more serious in contemporary socio-technical systems. Engineering work is distributed across disciplines, lifecycle stages, organisations, and digital tools. A design change in one domain can affect interfaces, constraints, analyses, tests, and supporting evidence elsewhere. As a result, the quality of engineering decisions depends increasingly on whether information can be located, interpreted, connected, and updated across the lifecycle, rather than on whether individual documents exist in isolation [38, 40, 34].

Digital Engineering has emerged as a response to this need for lifecycle-connected engineering information. The United States Department of Defense defines Digital Engineering as an integrated digital approach that uses authoritative sources of system data and mod-

els as a continuum across disciplines to support lifecycle activities from concept through disposal [38]. DoDI 5000.97 extends this position operationally by describing Digital Engineering as the use and integration of digital models and underlying data to support system development, test and evaluation, and sustainment, while moving the primary means of communicating system information from documents to digital models and their underlying data [40]. In this view, models are not merely digital versions of documents. They are intended to become central carriers of engineering information, connected to the data, artefacts, and processes needed for lifecycle decision-making.

Two related concepts are especially important for Digital Engineering: the *digital thread* and the *authoritative source of truth*. The digital thread can be understood as the connective structure that links engineering information across the product lifecycle. It connects requirements, models, analyses, simulations, tests, decisions, and evidence so that information can be accessed, traced, and reused across lifecycle stages. Singh and Willcox describe the digital thread as a data-driven architecture that links information generated across the product lifecycle and can act as the primary or authoritative data and communication platform for a product at a given point in time [34]. DoDI 5000.97 similarly states that digital threads connect authoritative data and orchestrate digital models and information across a system's lifecycle, providing the capability to access, integrate, and transform data into actionable information [40].

The *authoritative source of truth* gives this connected information its trusted reference point. DoDI 5000.97 defines it as the reference point for models and data across the system life cycle, providing traceability as the system evolves, capturing historical knowledge, and connecting configuration-controlled versions of models and data [40]. In other words, the digital thread explains how lifecycle information is connected, while the authoritative source of truth explains which information is trusted, current, configuration-controlled, and suitable to use for engineering decisions. The two concepts therefore depend on each other. A digital thread without authoritative sources may connect information that is inconsistent or outdated. Conversely, an authoritative source of truth without a digital thread may remain isolated and difficult to use across lifecycle activities.

Importantly, the authoritative source of truth should not be interpreted as a single central repository containing all engineering information. In practice, authority may be distributed across several controlled tools, databases, models, and repositories. What makes these sources authoritative is not their physical location, but the governance, provenance, version control, traceability, and trust mechanisms that make their information reliable for use. The digital thread then connects these authoritative sources so that engineering information can be followed, reused, and assessed across the system lifecycle [40, 34, 15].

This distinction helps separate *digitising documents* from *creating connected engineering information*. Scanning or storing documents digitally may improve access, but it does not automatically turn document content into connected engineering knowledge. For example, explicit identifiers make it possible to refer to requirements, components, assumptions, or analysis results without ambiguity. Dependency relations show how one item depends on another, such as a requirement depending on a design decision or a verification result depending on a test. Versioned traces preserve which version of a model or evidence item supported a particular conclusion. Semantic alignment helps ensure that different tools or teams use terms with compatible meanings. Machine-actionable connections allow software tools to query, check, or propagate information without relying entirely on manual interpretation. Digital Engineering aims to establish these kinds of connections so that engineering information becomes accessible, understandable, trustworthy, interoperable, and reusable over time [39, 40].

Table 2.1 summarises this shift from document-based systems engineering to MBSE and

Digital Engineering. The table is based on literature describing MBSE, Digital Engineering policy, digital threads, and tool-chain integration challenges [10, 26, 6, 38, 40, 34, 25]. It shows that the shift is not simply a movement from paper to digital files, but a movement toward information that is more connected, traceable, and suitable for downstream analysis.

Table 2.1: Shift from document-based systems engineering to MBSE and Digital Engineering

Paradigm	Primary artefact	Main strength	Main limitation	Relevance to model integrity	References
Document-based systems engineering	Specifications, reports, spreadsheets, drawings	Familiar, readable, and useful for contractual review	Relationships are often fragmented, duplicated, and manually updated	Integrity problems may remain hidden until downstream work is manually reconciled	[10, 26, 6]
Model-Based Systems Engineering	System models and model relations	Provides stronger structure, traceability, analysis integration, and communication	Benefits depend on method, governance, modelling discipline, and tool integration	Improves information structure, but does not guarantee that model content remains current or trustworthy	[10, 26, 21, 25]
Digital Engineering	Connected models, data, simulations, services, and lifecycle traces	Supports lifecycle continuity, reusable information, and data-driven decisions	Depends strongly on semantics, interoperability, governance, and curation	Makes integrity a first-order concern because downstream work depends directly on connected digital information	[38, 40, 34, 39]

For this thesis, the key implication is that Digital Engineering raises the standard for engineering information. Once downstream analyses rely on connected models and digital threads, poor connectivity, weak traceability, stale assumptions, or broken evidence links are no longer minor documentation inconveniences. They become obstacles to trustworthy engineering analysis and to the maintenance of that analysis after model changes.

2.2 Model-Based Systems Engineering

MBSE is widely treated as the principal systems-engineering foundation for Digital Engineering. Estefan's influential INCOSE survey defines MBSE as "the formalized application of modeling to support system requirements, design, analysis, verification and validation activities beginning in the conceptual design phase and continuing throughout development and later life cycle phases" [10]. Madni and Sievers similarly describe MBSE as a systems-engineering approach centred on the evolving system model, with the purpose of managing increasing system complexity while maintaining consistency and traceability [26].

The relation between MBSE and Digital Engineering is therefore foundational but not identical. MBSE focuses on the use of models to represent and reason about the system. Digital Engineering is broader: it concerns the connected digital environment in which models, data, analyses, simulations, evidence, and lifecycle information are managed and reused.

In this sense, MBSE provides a model-based foundation for Digital Engineering, while Digital Engineering extends beyond MBSE by emphasising lifecycle connectivity, authoritative data, interoperability, governance, and digital-thread continuity.

In practical terms, MBSE aims to represent the system in ways that are rich enough to support engineering reasoning rather than only documentation. The Systems Modeling Language (SysML) is the most widely used general-purpose modelling language for MBSE. It extends the Unified Modeling Language (UML) with systems-engineering constructs for representing requirements, structure, behaviour, interfaces, properties, parametric relations, verification cases, and relationships among them [28, 12]. Official SysML descriptions and MBSE practice literature show that such models may capture more than one kind of engineering information in a single model environment [10, 28, 30]. This representational breadth is important because downstream engineering problems often require coordinated access to several kinds of information. For example, a change to a requirement may affect architectural elements, allocated functions, parameter values, simulations, test cases, and supporting evidence.

The literature commonly associates MBSE with several benefits. Models can improve communication by giving stakeholders shared abstractions of the system and its decomposition [26, 21]. They can also improve traceability by making relations among requirements, design constructs, analyses, and verification artefacts explicit [10, 26]. In addition, models can support consistency management, reuse, transformation, and earlier analysis because behaviour, structure, and constraints become available in forms that can be inspected, simulated, or transformed before costly implementation steps [26, 25, 30]. Finally, models can support lifecycle impact analysis, since a change can in principle be followed through model relations rather than rediscovered manually across disconnected documents [21, 25].

These benefits should not be overstated. MBSE literature consistently notes that the existence of models does not ensure successful outcomes. Huldt and Stenius show that MBSE adoption and perceived value vary significantly across organisations and depend on how MBSE is implemented in practice [21]. De Saqui-Sannes et al. argue that MBSE must be understood through the combined choice of *languages*, *tools*, and *methods*, because weaknesses often arise at their intersections rather than from notation alone [6]. Likewise, toolchain reviews emphasise interoperability, lifecycle integration, transformation, and traceability as recurring concerns rather than solved problems [25]. Put differently, MBSE can improve the structure and availability of engineering information, but it does not automatically guarantee that model information is reliable, current, justified, or easy to maintain after change.

That distinction is central to this thesis. A project may have an MBSE repository and still suffer from stale parameter values, missing trace links, unrecorded assumptions, locally meaningful but globally inconsistent terms, or analysis results that cannot be reconnected to revised model content. These weaknesses matter especially in Digital Engineering, because downstream analyses increasingly depend on connected model information. In such cases, the engineering problem is not the absence of models. It is insufficient integrity of the information within and between those models.

2.3 Descriptive Modelling Notations

A *descriptive modelling notation* is a language or notation used to describe a system in a structured way. It provides a vocabulary of modelling elements, such as requirements, components, functions, interfaces, behaviours, constraints, and allocations, together with rules for how these elements may be related. In MBSE practice, such notations allow engineering

information to be organised, externalised, shared, and processed by tools [6, 26]. Descriptive modelling notations matter because they influence what can be represented explicitly and what remains hidden in informal conventions or free text.

SysML is one example of a descriptive modelling notation, but it is not the only one. Other modelling approaches and notations used in systems engineering and adjacent fields include UML-based modelling, architecture description languages, domain-specific modelling languages, entity–relationship modelling, process modelling notations, and specialised simulation or analysis models. In this thesis, SysML is emphasised because it is a widely used MBSE language and because SysML v2 and KerML are directly relevant to the future of model-based Digital Engineering [28, 30, 29]. The relation is therefore as follows: MBSE is the engineering approach, while descriptive modelling notations such as SysML provide the modelling language used to express system information within that approach.

SysML v1 became the dominant general-purpose systems modelling language in MBSE because it extended UML with facilities relevant to systems engineering, including requirements, structural composition, interfaces, behaviours, parametrics, and allocation relations [28, 12]. Its broad uptake made it an important descriptive notation for architecture representation and traceability-oriented modelling across sectors. The lasting contribution of SysML v1 is therefore not that it solved all systems-engineering modelling problems, but that it provided a shared descriptive language around which tools, practices, tutorials, and organisational methods could develop.

At the same time, the literature also shows why SysML v1 should not be treated as sufficient for the integrity problem addressed in this thesis. Its semantics were historically only partially precise, meaning that the exact interpretation of some model elements and relations depended on modelling conventions, tool behaviour, or project-specific methods [3, 6]. Many practical modelling decisions therefore remained method-dependent. For example, two teams could both use SysML but apply different rules for decomposition, allocation, naming, or traceability. Interoperability across tools was also limited, because different tool implementations did not always exchange or interpret model content in the same way. Thus, even when teams used the same notation, meaning and usage patterns could still vary across contexts.

SysML v2 and KerML represent a substantial development [3, 29, 30]. The formal OMG material describes KerML as an application-independent modelling language with a well-grounded formal semantics, and SysML v2 as a systems-engineering specialisation built on that foundation [29, 30]. Official OMG material and the supporting literature emphasise intended improvements over SysML v1: greater precision and expressiveness, more consistent language concepts, complementary textual and graphical representations, and standardised APIs and services for accessing and operating on KerML-based and SysML-based models [3, 30, 31]. These developments are especially relevant in Digital Engineering environments because they improve the conditions for automation, model transformation, repository access, fine-grained integration, and model-based services.

The significance of SysML v2 for this thesis lies in the kind of support it can provide for higher model integrity. A more precise language and a standard Application Programming Interface (API) can make it easier to create model elements that are better structured, more consistently interpreted, and more readily available for downstream consumption. They can reduce ambiguity at the notation and tooling layers, and they can provide a stronger substrate for traceability, query, and transformation services [3, 31].

However, the literature also cautions against a simplistic "language-upgrade" narrative. Almeida et al. show that the semantic foundation of KerML and SysML v2 is richer and more demanding than a straightforward replacement story suggests [1]. Better formal underpinnings improve the potential for rigour, but they do not by themselves solve every integrity

concern. Semantic alignment across domains still requires agreement that terms used in different disciplines refer to compatible concepts. Evidence curation still requires deciding which analyses, tests, assumptions, or documents support a claim. Modelling governance is still needed to define who may change the model, how changes are reviewed, and how modelling rules are enforced. Downstream maintainability still requires mechanisms that show which analyses must be revisited when model content changes. These concerns go beyond the modelling language itself.

The practical distinction, therefore, is between *representing information* and *maintaining trustworthy connected information*. Descriptive notations help engineers represent requirements, structure, behaviour, interfaces, and analytical context in more systematic ways than free-text documents usually allow. Yet model integrity also depends on several additional conditions. Terminology must be consistent across teams, tools, and lifecycle stages, not only within the SysML language definition. Identifiers and links must be maintained so that model elements, analysis results, requirements, and evidence can still be connected after updates. Evidence chains must be preserved so that a conclusion can be traced back to the assumptions, model elements, tests, or analyses that support it. Version changes must be reflected in downstream artefacts so that old analyses are not mistakenly treated as valid for a revised model. Finally, model content must be queryable and reusable without requiring engineers to manually reinterpret diagrams, naming conventions, or hidden assumptions each time. Table 2.2 summarises why descriptive notations are essential for model integrity, but not sufficient on their own.

Table 2.2: MBSE, SysML, and SysML v2 contributions to model integrity

Approach	**What it helps represent**	**Contribution to model integrity**	**Remaining limitation**
MBSE as engineering approach	Requirements, architecture, behaviour, interfaces, analysis, and V&V relations	Provides a model-centred basis for connected engineering information	Does not ensure that model content is current, semantically aligned, or evidence-linked
SysML v1.x	Widely used descriptive representation of requirements, structure, behaviour, and parametrics	Makes relations more explicit than document-based practice and supports traceability-oriented modelling	Semantic interpretation and interoperability often remain tool- and method-dependent
KerML + SysML v2	More precise foundations, complementary textual and graphical modelling, model services, and standard APIs	Improves precision, automation potential, interoperability support, and access to models as data	Still does not by itself guarantee consistency, governance, evidence quality, or maintainability after change

2.4 Model Integrity in Digital Engineering

Although the literature regularly discusses model quality, consistency, traceability, interoperability, and trust in related ways, there is no single universally adopted definition of *model integrity* in Digital Engineering. For the purposes of this thesis, model integrity is defined

as *the reliability, traceability, currency, consistency, and usability of model information for downstream engineering analysis*. This is a practical rather than purely theoretical definition. It is intended to capture the properties that make model information dependable when engineers need to analyse, justify, revise, and maintain downstream work as system information evolves.

Table 2.3 summarises the five model integrity concerns used in this thesis. *Reliability* means that the information can be trusted sufficiently for engineering reasoning. This does not require that every model be perfect or final, but it does require that the status, provenance, and intended use of the information are clear enough that engineers can judge whether it is fit for purpose [40, 39]. *Traceability* means that claims, outputs, and decisions can be linked back to the model elements, assumptions, and supporting evidence on which they depend [26, 40]. *Currency* means that model information remains valid, or at least recognisably out of date, when upstream changes occur. It therefore includes change propagation, version control, and visibility of what needs to be revisited after a modification [40, 39]. *Consistency* means that relations, identifiers, and terminology do not conflict across the connected model environment [39, 7]. Finally, *usability* means that downstream tasks can actually consume the model information. Information may be present somewhere in a repository and still be unusable if it is scattered, weakly linked, semantically unclear, or difficult to retrieve in a form needed for analysis.

This focus on model integrity is necessary because engineering models evolve continuously. Requirements are clarified, interfaces change, parameters are recalibrated, simulations are rerun, test results accumulate, and assumptions are revised. Under these conditions, the central practical question is not only whether a model is syntactically valid, but whether the information relevant to later analysis can still be found, understood, justified, and updated when the source model changes. DoDI 5000.97 is explicit that programmes should maintain digital models and associated data throughout the lifecycle so that stakeholders can extract and analyse consistent and up-to-date system information, and that authoritative sources of truth should remain current, consistent, and enduring [40]. This policy language closely matches the integrity concerns addressed in the present thesis.

Model integrity failures tend to appear in immediately recognisable engineering forms. Missing links between requirements and architecture make it difficult to judge whether a downstream analysis covers the revised design. Implicit assumptions buried in notes, emails, or analyst memory mean that a later user may not know why a certain parameter value or failure interpretation was chosen. Stale analysis outputs can remain attached to a model even after the source configuration has changed. Inconsistent terminology across teams or tools can make components, functions, operating states, or evidence sets appear different when they are not, or equivalent when they should not be. Broken evidence chains make it difficult to justify why a conclusion, risk statement, or recommendation should still be trusted [4, 40, 8]. In each case, the problem is not simply bad modelling style. It is degradation of the engineering information needed for downstream reasoning.

The downstream consequences are substantial. Analyses become slow because engineers must manually reconstruct context that ought to have been explicit. They become fragile because small upstream changes can silently invalidate large portions of the reasoning chain. They become difficult to justify because provenance is incomplete or disconnected. They also become difficult to maintain because each revision requires rediscovering what earlier work depended on. Weak model integrity therefore creates a time-dependent engineering burden: as the system evolves, earlier analyses become progressively harder to reuse or defend, even when the original technical reasoning was sound. This is one reason why the thesis places model integrity at the centre rather than treating it as a secondary quality property of models.

The practical thesis contribution is not to claim a universal taxonomy, but to provide a working integrity concept tied directly to downstream engineering use. In this sense, model integrity is less about abstract conformance and more about whether model information continues to support engineering tasks after changes that occur in real projects.

Table 2.3: Practical model integrity concerns in this thesis

Integrity concern	Definition	Typical failure symptom	Effect on downstream analysis
Reliability	Information can be trusted for its intended analytical use	Unclear provenance, uncertain status, unsupported values or results	Conclusions are hard to defend and may be quietly unreliable
Traceability	Outputs and claims can be linked to source model elements and evidence	Missing satisfy/verify links, unexplained assumptions, broken evidence paths	Analysts must reconstruct rationale manually and coverage becomes doubtful
Currency	Information remains valid after model and evidence changes	Stale analysis tables, outdated parameters, unrevised assumptions	Earlier results become partially invalid or misleading after change
Consistency	Terminology and relationships do not conflict across connected artefacts	Duplicate concepts, contradictory identifiers, incompatible relations	Queries, transformations, and comparisons become brittle or ambiguous
Usability	Information can actually be consumed by downstream tasks	Relevant content exists but is scattered, implicit, or inaccessible	Analysis becomes slow, manual, and difficult to repeat or maintain

2.5 Semantic Representations: Ontologies and Knowledge Graphs

Descriptive syntax alone is insufficient when engineering information must remain connected across tools, domains, and lifecycle stages. Here, *syntax* refers to the permitted form of a model: the modelling elements, symbols, relations, and rules used to express information. *Semantics* refers to the meaning of that information: what the concepts refer to, how they are interpreted, and which relationships are valid in a given engineering context. A model element may therefore be syntactically well formed while its intended meaning remains ambiguous outside the local modelling convention in which it was created. This is why a substantial part of the literature turns from notation to explicit semantics.

Foundational ontology literature offers a useful starting point. Gruber's well-known definition describes an ontology as an explicit specification of a conceptualisation [13]. In this definition, a *conceptualisation* is a shared view of the relevant concepts in a domain, such as components, functions, requirements, failures, causes, controls, or evidence. A *specification* records those concepts and their relationships in a structured form. The word *explicit* means that the concepts and relations are not left implicit in human interpretation, but are stated in a form that can be inspected, reused, and, where appropriate, processed by software.

Guarino and colleagues later sharpened this view by treating computational ontologies as formal artefacts used to model relevant entities and relations in a system of interest, while also emphasising shared conceptualisation and machine-readable representation [14]. A computational ontology is therefore an ontology designed for use by computational systems. It does not only describe concepts for human readers; it represents them in a sufficiently formal way that software can store, query, check, or reason over them. Noy and McGuinness place the practical engineering value clearly: an ontology defines the data and structure that other software applications can use, and enables formal analysis, reuse, and extension of domain knowledge [27]. In engineering terms, ontologies are useful when teams need more than a notation. They need shared, inspectable meanings for the concepts and relations that recur across modelling tasks.

Knowledge graphs and related graph-based representations extend this value into settings where multiple heterogeneous datasets, models, and identifiers must be connected. Hogan et al. describe knowledge graphs as graph-based structures designed to support schema, identity, validation, and querying across diverse data [17]. In this context, *schema* means the structural pattern that describes which kinds of entities and relations exist. *Identity* concerns whether two references in different sources point to the same real-world or engineering object. *Validation* concerns checking whether the graph satisfies expected rules or constraints. *Querying* concerns retrieving connected information by following graph relations. These capabilities matter in Digital Engineering because relevant information is often distributed across system models, requirements tools, simulation assets, test repositories, configuration records, and operational data sources.

Graph-based representations can make such connections explicit by representing engineering objects as nodes and their relationships as edges. For example, a requirement, function, component, simulation result, and test report can each be represented as separate nodes. Edges can then show that a component realises a function, that the function satisfies a requirement, that a simulation evaluates a parameter of the component, and that a test provides evidence for the requirement. This turns a chain of hidden or document-based assumptions into a queryable structure that can be inspected and followed.

The literature suggests several ways in which semantic representations can support model integrity. First, they can clarify *meaning*. If concepts such as function, component, interface, requirement, evidence item, analysis result, or operational state are explicitly represented, then later users and tools have a better basis for interpreting relations consistently [14, 7]. Second, they can expose *relationships and dependencies*. Rather than storing traceability in isolated pairwise links, graph and ontology approaches can reveal larger dependency structures. For example, a query may identify not only which component is linked to a requirement, but also which functions, assumptions, analyses, and evidence items are indirectly affected when that component changes [15, 8]. Third, they can support *reuse and integration* across heterogeneous sources by providing a layer of semantic alignment above local schemas or tool formats [7, 15]. This means that different tools may keep their own internal data structures, while their concepts are mapped to a shared meaning layer. For instance, one tool may use the term *part*, another may use *component*, and a third may use *item*; semantic alignment helps specify whether these terms refer to the same kind of engineering concept in a particular context.

Fourth, semantic representations can support *checking and verification*. Checking means inspecting whether expected relations exist or whether inconsistent patterns appear in the connected data. Verification, in this context, concerns whether the connected engineering information satisfies defined rules or constraints. Dunbar et al. show that ontology-aligned graph data can be used for higher-level verification using open-world and closed-world views together with graph algorithms, providing a tool-agnostic means of analysing connected engineering data [8]. Fifth, semantic representations can support *evidence and*

digital thread queries, since graph structures are well suited to traversing linked lifecycle information [15, 7].

These advantages are directly relevant to model integrity. Reliability benefits when model meanings are explicit and inspectable. Traceability benefits when relations are queryable rather than buried in disconnected sources. Currency benefits when dependencies can be traversed to assess change impact. Consistency benefits when shared vocabularies and rules constrain how concepts are related. Usability benefits when downstream tasks can retrieve contextual information without relying on manual repository archaeology.

However, semantic representations are not cost free. Ontology development requires agreement on concepts, boundaries, and governance. It introduces extra modelling effort and ongoing maintenance responsibilities. Domain-specific ontologies may be difficult to generalise; overly ambitious ontological schemes risk becoming hard to apply consistently; and the benefits may not justify the overhead for every project or subsystem [27, 14]. There is also a risk of over-engineering: an ontology that is formally elegant but poorly aligned with the actual engineering workflow may add complexity without improving practice. For this reason, the thesis treats ontologies and graph-based representations as enabling methods that can support model integrity when the problem requires explicit semantics, integration, and queryable dependency structures. Table 2.4 summarises the main semantic methods considered in this thesis.

Table 2.4: Semantic representations, ontologies, and knowledge graphs for model integrity

Method	**Contribution**	**Example use**	**Limitation**
Ontology	Defines concepts, properties, and relations explicitly	Clarifying what counts as a function, component, failure, evidence item, or verification relation	Requires modelling effort, agreement, and governance
Knowledge graph	Connects heterogeneous entities and relations in queryable graph form	Traversing requirement–function–component–analysis–evidence chains	Quality depends on identifiers, mapping quality, and maintenance
Semantic mapping	Aligns local tool schemas and vocabularies to shared meaning	Relating model elements from different repositories or disciplines	Mapping work can be brittle and domain-specific
Rule/query layer	Adds validation, reasoning, and retrieval over connected representations	Detecting missing links, inconsistent patterns, or change-impact candidates	Rules need maintenance and can be incomplete with respect to tacit engineering judgement

2.6 Artificial Intelligence as a Supporting Method

Artificial intelligence is highly relevant to the model integrity challenge of Digital Engineering. Recent developments in large language models (LLMs) make AI a promising means of addressing several limitations identified in Table 2.1 and Table 2.4. In particular, AI may help engineers work with heterogeneous sources, extract structure from text, suggest mappings

between concepts, and review connected information more quickly. At the same time, AI is not treated in this thesis as an independent source of engineering truth. Its value depends on how it is bounded by structured model information, semantic guidance, traceable evidence, and human review.

Current literature indicates several practical tasks for which LLMs can offer support. These include summarisation of large artefact sets, classification of requirements or issues, extraction of structured information from text, generation of candidate links or candidate explanations, review support, and the drafting of intermediate artefacts that still require expert validation [5, 9]. In requirements-related studies, for example, LLMs have been shown to assist in reformulating textual requirements and assessing their adherence to quality criteria, with promising but not definitive results [9]. This directly relates to limitations in Table 2.4: AI can help propose classifications, candidate semantic mappings, or missing-link candidates, while the ontology or knowledge graph provides the structure against which those suggestions can be checked.

The literature is equally clear that risks remain substantial. Hallucination surveys describe a persistent tendency for LLMs to generate plausible but nonfactual or unsupported content [18]. Ferrari and Spoletini argue that concerns around correctness, fairness, and trustworthiness remain central when LLMs are used for requirements-engineering tasks [11]. In Digital Engineering contexts, these concerns translate into familiar integrity problems. Unsupported claims weaken reliability. Missing provenance weakens traceability. Overconfident explanations obscure uncertainty. Partial context can cause a model to produce answers that appear coherent while silently omitting critical dependencies.

For these reasons, the most credible role for AI in the model integrity challenge of Digital Engineering is *bounded support*. AI can help with extraction, classification, summarisation, candidate generation, mapping support, and explanation support, but it should do so within a structure supplied by the engineering environment rather than as an unconstrained generator of engineering truth. Buchmann et al. explicitly frame LLM relevance in relation to semantics-driven systems engineering, that is, in relation to engineered knowledge structures rather than free-form generation alone [5]. This aligns closely with the framework logic pursued by the thesis.

Three forms of bounding are especially important. First, AI should be grounded in *structured model context*. Model elements, identifiers, relationships, and parser outputs should constrain what the model sees and what it is asked to do. This is consistent with the broader literature on using structure and semantics to support model-based and semantics-driven engineering [5, 7]. Second, AI should be informed by *semantic guidance*. Ontologies, vocabularies, and graph context can reduce ambiguity and help place candidate outputs in an explicit meaning structure [5, 17]. Third, AI should remain tied to *retrieved evidence and reviewable provenance*. This means that generated outputs should be connected to the model elements, ontology concepts, sources, assumptions, or evidence items that support them, so that engineers can inspect why an output was produced and whether it is acceptable.

Retrieval-augmented generation (RAG) is relevant to this third form of bounding. In RAG, a language model is provided with retrieved external information, such as model fragments, ontology nodes, requirements, or evidence records, before generating an answer [24]. This gives the model a more specific basis for its output than relying only on its learned parameters. However, retrieval does not guarantee correctness. The retrieved information may be incomplete, the model may combine it incorrectly, or the final answer may still require domain interpretation. Therefore, human review remains necessary.

The combination of AI with semantic representations and ontologies is therefore especially promising for model integrity in Digital Engineering. AI may increase the speed with which engineers can inspect information, propose mappings, identify missing links, generate can-

didate explanations, or update analysis artefacts after model changes. Ontologies and knowledge graphs can provide the meaning structure and traceability context that make such AI outputs easier to constrain and review. The thesis investigates this promise by positioning AI as a supporting method within a broader framework, rather than as a replacement for modelling discipline or engineering judgement.

Table 2.5 summarises the AI-supported tasks considered in this thesis, together with their possible contribution, risk, and required control.

Table 2.5: Role of AI as a supporting method

AI-supported task	Possible contribution	Risk	Required control
Extraction	Pulling candidate entities, properties, or relations from text and mixed artefacts	Omission or fabricated extraction	Structured input context and human verification
Classification	Sorting requirements, issues, evidence, or change items into useful categories	Misclassification caused by limited context	Clear schema, examples, and traceable outputs
Summarisation	Condensing large artefact sets for review and orientation	Loss of nuance or unsupported compression	Source retrieval and review against originals
Candidate generation	Suggesting links, failure modes, causes, effects, or rationale text	Hallucinated content presented as fact	Evidence binding, constrained prompts, expert approval
Explanation	Producing accessible descriptions of model content or reasoning paths	Overconfident explanations that exceed evidence	Citation to retrieved context and uncertainty-aware use
Review and revision support	Flagging incompleteness, ambiguity, or update candidates	False confidence and missed defects	Human-in-the-loop judgement and explicit acceptance criteria

2.7 Research Gap

The literature reviewed in this chapter supports a clear overall argument. Digital Engineering aims to move engineering practice beyond disconnected documents toward lifecycle-connected digital models and data that can support decision-making across lifecycle stages such as development, verification, operation, and sustainment [38, 40, 34]. MBSE provides the main engineering foundation for this shift by representing requirements, structure, behaviour, interfaces, parameters, and verification relations in model form [10, 26]. Descriptive notations, especially SysML, give this information a common representational structure, and recent developments in SysML v2, KerML, and standard APIs improve precision and interoperability potential [28, 30, 29, 31]. However, the literature also makes clear that models and

notations alone do not guarantee that engineering information remains reliable, traceable, current, consistent, or usable after changes [21, 6, 25].

Semantic representations and ontologies address part of this gap by making concepts, relations, and dependencies explicit in machine-interpretable form, while graph-based representations make heterogeneous engineering information more queryable and traversable [13, 14, 17, 7, 8]. Artificial intelligence can support selected information-processing tasks, especially where engineering context is partly textual or heterogeneous. Current literature nevertheless shows that trustworthiness depends on grounding, structure, and review rather than on unconstrained generation [5, 11, 18, 24]. FMEA is a useful demonstration surface because it is strongly information-dependent and makes the practical consequences of weak model integrity visible [22, 37, 35].

The research gap follows from the way these areas are typically discussed. The literature contains many useful methods, but often in separation: Digital Engineering vision and policy; MBSE methods; modelling languages; ontology-aligned integration; graph-based verification; AI support for selected tasks; and FMEA automation or augmentation. What is less developed is a practical framework that organises these methods around the problem of *improving model integrity* in Digital Engineering environments and preserving the usefulness of downstream engineering analysis after source models change. This gap is especially important for real engineering work, where the challenge is rarely the absence of modelling notation or isolated automation, but the difficulty of keeping connected engineering information reliable, traceable, current, consistent, and usable over time.

The next chapter addresses this gap by developing a framework for improving model integrity in Digital Engineering environments. The framework combines descriptive modelling, semantic representation, evidence-oriented traceability, and bounded AI support. It is then demonstrated and evaluated through an FMEA case study, not because FMEA is the thesis goal, but because FMEA provides a practical and audit-friendly way of observing how model integrity affects downstream engineering analysis.

Chapter 3

Case Study

3.1 Introduction

This chapter introduces Failure Mode and Effects Analysis (FMEA) as the case study used in this thesis. The purpose of the chapter is not to make FMEA the main research goal, but to explain why FMEA is a suitable demonstration task for evaluating a framework that aims to improve model integrity in Digital Engineering environments. Chapter 2 argued that Digital Engineering depends on connected, reliable, traceable, current, consistent, and usable model information. FMEA is a useful case study because it depends strongly on exactly those properties. A credible FMEA requires information about system items, functions, interfaces, operating conditions, possible failure modes, causes, effects, controls, assumptions, evidence, and risk priorities. If this information is missing, stale, inconsistent, or weakly linked, the weakness becomes visible in the FMEA table.

IEC 60812:2018 describes FMEA as a systematic method for establishing how items or processes might fail to perform their function so that treatments can be identified. The standard also covers the failure modes, effects and criticality analysis variant, commonly abbreviated as FMECA, and explains how such analyses are planned, performed, documented, and maintained [22]. MIL-STD-1629A similarly defines FMEA as a procedure by which each potential failure mode in a system is analysed to determine its effect on the system and to classify each failure mode according to severity [37]. Across these standards, FMEA can be understood as a structured engineering method for asking a simple but powerful question: *what can fail, why can it fail, what happens if it fails, how serious is it, how likely is it, can it be detected, and what should be done about it?*

This chapter is organised as follows. Section 3.2 introduces the background and purpose of FMEA. Section 3.3 discusses different types and variants of FMEA. Section 3.4 reviews the main standards and guidance documents relevant to this thesis. Section 3.5 presents the seven-step FMEA process used in modern automotive guidance and relates it to the needs of Digital Engineering. Section 3.6 explains common scoring formulas and risk-prioritisation approaches. Section 3.7 reviews developments in automated, model-based, ontology-supported, and AI-supported FMEA. Section 3.8 explains why FMEA is suitable for demonstrating the proposed model integrity framework. The chapter ends by summarising the role of FMEA in the remainder of the thesis.

3.2 Background and Purpose of FMEA

FMEA is one of the most established methods in reliability engineering, safety engineering, and quality engineering. Its origins are commonly associated with military and aerospace reliability work, where engineers needed a systematic way to identify potential failures before they occurred in operation. MIL-STD-1629A formalised procedures for performing Failure Mode, Effects, and Criticality Analysis and emphasised that FMECA should support design decisions from concept through development [37]. The standard also states that the method should be initiated early, as preliminary design information becomes available, and extended to lower levels as more detailed information becomes known [37]. This early-use characteristic is important: FMEA is not only a documentation exercise after a design is finished. It is intended to support design improvement while changes are still possible.

The basic logic of FMEA is inductive. It starts from a lower-level item, function, process step, or component, asks how that element may fail, and then reasons forward to the local, next-higher-level, and system-level effects. This makes FMEA different from deductive methods such as Fault Tree Analysis (FTA), which typically start from an undesired top event and reason backwards toward possible causes. FMEA is therefore particularly useful when the analyst wants to review many system elements systematically and identify how individual failures may affect higher-level performance, safety, reliability, maintainability, or customer value.

A typical FMEA row contains at least the following information: the item or function being analysed, the potential failure mode, the effect of that failure, the possible causes, current prevention and detection controls, severity, occurrence, detection, recommended actions, and responsibility for those actions. Different industries use different worksheet formats, but the underlying reasoning remains similar [22, 2]. The method converts engineering understanding into a structured risk table that can be reviewed, prioritised, updated, and used for decision-making.

FMEA has three major purposes in the context of this thesis. First, it supports *failure identification*. It helps engineers systematically identify how a product, system, function, or process might fail. Second, it supports *risk prioritisation*. It helps teams decide which failure modes need attention based on severity, likelihood, detectability, or criticality. Third, it supports *learning and traceability*. A good FMEA records the reasoning behind risk decisions and links them to functions, components, controls, tests, assumptions, or design changes. This third purpose is especially relevant for Digital Engineering because the value of an FMEA depends not only on the table itself, but on whether its entries can be traced back to the engineering information that justifies them.

A weak FMEA can still look complete on the surface. It may contain rows, scores, and recommended actions, but those rows may be generic, duplicated, unsupported, or disconnected from the current design. This is why FMEA is suitable as a case study for model integrity. If the underlying model information is reliable, traceable, current, consistent, and usable, then FMEA generation and maintenance should become more structured and defensible. If the model information is weak, the FMEA will expose the weakness through missing failure modes, unclear causes, inconsistent effects, unsupported scores, and difficulty updating the table after model changes.

3.3 Types and Variants of FMEA

FMEA is not one single fixed worksheet. It is a family of related analysis methods that can be adapted to different lifecycle stages, levels of abstraction, and application domains. IEC 60812:2018 explicitly states that FMEA is applicable to hardware, software, processes, human actions, and interfaces in any combination [22]. This broad applicability explains why many variants exist.

A common distinction is between *functional FMEA*, *design FMEA*, and *process FMEA*. Functional FMEA is usually performed early in development, when the system functions are known but the detailed design may not yet be fixed. It asks how required functions may fail, for example by being absent, degraded, intermittent, late, excessive, or unintended. Functional FMEA is useful in early systems engineering because it can be applied before every component is selected. Its limitation is that it may remain abstract if the functional model is not connected to later physical design information.

Design FMEA, often abbreviated as DFMEA, focuses on product or system design. It analyses how design elements may fail to fulfil their intended function and how those failures affect users, neighbouring subsystems, or the whole system. DFMEA is especially relevant when functions have been allocated to components, interfaces, or subsystems. It therefore depends heavily on a clear connection between functions, structures, interfaces, design assumptions, and verification evidence. If these links are weak, the DFMEA may become generic or difficult to justify.

Process FMEA, or PFMEA, focuses on manufacturing, assembly, service, or operational processes. Instead of asking only how a product design may fail, it asks how a process step may fail to produce the required output. Typical process failure modes include incorrect assembly, missing operation, wrong parameter, insufficient inspection, contamination, damage, wrong material, or incorrect sequence. PFMEA is important because a design may be technically sound but still fail in practice if the process used to realise it is weak.

FMECA extends FMEA by adding a criticality analysis. In FMEA, risk prioritisation is often qualitative or semi-quantitative. In FMECA, the analysis may include more explicit criticality ranking based on severity and probability, or on failure-rate information when such data are available [37, 22]. MIL-STD-1629A includes quantitative criticality formulas for failure mode criticality and item criticality [37]. This makes FMECA more data-intensive than a basic FMEA and more dependent on reliable failure-rate data, mission profiles, severity classifications, and failure-mode ratios.

Other variants include software FMEA, service FMEA, machinery FMEA, interface FMEA, and failure modes, effects, and diagnostic analysis (FMEDA). Software FMEA considers failure modes related to software behaviour, such as incorrect logic, timing issues, data handling problems, unsafe state transitions, or failed exception handling. FMEDA is particularly used in functional safety contexts and adds diagnostic coverage and quantitative failure-rate considerations. These variants show that FMEA is flexible, but also that its quality depends strongly on choosing the right level of analysis and the right source information.

Table 3.1: Common FMEA types and their relevance to this thesis

FMEA type	Main focus	Typical input information	Relevance to model integrity
Functional FMEA	Failure of required system functions	Functional architecture, operating modes, mission phases, functional requirements	Reveals whether functions are complete, clearly named, and linked to system-level effects
Design FMEA	Failure of design elements to fulfil intended functions	Requirements, components, interfaces, function allocation, design assumptions, verification plans	Requires traceability between requirements, functions, components, interfaces, and evidence
Process FMEA	Failure of manufacturing, assembly, service, or operational process steps	Process flow, process parameters, controls, inspections, quality requirements	Depends on current process knowledge and consistent links between process steps and product requirements
FMECA	FMEA with explicit criticality analysis	FMEA content plus severity classes, probability or failure-rate data, mission phase, criticality assumptions	Requires stronger quantitative evidence and clear assumptions for defensible prioritisation
Software FMEA	Failure of software behaviour or logic	Software functions, states, interfaces, timing, data flows, requirements, test evidence	Requires careful treatment of behavioural and interface information, not only hardware structure
FMEDA	Failure modes, effects, and diagnostic analysis	Component failure rates, diagnostic coverage, safe/dangerous failure classification	Depends on reliable failure data and explicit diagnostic assumptions

For this thesis, the most relevant forms are functional FMEA and design-oriented FMEA. The framework proposed later uses FMEA as a downstream engineering analysis that depends on modelled functions, components, interfaces, assumptions, and evidence. The case study therefore does not aim to cover every possible industrial FMEA variant. Instead, it uses FMEA as a representative analysis task through which the effect of model integrity can be observed.

3.4 Standards and Guidance for FMEA

Several standards and handbooks shape FMEA practice. They differ in domain, terminology, level of detail, and risk-prioritisation method, but they share a common concern: failure analysis should be systematic, documented, and connected to design or process improvement.

MIL-STD-1629A is historically important because it formalised procedures for FMECA in military and aerospace contexts. It presents FMECA as an essential design function and emphasises that the method should be iterative, tailored to the programme, and useful for decision-making [37]. The standard distinguishes FMEA from criticality analysis and provides procedures for identifying failure modes, classifying effects by severity, identifying single failure points, and ranking failures [37]. It also provides quantitative criticality calculations, discussed later in this chapter.

IEC 60812:2018 is a more recent and generic international standard. It covers FMEA and FMECA across hardware, software, processes, human action, and interfaces [22]. It explains how FMEA is planned, performed, documented, and maintained, and it explicitly recognises different reporting formats, tailoring for different applications, alternative ways of calculating risk priority numbers, and criticality matrix approaches [22]. This makes IEC 60812 especially useful for this thesis because it treats FMEA as a general dependability analysis method rather than a method restricted to one industry.

The AIAG and VDA FMEA Handbook is influential in automotive quality and product-development practice. It harmonised earlier automotive FMEA guidance and introduced a seven-step approach: planning and preparation, structure analysis, function analysis, failure analysis, risk analysis, optimisation, and results documentation [2]. The handbook is especially relevant because it makes the analysis process more explicit and connects FMEA work to structured system, function, failure, and risk reasoning. It also shifts emphasis from the older Risk Priority Number-only approach toward Action Priority, which is intended to guide whether further action is needed based on severity, occurrence, and detection combinations [2].

SAE guidance is also relevant, particularly in aerospace and automotive contexts. SAE ARP5580 provides recommended FMEA practices for non-automobile applications, while SAE J1739 has historically provided guidance for design and process FMEA in automotive and related industries [32, 33]. Domain-specific safety standards may also require or recommend FMEA-like analyses as part of a broader safety case or reliability programme.

Table 3.2: Selected FMEA standards and guidance documents

Source	Domain emphasis	Main contribution	Relevance to this thesis
MIL-STD-1629A	Military and aerospace reliability	Provides procedures for FMECA, including severity classification, criticality analysis, and early design use	Shows the historical, structured, and quantitative roots of FMEA/FMECA
IEC 60812:2018	Generic dependability analysis	Explains planning, performance, documentation, maintenance, tailoring, RPN alternatives, and criticality matrices	Provides a broad standard basis for treating FMEA as a general engineering analysis method
AIAG–VDA FMEA Handbook	Automotive product and process quality	Introduces the seven-step approach and Action Priority logic	Useful because it makes structure, function, failure, risk, optimisation, and documentation steps explicit
SAE ARP5580 / SAE J1739	Aerospace, automotive, and related applications	Provides recommended practices and domain-oriented FMEA guidance	Useful as supporting guidance for adapting FMEA beyond one industrial domain

A standards-based view is important for the thesis because it prevents FMEA from being treated as a loose brainstorming exercise. Standards show that FMEA requires preparation, scope definition, system structuring, function definition, failure reasoning, risk prioritisation, action tracking, documentation, and maintenance. These requirements directly connect FMEA to model integrity: if the model cannot provide current functions, item structures, interfaces, evidence, and change information, then the FMEA becomes harder to perform and maintain.

3.5 The Seven-Step FMEA Process

Modern automotive FMEA practice is often organised around the seven-step approach of the AIAG–VDA FMEA Handbook [2]. Although this handbook was developed for automotive practice, the process is useful more broadly because it separates FMEA into a sequence of reasoning steps. These steps are especially relevant to this thesis because they make clear which types of engineering information are required at each stage.

The first step is *planning and preparation*. The team defines the purpose, scope, boundaries, assumptions, interfaces, available information, analysis level, and team responsibilities. In Digital Engineering terms, this step asks which part of the model and which evidence sources are authoritative for the analysis. Poor scope definition can lead to missing failure modes, duplicated work, or irrelevant analysis rows.

The second step is *structure analysis*. The system, product, or process is decomposed into relevant elements. For a design FMEA, this may include systems, subsystems, components, interfaces, and design features. For a process FMEA, it may include process steps, worksta-

tions, operations, and process inputs. This step is closely related to model structure. If the system model does not clearly represent decomposition and interfaces, the FMEA structure becomes ambiguous.

The third step is *function analysis*. The team identifies what each element is intended to do. Functions may be stated at different levels, from system-level functions to component-level functions or process-step functions. This is one of the most important steps for model integrity because failure modes are normally derived from functions. A missing, vague, or duplicated function can directly lead to a missing, vague, or duplicated failure mode.

The fourth step is *failure analysis*. The team identifies potential failure modes, failure effects, and failure causes. In many FMEA approaches, a failure mode is the way in which a function fails, the effect is the consequence of that failure, and the cause is the reason the failure mode might occur. This step depends strongly on clear links between functions, physical realisation, operating conditions, assumptions, historical knowledge, and evidence.

The fifth step is *risk analysis*. The team evaluates the risk associated with each failure chain. Traditional FMEA often uses severity, occurrence, and detection ratings, while FMECA may use criticality analysis or criticality matrices. AIAG–VDA uses Action Priority tables rather than relying only on RPN [2]. Regardless of the exact scoring method, risk analysis requires justified ratings. Unsupported scores are a major integrity weakness because they make the FMEA appear quantitative without providing a defensible basis.

The sixth step is *optimisation*. The team defines and tracks actions to reduce risk, usually by reducing occurrence, improving detection, or changing the design so that severity is reduced where possible. Optimisation makes the FMEA active: the table is not only a record of risk but also a driver for design and process improvement. This step creates a need for traceability between failure modes, actions, responsibilities, due dates, design changes, verification activities, and residual risk.

The seventh step is *results documentation*. The team documents the analysis, decisions, assumptions, ratings, actions, and remaining risks. Documentation should make the analysis reviewable and maintainable. In a Digital Engineering context, this does not only mean producing a static report. It also means preserving links to model elements, evidence, assumptions, and change history so that the FMEA can be updated when the system changes.

Table 3.3: Seven-step FMEA process and required model information

Step	Purpose	Needed engineering information	Model integrity dependency
1. Planning and preparation	Define purpose, scope, boundaries, assumptions, team, and inputs	System boundary, lifecycle stage, assumptions, applicable standards, authoritative sources	Requires reliable scope and clear source authority
2. Structure analysis	Decompose system, product, or process into relevant elements	System breakdown, components, interfaces, process steps, hierarchy	Requires current and consistent structure
3. Function analysis	Identify intended functions of each element	Functions, requirements, operating conditions, performance expectations	Requires clear function definitions and traceability to requirements
4. Failure analysis	Identify failure modes, effects, and causes	Function-failure relations, component behaviour, causal knowledge, operating context	Requires semantic clarity and evidence-supported failure reasoning
5. Risk analysis	Evaluate severity, occurrence, detection, or criticality	Rating criteria, historical data, controls, detection methods, assumptions	Requires justified and traceable scores
6. Optimisation	Define and track actions to reduce risk	Design changes, controls, responsibilities, verification actions, residual risk	Requires change traceability and action status control
7. Results documentation	Document results, decisions, assumptions, and remaining risks	FMEA table, rationale, evidence links, version information, review records	Requires maintainable evidence chains and update history

The seven-step process shows why FMEA is a demanding downstream analysis. Each step needs different but connected information. A model that contains components but not functions cannot fully support failure analysis. A model that contains functions but not evidence cannot fully support risk justification. A model that contains analysis results but not version history cannot fully support maintenance after change. The process therefore provides a useful structure for evaluating whether a model integrity framework actually helps downstream engineering work.

3.6 Risk Evaluation, Formulas, and Prioritisation

FMEA is often associated with numerical risk scoring. However, not all FMEA standards use the same scoring logic, and numerical results should be interpreted cautiously. The purpose of scoring is not to create false precision, but to help prioritise attention and justify improvement actions.

The most familiar traditional formula is the Risk Priority Number (RPN):

$$RPN = S \times O \times D \tag{3.1}$$

where S is severity, O is occurrence, and D is detection. Severity usually represents the seriousness of the failure effect. Occurrence represents the estimated likelihood of the cause or failure mode. Detection represents the likelihood that existing controls will detect the cause or failure before the effect reaches the user or system. In many industrial FMEA practices these factors are rated on a scale from 1 to 10, although the exact interpretation depends on the organisation or standard used.

The RPN is easy to calculate and easy to communicate. This is why it has been widely used. However, it has well-known limitations. The ratings are often ordinal scores, meaning that the difference between rating 2 and 3 is not necessarily the same as the difference between rating 8 and 9. Multiplying ordinal scores can therefore produce rankings that look mathematically precise but may not reflect real risk differences. Different combinations of severity, occurrence, and detection can also produce the same RPN. For example:

$$S = 10,\ O = 2,\ D = 2 \Rightarrow RPN = 40 \tag{3.2}$$

$$S = 4,\ O = 5,\ D = 2 \Rightarrow RPN = 40 \tag{3.3}$$

Both cases produce the same RPN, but the first may deserve more urgent attention because the severity is much higher. This is one reason why modern guidance has moved away from relying only on RPN. The AIAG–VDA Handbook uses Action Priority (AP), where combinations of severity, occurrence, and detection are mapped to high, medium, or low action priority [2]. AP is not a simple multiplication formula. It is a decision table intended to guide whether improvement actions are needed.

Another common approach is to use a risk matrix. A risk matrix evaluates risk by combining severity and probability or occurrence categories. A simplified expression is:

$$Risk = f(S, P) \tag{3.4}$$

where S is severity and P is probability or occurrence class. The output is usually not a precise number but a risk region, such as low, medium, high, or unacceptable. Risk matrices are common in safety and reliability practice because they are understandable and useful for prioritisation. Their limitation is that they can hide uncertainty and may be sensitive to how categories are defined.

FMECA adds more explicit criticality analysis when suitable data are available. MIL-STD-1629A defines a failure mode criticality number, commonly written as C_m, as the portion of item criticality associated with a particular failure mode under a particular severity classification [37]. The standard gives the following form:

$$C_m = \beta \alpha \lambda_p t \tag{3.5}$$

where β is the conditional probability of mission loss given that the failure mode has occurred, α is the failure mode ratio, λ_p is the part failure rate, and t is the duration of the applicable mission phase or operating time [37]. The item criticality number is then calculated by summing the relevant failure mode criticality numbers:

$$C_r = \sum_{n=1}^{j} (C_m)_n \tag{3.6}$$

where j is the number of failure modes in the item under the relevant severity classification [37]. These formulas are more data-intensive than RPN. They require failure-rate estimates, failure-mode ratios, mission duration, severity classification, and conditional probability assumptions. They can therefore support more quantitative reasoning, but only when the underlying data are credible.

Table 3.4: Common FMEA and FMECA risk-prioritisation approaches

Approach	Typical expression	Strength	Limitation
Risk Priority Number	$RPN = S \times O \times D$	Simple and familiar; supports quick ranking	Can create misleading precision and rank reversals because ordinal ratings are multiplied
Action Priority	Decision table based on S, O, and D combinations	Emphasises need for action rather than only numerical ranking	Requires use of handbook-specific AP tables and judgement
Risk matrix	$Risk = f(S, P)$	Easy to communicate; useful for severity-probability regions	Sensitive to category definitions and may hide uncertainty
Failure mode criticality	$C_m = \beta \alpha \lambda_p t$	Uses failure-rate and mission information when available	Requires credible quantitative data and assumptions
Item criticality	$C_r = \sum (C_m)_n$	Aggregates criticality across failure modes of an item	Depends on completeness of failure modes and correct classification

For this thesis, the formulas are important for a specific reason: every risk score depends on information integrity. Severity depends on correct effect interpretation. Occurrence depends on credible failure knowledge or assumptions. Detection depends on the existence and performance of controls. Criticality depends on failure rates, mission duration, failure-mode ratios, and conditional probability estimates. If these inputs are not traceable, current, or evidence-supported, the resulting score may be difficult to defend. Therefore, risk scoring does not remove the need for model integrity; it increases the need for it.

3.7 Developments in Automated FMEA

FMEA is mature, but it remains labour-intensive. It requires cross-disciplinary expertise, systematic review, careful documentation, and repeated updates after design or process changes. This creates a natural interest in automation. The literature on automated FMEA can be grouped into several broad developments: rule-based and expert-system support, model-based support, ontology-supported FMEA, data-driven and machine-learning support, and LLM-supported or hybrid approaches.

Early automation work often focused on rule-based support. In such approaches, domain experts encode rules that help identify potential failure modes, effects, or recommended actions. Rule-based systems can improve consistency when the domain is narrow and well understood. However, they are brittle when the design changes, when the domain expands, or when failure reasoning depends on tacit engineering judgement. The knowledge-engineering effort can also become high.

A second development is model-based FMEA. Here, failure reasoning is connected to system models, function models, architecture models, or process models. Huang et al. discuss MBSE-assisted FMEA and identify both opportunities and challenges in connecting model-based systems engineering to FMEA practice [20]. Later work on FMEA and MBSE integration argues that model-based representations can support the generation or maintenance of FMEA content by making the system structure and function information more explicit [19]. Korsunovs et al. similarly investigate automatic FMEA generation in a model-based systems engineering approach for robotic manufacturing process modelling [23]. These works are important because they directly connect FMEA to the model integrity problem. If the model contains explicit functions, structures, and relations, then parts of the FMEA can be derived or checked more systematically. If the model lacks those relations, automation becomes shallow.

A third development is ontology-supported FMEA. Ontologies can define the meaning of failure-related concepts such as function, failure mode, cause, effect, control, detection method, and evidence. Hodkiewicz et al. propose an ontology for reasoning over engineering textual data stored in FMEA spreadsheet tables, showing how semantic structure can support interpretation and reuse of FMEA knowledge [16]. Younus et al. discuss ontology-supported FMEA in advanced systems engineering and emphasise the role of formalised system knowledge, semantic reasoning, traceability, and cross-domain interoperability [41]. Ontology-supported FMEA is relevant to this thesis because it addresses a key weakness of traditional FMEA tables: they often contain valuable engineering knowledge but in a form that is difficult to query, reuse, or link to system models.

A fourth development is data-driven FMEA. Machine learning and data mining can be used to analyse historical failure data, maintenance records, quality reports, incident databases, or operational logs. Such methods may help identify recurring failure patterns, estimate likelihood, classify failure modes, or support prioritisation. Spreafico et al. review many developments and adaptations of FMEA/FMECA, including attempts to improve prioritisation and address limitations of traditional practice [35]. Data-driven methods are promising where enough reliable data exist, but they depend strongly on data quality, consistent terminology, and contextual interpretation. A model may detect a statistical pattern, but engineers still need to know whether the pattern is relevant to the current system design and operating context.

A fifth and recent development is LLM-supported FMEA. Large language models can assist with summarisation, extraction, classification, candidate failure mode generation, explanation, and review support. Recent review work on AI- and ontology-based enhancements to FMEA argues that AI, natural language processing, ontologies, and hybrid approaches can make FMEA more dynamic, model-integrated, and semantically enriched, while also identifying challenges such as data quality, explainability, standardisation, and interdisciplinary adoption [42]. In this thesis, LLMs are therefore treated as supporting tools, not as autonomous safety or reliability engineers. They may suggest candidate failure modes, causes, or effects, but such suggestions require grounding in model structure, semantic context, retrieved evidence, and human review.

Table 3.5: Development of automated and model-supported FMEA

Development	Main idea	Potential contribution	Main limitation
Rule-based support	Encode expert rules for failure reasoning	Can improve consistency in narrow domains	Brittle when system context changes or tacit judgement is needed
Model-based FMEA	Use system, function, architecture, or process models as FMEA input	Can derive or check parts of the FMEA from explicit model relations	Depends on model completeness, traceability, and modelling discipline
Ontology-supported FMEA	Represent FMEA concepts and relations semantically	Improves meaning, reuse, queryability, and cross-domain alignment	Requires ontology development, governance, and mapping effort
Data-driven FMEA	Use historical failure, quality, maintenance, or operational data	Can identify patterns and support likelihood or prioritisation estimates	Depends on data quality, context, and consistent terminology
LLM-supported FMEA	Use language models for extraction, summarisation, candidate generation, and review	Can speed up language-heavy tasks and generate candidate content	May hallucinate, omit context, or produce unsupported claims without controls
Hybrid FMEA	Combine models, ontologies, rules, evidence, and AI support	Best aligned with Digital Engineering and model integrity needs	Requires careful workflow design and human review

The direction of the literature is clear: automation is most credible when it is not based on free-form generation alone. Stronger approaches connect FMEA to structured models, semantic representations, evidence, and review mechanisms. This supports the thesis logic. The research contribution is not simply to automate FMEA, but to use FMEA as a case study for examining how model integrity affects downstream analysis and how a framework can support that analysis.

3.8 FMEA and Model Integrity

FMEA exposes model integrity problems because it sits downstream of many kinds of engineering information. It needs the system structure, but structure alone is not enough. It also needs functions, operating conditions, interfaces, causes, effects, controls, ratings, assumptions, and evidence. This makes FMEA a good test of whether Digital Engineering information is genuinely connected and usable.

The reliability dimension of model integrity appears in FMEA when engineers ask whether the analysis is based on trustworthy information. For example, if an occurrence rating is based on unknown or outdated failure data, the score may be unreliable. If a severity rating is based on an unclear interpretation of the system effect, the ranking may be difficult to defend. Reliable FMEA therefore requires clear provenance and status for model information, assumptions, and evidence.

The traceability dimension appears when engineers ask why a row exists and what supports it. A failure mode should be traceable to a function or item. A cause should be traceable to a mechanism, design feature, process step, or assumption. An effect should be traceable through the system hierarchy. A control should be traceable to verification, inspection, detection, or prevention evidence. Without such links, the FMEA becomes a static table whose rationale must be reconstructed manually.

The currency dimension appears when the model changes. If a function is modified, a component is replaced, an interface is redesigned, or a detection control changes, the FMEA may need to be updated. In a weakly connected environment, engineers may not know which FMEA rows are affected. This creates a risk that the FMEA appears complete while reflecting an old version of the system. A model integrity framework should therefore help identify which downstream analysis rows depend on changed model information.

The consistency dimension appears in terminology and relations. Similar functions may be named differently, or different concepts may be given similar names. Failure modes may be duplicated because different analysts use different wording. Causes and effects may be mixed because the level of analysis is unclear. Semantic representations and controlled vocabularies can help reduce these problems by making concepts and relationships explicit.

The usability dimension appears when information exists but cannot be used efficiently. A system model may contain the relevant functions, but if they are not linked to components, operating modes, or evidence, the FMEA team still needs to search manually. Similarly, evidence may exist in reports or spreadsheets, but if it is not connected to the relevant model elements, it cannot easily support scoring or review. In this sense, FMEA does not only require information; it requires information in a usable form.

Table 3.6: How model integrity affects FMEA quality

Model integrity concern	**FMEA dependency**	**Typical failure symptom**	**Consequence for analysis**
Reliability	FMEA ratings and rows depend on trustworthy source information	Unsupported severity, occurrence, or detection scores	Risk prioritisation is difficult to defend
Traceability	Rows should link to functions, components, assumptions, controls, and evidence	Failure modes or causes cannot be traced to model elements	Coverage and rationale become uncertain
Currency	FMEA should reflect the current model and evidence state	Old FMEA rows remain after design or process changes	The table becomes stale and misleading
Consistency	Terminology and levels of analysis should be coherent	Duplicate failure modes, mixed abstraction levels, inconsistent terms	The FMEA becomes hard to compare, query, and maintain
Usability	Model information should be accessible for downstream analysis	Relevant information exists but is scattered or implicit	FMEA creation and update remain manual and slow

This mapping explains why FMEA is used as a demonstration task in the thesis. It is structured enough to evaluate, but rich enough to expose real integrity problems. A framework that improves model integrity should make it easier to identify relevant functions, derive candidate failure modes, trace effects, justify scores, link evidence, and update affected rows

after model changes. Conversely, if the framework cannot support an FMEA, it is unlikely to support more complex downstream engineering analyses.

3.9 Role of FMEA in This Thesis

This thesis does not aim to create a new FMEA standard, replace expert judgement, or fully automate safety analysis. Instead, FMEA is used as a case study for evaluating how model integrity affects downstream engineering work. This distinction is important. The research goal is the development of a framework for improving model integrity in Digital Engineering environments. FMEA is the analysis surface on which the usefulness of that framework is demonstrated.

FMEA is suitable for this role for five reasons. First, it has a recognisable structure. Most engineers can understand an FMEA table because it contains items, functions, failure modes, effects, causes, controls, and actions. Second, it is information-dependent. It requires connected knowledge about the system, not only isolated text. Third, it is sensitive to weak integrity. Missing links, stale assumptions, inconsistent terms, and unsupported evidence quickly degrade FMEA quality. Fourth, it is maintainability-sensitive. When the system model changes, the FMEA should change with it. Fifth, it is auditable. Rows, ratings, causes, effects, and actions can be inspected and challenged.

The case study therefore provides a practical evaluation setting for the proposed framework. The framework can be assessed by asking whether it improves the ability to:

- identify the model elements relevant to FMEA;
- connect functions, components, interfaces, and operating conditions;
- generate or review candidate failure modes in a controlled way;
- link FMEA entries to evidence and assumptions;
- detect missing or weak traceability;
- identify which FMEA rows may be affected by model changes;
- support human review rather than replace engineering judgement.

In this role, FMEA acts like a stress test for model integrity. If the connected model environment is weak, the FMEA process reveals the weakness. If the connected model environment is stronger, the FMEA process should become easier to justify, update, and maintain. That is why FMEA is placed in this chapter as the case study background before the methodology chapter. Chapter 4 can then focus on the proposed framework and its evaluation, while the present chapter provides the necessary FMEA foundation.

3.10 Chapter Summary

This chapter explained the background and relevance of FMEA as the case study for this thesis. FMEA is a structured method for identifying potential failure modes, their causes, their effects, and possible treatments. It has a long history in reliability and safety engineering and is supported by standards and guidance such as MIL-STD-1629A, IEC 60812:2018,

SAE guidance, and the AIAG–VDA FMEA Handbook [37, 22, 32, 2]. Different variants exist, including functional FMEA, design FMEA, process FMEA, software FMEA, FMECA, and FMEDA.

The chapter also explained the seven-step FMEA process and the main risk-prioritisation approaches, including RPN, Action Priority, risk matrices, and FMECA criticality calculations. These methods are useful, but they depend on trustworthy source information. Scores and rankings are only defensible when the underlying functions, causes, effects, controls, assumptions, and evidence are reliable and traceable.

Finally, the chapter reviewed developments in automated FMEA. The literature shows movement from manual worksheets toward rule-based, model-based, ontology-supported, data-driven, AI-supported, and hybrid approaches. These developments support the central argument of the thesis: downstream engineering analysis cannot be maintained well unless the model information that feeds it has sufficient integrity. FMEA is therefore used not as the thesis goal, but as a practical and auditable demonstration task for evaluating a model integrity framework.

Chapter 4

Methodology

4.1 Research Design

This thesis adopts a design-oriented research methodology. The objective is to develop a framework for improving model integrity in Digital Engineering environments and to demonstrate its application through a downstream engineering analysis task. The focus is not on replacing engineering judgement, but on examining how model-based information can be structured, bounded, semantically guided, and used to support traceable and reviewable analysis.

The methodology is based on four main elements: model representation, model parsing, ontology-based reasoning guidance, and AI-supported reasoning. The model representation provides the system-specific engineering information. The model parser extracts and structures relevant information from this model. The ontology provides a domain-agnostic reasoning structure for the selected analysis task, which is FMEA in this thesis. The AI component then uses both the parsed model context and the ontology to support FMEA generation, review, scoring, explanation, and revision.

In this thesis, FMEA is used as the downstream engineering analysis task. It is selected because it depends strongly on connected engineering information, including system items, functions, failure modes, causes, effects, controls, scores, and supporting evidence. FMEA is therefore not treated as the main research goal. Instead, it provides a concrete demonstration task for examining whether the proposed framework can support reliability, traceability, maintainability, and reviewability in downstream analysis.

The methodology is demonstrated using a toy flashlight system represented in a textual SysML v2-style notation. The flashlight model includes requirements, actions, parts, ports, attributes, connections, state behaviour, and requirement allocations. This makes it suitable as a compact case study: it is small enough to remain inspectable, but rich enough to contain the kinds of relationships that are important for model integrity. The full flashlight model is provided in Appendix A. The generated outputs and the evaluation of the demonstration are presented in Chapter 5.

4.2 Framework Overview

The proposed framework combines four main elements: model representation, model parser, ontology, and AI-supported reasoning. These elements are arranged as shown in Figure 4.1. The model parser and the ontology provide two complementary inputs to the AI-supported reasoning module. The parser provides system-specific context from the SysML model, while the ontology provides domain-general reasoning guidance for the particular analysis task to be carried out, namely FMEA.

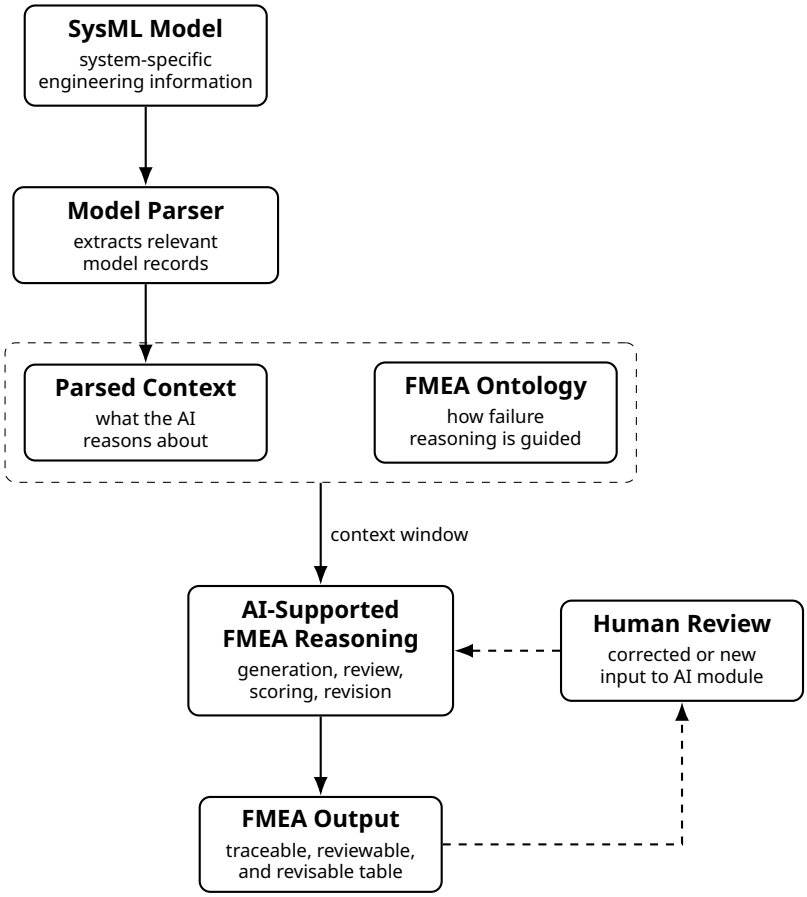


Figure 4.1: Framework overview showing the relationship between model representation, model parsing, ontology-based reasoning guidance, AI-supported reasoning, and downstream analysis output.

The model representation is the SysML v2-style description of the system. The parser extracts bounded model context from this representation. Bounded context means that only the model information relevant to a particular part, function, interface, or group is selected for the reasoning task, instead of sending the whole model to the AI component. The ontology then guides how this selected information should be interpreted in FMEA terms, such as item, function, failure mode, cause, effect, control, evidence, and risk score.

The proposed framework can therefore be understood as follows:

- the model representation provides the system-specific engineering information;

- the model parser extracts bounded and relevant model context;
- the ontology provides domain-agnostic reasoning structure for FMEA;
- the AI component uses both inputs to carry out FMEA-specific reasoning tasks;
- the human reviewer inspects the generated FMEA table and can provide corrected or new input to the AI module.

Table 4.1 summarises the role of each framework component.

Table 4.1: Main components of the proposed model integrity framework

Component	**Role in the proposed framework**	**Contribution to model integrity**
Model representation	Provides the system-specific SysML v2-style engineering information.	Captures system structure, behaviour, requirements, functions, interfaces, and allocations.
Model parser	Extracts and structures relevant context from the SysML model.	Makes model information bounded, explicit, traceable, and usable for downstream reasoning.
Ontology	Provides FMEA concepts, relationships, reasoning rules, and question templates.	Guides how failure-analysis concepts should be understood and related.
AI-supported reasoning	Uses parsed model context and ontology guidance to perform FMEA generation, review, scoring, explanation, and revision.	Assists analysis while remaining grounded in structured model information and semantic guidance.
Human review	Allows the user to inspect, question, correct, and revise the generated FMEA table.	Preserves engineering accountability and supports correction of incomplete or weak results.

4.3 Model Representation

The first element of the framework is the model representation. In this thesis, the system is represented using a textual SysML v2-style model. The model is not claimed to cover every construct in the full SysML v2 language. Instead, it uses a focused subset of SysML-style constructs that are relevant for downstream FMEA reasoning. These constructs describe what the system must satisfy, what it does, what it consists of, how its elements are connected, and which requirements are allocated to which parts or design features.

A SysML *package* is used as a named container for related model elements. In the flashlight model, separate packages are used to organise requirements, actions, parts, and requirement allocations. This grouping is useful because it makes the model easier to navigate and allows the parser to distinguish between different types of information.

An *action tree* describes the functional decomposition of the system. It explains what the system does and how a high-level action is decomposed into lower-level actions. For example, the flashlight has the high-level function of producing directed light. This can be decomposed into actions such as providing DC power, connecting DC power, generating light,

and directing light. These actions are relevant to FMEA because failure modes are usually defined in relation to intended functions.

A *parts tree* describes the structural decomposition of the system. It explains what the system consists of and how parts are nested within larger parts or assemblies. In the flashlight model, the top-level flashlight contains parts such as batteries, switch, lamp, optics, reflector, lens, and structure. This hierarchy is relevant to FMEA because failure modes may occur at component level, subsystem level, or system level.

In this thesis, the term *design feature* is used in a practical sense. It refers to model information associated with a part that may matter for design reasoning, such as a port, attribute, performed action, connection, binding, state behaviour, or requirement allocation. Design features are therefore not separate from parts. Rather, they describe properties, behaviours, or relations of parts that may be relevant for analysis.

The flashlight model uses the following SysML-style constructs as input to the framework:

Table 4.2: SysML-style constructs used as model input for FMEA reasoning

Construct	**Meaning in the model**	**Relevance for FMEA**
`package`	Groups related model elements, such as requirements, actions, parts, or allocations.	Helps locate the type of information being parsed.
`requirement`	States a required capability, constraint, or property of the system.	Supports effects, severity reasoning, and links between failures and requirements.
`action`	Describes a behaviour or function that the system or a part performs.	Supports identification of functions and function-based failure modes.
`part` and `ref part`	Describe system elements and their structural hierarchy.	Define the FMEA item or part being analysed.
`port`	Describes an interaction point through which material, energy, or information may pass.	Supports interface-related failure modes.
Attributes and values	Describe properties such as power, mass, size, or other design parameters.	Support causes, constraints, assumptions, and requirement-related effects.
`connect`, `flow`, and `bind`	Describe relations, interfaces, or parameter dependencies between model elements.	Support interface, propagation, and dependency-related failure reasoning.
`allocate`	Links a requirement, action, or other concern to a part or design feature.	Supports traceability between FMEA rows and model evidence.
States and transitions	Describe possible operating states or control behaviour.	Support failure modes related to incorrect state, transition, or control behaviour.

A simplified example of the type of information used by the framework is shown below. The example is not intended to reproduce the full model, but to illustrate how requirements, actions, parts, and allocations appear in the textual model.

```
package FlashlightRequirements {
    requirement LightPower { /* required illumination performance */ }
}

package FlashlightActions {
    action ProduceDirectedLight {
        action ProvideDCPower;
        action GenerateLight;
        action DirectLight;
    }
}

package FlashlightParts {
    part flashlight {
        part battery;
        part switch;
        part lamp;
        part optics;
        connect switch.output to lamp.input;
        connect lamp.lightOutput to optics.lightInput;
    }
}

package FlashlightAllocations {
    allocate LightPower to lamp;
    allocate DirectLight to optics;
}
```

For FMEA, the SysML model should contain at least four types of information. First, it should identify the items or parts that may fail. Second, it should describe the intended functions of those parts, usually through actions or performed behaviours. Third, it should describe interfaces and dependencies, such as ports, connections, flows, or bindings, because failures often propagate across interfaces. Fourth, it should include requirements, constraints, or allocations that help explain why a failure matters. Without this information, the generated FMEA may still contain plausible rows, but those rows would be less traceable to the model and harder to justify.

The model representation therefore provides the starting point for model integrity support. However, the raw textual model is not directly suitable as AI input in its original form. This is not because SysML lacks semantics. SysML v2 is specifically designed to provide a more precise semantic foundation. The practical problem is different: relevant information for one FMEA row may be distributed across several parts of the textual model, such as the parts tree, action tree, requirement package, and allocation package. A parser is therefore introduced to collect and restructure the relevant model information into bounded records that can be used more explicitly by the AI component.

4.4 Model Parser

The second element of the framework is the model parser. The parser transforms the raw textual SysML v2-style model into structured records that can be used by the AI-supported reasoning module. The parser output is needed because an FMEA row should not be generated from a vague impression of the whole model. It should be generated from identifiable parts, functions, interfaces, requirements, allocations, and evidence fragments.

The parser output is organised mainly around parts. Each part-level record represents one model item that may be relevant for FMEA. The record includes the part name, its nearest containing parent part, its computed hierarchical layer, the original text of the part declaration, and selected context from elsewhere in the model. The *nearest containing parent part* means the smallest part block that textually contains the current part declaration. For example, if `lamp` is declared inside `flashlight`, then `flashlight` is the parent of `lamp`. If a smaller nested part contains another part, that smaller part becomes the nearest parent.

The *hierarchical layer* is not a native SysML construct. It is a parser-derived value used by this thesis to support grouping. Leaf parts, which do not contain child parts, are assigned layer 0. A parent part is then assigned one plus the maximum layer of its children. This allows the method to distinguish between leaf components, intermediate subsystems, and top-level assemblies when grouping model records for FMEA generation.

The parser also creates *bounded system-specific context*. In this thesis, bounded context means a compact, selected subset of the model that is relevant to a particular part or group. For example, the bounded context for the lamp may include the lamp part declaration, its ports, the action it performs, connections to the switch and optics, relevant light-generation requirements, and any allocations involving the lamp. This is more useful than sending the full model to the AI because it reduces noise, makes the evidence easier to inspect, and helps connect generated FMEA rows to specific model elements.

The parser is designed to handle both normally formatted SysML-style text and collapsed one-line model text. Collapsed one-line model text means that line breaks or indentation have been removed, for example after copying from a tool, exporting through an automated workflow, or passing the model through a text-processing system. The parser therefore cannot rely only on line-by-line formatting. It uses braces, keywords, and character positions to recover the structure of declarations.

Table 4.3 shows the parser output schema. The field `context_text` is stored as a structured text block with labelled subsections, rather than as one undifferentiated paragraph. These subsections may include related actions, requirements, local features, connections, bindings, flows, allocations, and state/control statements.

Table 4.3: Output schema of the SysML model parser

Field	Description
`part_name`	Name of the SysML part that forms the primary item for the record.
`parent_part`	Name of the nearest containing parent part, if one exists.
`layer`	Parser-derived hierarchy level. Leaf parts have layer 0; parent layers are computed from child layers.
`part_chunk`	Original SysML text belonging to the part declaration, including local ports, attributes, actions, and internal relations.
`context_text`	Structured context sections containing related actions, requirements, ports, attributes, connections, bindings, flows, allocations, and state/control statements.

An illustrative parser output for the lamp part is shown below. The exact output depends on the full model text, but the example shows the intended structure.

```
part_name: lamp
```

```
parent_part: flashlight
layer: 0

part_chunk:
    part lamp {
        attribute efficiency: Real;
        port dcPwrInPort;
        port lightOutPort;
        perform produceDirectedLight.generateLight;
    }

context_text:
    [Related actions]
    produceDirectedLight
    generateLight

    [Related connections]
    connect switch.outPort to lamp.dcPwrInPort
    connect lamp.lightOutPort to optics.lightInPort

    [Related bindings]
    bind lightOutPort = optics.lightOutPort

    [Related flows]
    flow from connectDCPwr.dcPwrOut to generateLight.dcPwrIn
    flow from generateLight.light to directLight.lightIn
```

The parser proceeds in several stages. First, it removes surrounding code fences and normalises the text. Second, it masks comments while preserving character positions. This is necessary because comments may contain braces or keywords that should not be interpreted as model structure. Third, it identifies declarations such as parts, actions, and requirements. Fourth, it uses brace matching to determine where each declaration begins and ends, even when the model is collapsed into one line. Fifth, it assigns parent-child relationships and computes hierarchy layers. Sixth, it extracts relations such as connections, bindings, flows, performed actions, allocations, and transitions. Finally, it builds one record per part by combining local part information with relevant external context.

The full parser pseudocode is provided in Appendix D.

In summary, Algorithm 2 transforms the SysML v2-style model into analysis-ready records. The first part of the algorithm recovers the syntactic structure of the model. The second part computes hierarchy and part relationships. The third part collects relation statements and contextual evidence. The final part creates one bounded context record per part. These records form the system-specific input to the AI-supported FMEA reasoning module.

4.5 FMEA Ontology

The third element of the framework is the FMEA ontology. The ontology is used as an independent semantic input to the AI-supported reasoning process. It does not come after the model parser in a linear pipeline. Instead, it works in parallel with the parser output. The parser provides system-specific context from the SysML model, while the ontology provides domain-agnostic FMEA concepts, relationships, rules, and question templates. The parser therefore defines what system information the AI should reason about, while the ontology guides how the AI should structure that reasoning.

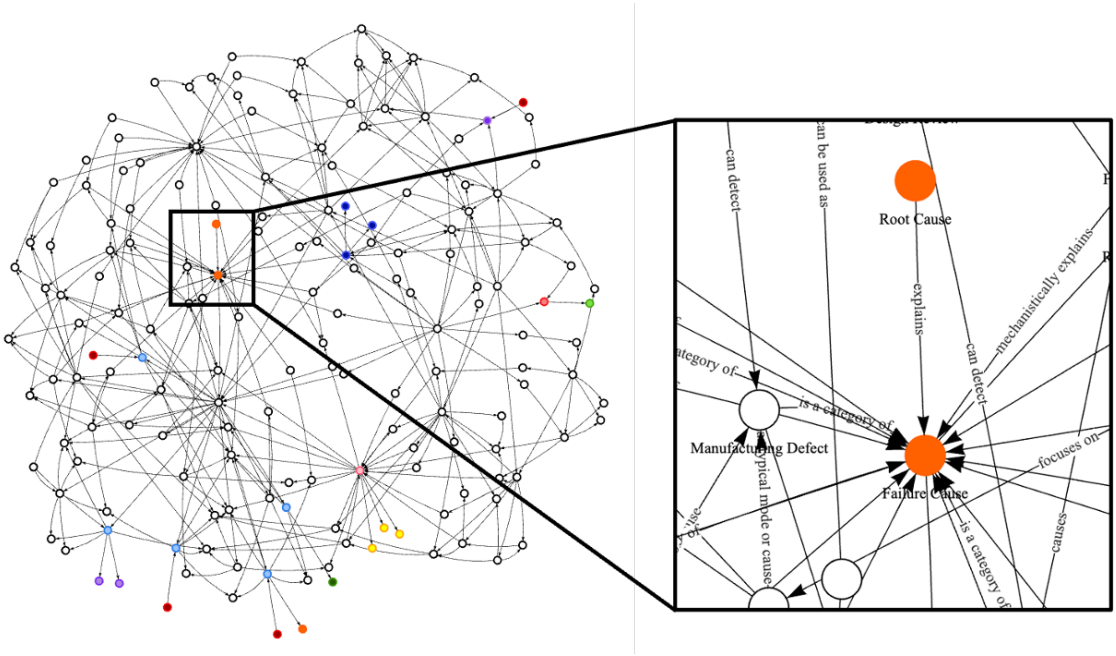


Figure 4.2: An overview of the FMEA ontology graph. The ontology used in this thesis contains 158 nodes and 355 edges

For the demonstration task, a domain-agnostic FMEA ontology is used. The ontology was developed from established FMEA and risk-analysis guidance, translated into a human-readable concept outline, formalised into a graph-like JSON structure, and prepared for GraphRAG-based use. The full ontology structure and creation process are described in Appendix C.

The ontology is modelled as a graph. In this graph, nodes represent FMEA-relevant concepts, reasoning rules, and AI question templates. Edges represent the relationships between these nodes. This graph structure is useful because FMEA reasoning depends on relationships. For example, an item performs a function, a function can fail through a failure mode, a failure mode can have a cause, a failure mode can produce an effect, and an effect can be evaluated using severity. The ontology makes these relationships explicit.

The ontology used in this thesis contains 158 nodes and 355 edges. The full graph is too dense to describe only in prose. Figure 4.2 therefore provides a visual overview of the ontology network. The node groups and edge relations are described in more detail in Appendix C.

The ontology includes several concept groups. Table 4.4 summarises the groups that are most relevant for the methodology.

Table 4.4: Main concept groups in the FMEA ontology

Concept group	**Examples**	**Role in AI-supported FMEA reasoning**
FMEA process concepts	scope definition, item identification, function analysis, failure analysis, risk evaluation, action planning	Guide the overall order and completeness of the FMEA workflow.
Failure logic concepts	item, function, failure mode, cause, effect, local effect, system effect	Structure the cause–failure–effect chain in each FMEA row.
Failure domains and categories	mechanical, electrical, software, material, interface, control, environmental failure	Help generate diverse but relevant candidate failure modes.
Risk and rating concepts	severity, occurrence, detection, RPN, criticality, residual risk	Support scoring and prioritisation of generated rows.
Control and action concepts	prevention control, detection control, recommended action, mitigation, design change	Support review of controls and improvement actions.
Evidence and traceability concepts	model element, requirement, assumption, source, rationale, evidence link	Help connect generated rows back to model context and supporting information.
AI question templates	generation prompt, review prompt, scoring prompt, explanation prompt, revision prompt	Provide reusable question structures for controlled AI interaction.
Reasoning rules	separate failure mode from cause and effect; preserve evidence; avoid unsupported rows; check duplicates	Constrain AI output and support systematic review.

The edges in the ontology define how concepts are related. For example, an *item* may *perform* a *function*; a *function* may *fail through* a *failure mode*; a *failure mode* may be *caused by* a *cause*; a *failure mode* may *lead to* an *effect*; an *effect* may be *evaluated by* severity; and an FMEA row should be *supported by* evidence or rationale. These edges allow the AI component to retrieve and use a structured view of FMEA logic instead of relying only on unconstrained text generation.

The AI question templates are represented as ontology nodes because they connect FMEA concepts to concrete information needs. For example, a generation template may ask: *Which failure modes can prevent this item from performing its function?* A review template may ask: *Is the failure mode separated clearly from its cause and effect?* A scoring template may ask: *Does the severity score match the described effect?* Representing these templates as nodes makes the prompting strategy explicit, reusable, and inspectable.

The ontology also includes reasoning rules. These rules do not prove that an FMEA row is correct, but they provide constraints that improve reviewability. Examples include: each row should identify an item and a function; a failure mode should describe how the function fails rather than why it fails; causes and effects should not be mixed; risk scores should be consistent with the described effect and controls; and generated claims should be supported by model context or a stated rationale. These rules are used by the AI component during generation, review, explanation, and revision.

By providing this semantic reasoning layer, the ontology helps make AI-supported analysis more bounded, consistent, explainable, and reviewable. It also supports model integrity because generated analysis outputs can be linked not only to system-specific model records, but also to explicit FMEA concepts and reasoning rules.

4.6 AI-Supported FMEA Reasoning

The fourth element of the framework is AI-supported FMEA reasoning. The AI component receives two complementary inputs. The first input is the parsed model context, which provides information about the specific system. The second input is the FMEA ontology, which provides the conceptual structure, reasoning rules, and question templates used for the analysis.

In implementation terms, each AI call receives a bounded prompt package. This package contains: the task instruction, the relevant parsed model records, selected ontology concepts or rules, the required output schema, and any previous review feedback. The purpose is to give the AI enough context to perform a specific reasoning task while avoiding unnecessary model text that may distract from the task.

Table 4.5 summarises how the AI uses the parser output and ontology in different FMEA reasoning tasks.

Table 4.5: AI prompt packages used for FMEA reasoning

AI task	Parsed model input	Ontology input	Expected output
Generation	Parent group, child parts, functions, ports, connections, requirements, allocations	FMEA concepts, failure categories, cause–effect rules	Candidate FMEA rows
Group review	Candidate rows and the same group context	Review rules and duplicate-check questions	Improved rows for one group
Global review	Combined rows from all groups and selected full-model context	Consistency, terminology, and traceability rules	Normalised table with duplicates reduced
Scoring	Reviewed rows, effects, causes, controls, and requirements	Severity, occurrence, detection, and RPN concepts	S, O, D, and RPN values with rationale
Explanation	Selected row and its supporting model context	Evidence and traceability concepts	Human-readable explanation of why the row exists
Revision	User feedback, selected rows, and relevant model records	Revision rules and question templates	Updated FMEA table and explanation of changes

For example, for the lamp part in the flashlight model, the parsed context may state that the lamp performs the action *GenerateLight*, receives power from the switch, sends light to the optics, and is allocated to a light-power requirement. The ontology then frames the AI

task in FMEA terms: identify the item, infer the function, propose ways the function can fail, separate failure modes from causes and effects, and provide supporting evidence.

An illustrative generation instruction for this example is:

> *Using the parsed context for the lamp and the FMEA ontology concepts item, function, failure mode, cause, effect, control, and evidence, generate candidate FMEA rows. Only use failure modes that can be related to the provided model context. Keep the failure mode, and effect separate.*

A possible generated row could identify the item as *lamp*, the function as *generate light*, the failure mode as *insufficient light output*, the cause as *insufficient electrical input or lamp degradation*, the effect as *reduced illumination performance*, and the evidence as the performed action, power connection, light-output connection, and allocated light-power requirement. This example illustrates how the AI uses the parser output for system grounding and the ontology for FMEA structure.

The AI output is not treated as automatically correct. Human review remains necessary because FMEA involves judgement, interpretation, prioritisation, and risk-related decisions. The AI is therefore used as an assisted reasoning layer built on top of structured model information and semantic guidance, not as a replacement for engineering responsibility.

The failure analysis generation procedure uses the parsed model records and the FMEA ontology as complementary inputs to the AI component. This relationship has already been introduced in Sections 4.4 and 4.5: the parser provides the bounded system-specific context, while the ontology provides the reasoning structure. Together, these inputs are used to generate, review, score, and revise a failure-analysis table.

The parsed records are first grouped according to their parent part. This grouping is useful because failures are often meaningful at subsystem or functional-group level rather than only at isolated component level. For example, the child parts of the flashlight, such as the battery, switch, lamp, and optics, are meaningful both as individual components and as part of the larger flashlight function of producing directed light.

For each part in the same parent group, the AI receives a prompt package consisting of grouped model context, selected ontology concepts and rules, and the required FMEA table schema. It then generates candidate FMEA rows for that part. These rows are reviewed at group level to improve relevance and cause-effect consistency. After all groups are processed, the rows are combined into one table. A global review is then performed across all groups to remove duplicates, normalise terminology, and improve consistency. Finally, severity, occurrence, and detection scores are assigned using a scoring rubric.

The scoring prompt operationalises the risk-rating step of the generated FMEA table. It instructs the AI to evaluate each failure mode using Severity (S), Occurrence (O), and Detection (D) scores on a scale from 1 to 10. Severity captures the seriousness of the failure effect, Occurrence captures the expected likelihood of the failure, and Detection captures how difficult the failure is to detect before its effect occurs. Since a higher Detection value represents weaker detectability, failures with poor detection mechanisms receive higher risk scores. The Risk Priority Number is calculated as $RPN = S \times O \times D$, after which the row is assigned a low, medium, or high risk class according to predefined thresholds and severity override rules. The prompt also requires a recommended action, scoring reasons, and explicit assumptions when information is missing. In this way, the scoring step remains structured, explainable, and suitable for human review rather than being treated as an automatic final judgement.

Algorithm 1 presents the FMEA generation procedure. The first part groups the parsed model records. The second part generates and reviews FMEA rows per group. The third

part combines and globally reviews all rows. The final part assigns risk scores and calculates the Risk Priority Number.

Algorithm 1 Ontology-Guided FMEA Generation

Require: Parsed model records $\mathscr{P}$, original model M, FMEA ontology $\mathscr{O}$, AI model A, scoring rubric $\mathscr{R}$
Ensure: Reviewed and scored FMEA table $\mathscr{F}$
$\mathscr{G} \leftarrow$ group records in $\mathscr{P}$ by `parent_part`
$\mathscr{F}_{all} \leftarrow \emptyset$
for all group $g \in \mathscr{G}$ **do**
    $parent_g \leftarrow$ parent part of group g
    $children_g \leftarrow$ all parsed records belonging to $parent_g$
    $X_g \leftarrow$ construct bounded group context from:
     parent name,
     child part names,
     child part chunks,
     child context texts,
     relevant actions, ports, connections, requirements, and allocations
    $\mathscr{F}_g^{raw} \leftarrow A(X_g, \mathscr{O})$ ▷ generate candidate FMEA rows
    $\mathscr{F}_g^{reviewed} \leftarrow A(\mathscr{F}_g^{raw}, X_g, \mathscr{O})$ ▷ review rows within the same parent group
    Remove vague, duplicated, or unsupported rows inside $\mathscr{F}_g^{reviewed}$
    Improve consistency between item, function, failure mode, cause, effect, control, and evidence
    Add $\mathscr{F}_g^{reviewed}$ to $\mathscr{F}_{all}$
end for
$\mathscr{F} \leftarrow$ combine all reviewed group-level rows from $\mathscr{F}_{all}$
$\mathscr{F} \leftarrow A(\mathscr{F}, M, \mathscr{O})$ ▷ global review across all groups
Remove duplicated failure modes across groups
Normalise repeated terminology for items, functions, causes, effects, and controls
Check consistency of similar failure modes across the full table
Strengthen weak evidence or rationale where possible
for all row $r \in \mathscr{F}$ **do**
    $r.S \leftarrow$ assign severity score using r, M, $\mathscr{O}$, and $\mathscr{R}$
    $r.O \leftarrow$ assign occurrence score using r, M, $\mathscr{O}$, and $\mathscr{R}$
    $r.D \leftarrow$ assign detection score using r, M, $\mathscr{O}$, and $\mathscr{R}$
    $r.RPN \leftarrow r.S \times r.O \times r.D$
end for
return $\mathscr{F}$

4.7 FMEA Table Structure

The generated FMEA table contains the information required for engineering review. Each row represents one candidate failure mode linked to the parsed model context and guided by the FMEA ontology. The terminology follows the standard FMEA logic in which an item performs a function, the function may fail through a failure mode, the failure mode may have causes and effects, and controls may prevent or detect the failure. The table schema used in this thesis is shown in Table 4.6.

The Risk Priority Number is calculated as described in Formula 3.1. In this thesis, these scores are used as review-support values rather than as final safety decisions. The purpose

Table 4.6: Generated FMEA table schema

Column	Description
ID	Unique 4 digit ID assigned to each failure mode in the table.
Part Name	The model element, component, subsystem, or part being analysed.
Failure mode	The way in which the item or function may fail.
Effect	The local or system-level consequence of the failure mode.
Severity (S)	Score representing the seriousness of the effect along with justification.
Occurrence (O)	Score representing the likelihood of the cause or failure mode along with justification.
Detection (D)	Score representing how difficult the failure is to detect before the effect occurs along with justification.
RPN	Risk Priority Number calculated from severity, occurrence, and detection.

is to demonstrate whether the framework can produce consistent and reviewable risk information, not to certify the flashlight design.

4.8 Interactive Review and Revision

The generated FMEA table is not treated as a final static result. Instead, it is presented to the user through an interface that supports review and revision. This is important because engineering analysis requires human judgement. The purpose of the interface is to allow the user to understand, challenge, correct, and improve the generated FMEA table.

The interaction has two modes: elicitation and revision. In elicitation mode, the user asks questions about the current FMEA table. For example, the user may ask why a failure mode was generated, which model element supports a row, why a severity score was assigned, or which failure modes belong to a subsystem. In this mode, the system provides an explanation but does not modify the table.

In revision mode, the user asks the system to modify the FMEA table. Example requests include removing a duplicated failure mode, adding a missing interface failure, rewriting a cause more clearly, changing a score, or splitting one failure mode into two separate rows. In this mode, the system returns both a response to the user and an updated FMEA table.

Table 4.7 summarises the two interaction modes.

Table 4.7: Interactive review modes

Mode	Purpose	Output
Elicitation	Help the user understand the current FMEA table.	Chat response only.
Revision	Modify, correct, or improve the FMEA table.	Chat response and updated FMEA table.

This interaction supports model integrity by keeping analysis outputs reviewable and maintainable. It also ensures that AI-supported outputs remain subject to human judgement rather than being accepted as final automatically.

4.9 Evaluation Method

The framework is evaluated in two complementary ways: verification and validation. In this thesis, verification means checking whether the generated FMEA output satisfies predefined structural and consistency tests. Validation means assessing whether the output appears useful, traceable, maintainable, and understandable for the intended engineering analysis purpose. In simple terms, verification asks whether the generated table is built correctly according to expected rules, while validation asks whether the result is meaningful and useful for engineering review.

4.9.1 Verification Method

The verification step evaluates the generated FMEA table against a set of predefined tests. These tests are written before analysing the output and are used to check whether the generated FMEA satisfies basic structural, model-coverage, and reasoning requirements. Each test is evaluated as pass, partial pass, or fail.

The purpose of verification is not to prove that every generated failure mode is correct. Rather, it checks whether the output follows the expected structure and whether it remains sufficiently connected to the model and ontology. This is important because an FMEA table may look complete while still missing important parts, mixing causes with effects, or failing to provide evidence for generated rows.

Table 4.8 lists the verification tests used in this thesis.

The generated FMEA table is inspected manually by the author against the tests in Table 4.8. A pass means the requirement is satisfied clearly. A partial pass means the requirement is satisfied for some rows but not consistently across the table. A fail means the requirement is mostly absent or not demonstrated. The verification results are reported in Chapter 5.

4.9.2 Validation Method

The validation step evaluates whether the framework output is useful for the intended purpose of supporting downstream engineering analysis. The validation focuses on four criteria: reliability, traceability, maintainability, and usability. These criteria are used to judge whether the generated analysis is meaningful, reviewable, and useful in the context of the case study.

Reliability examines whether the generated analysis is consistent with the available model information. For example, a failure mode should be relevant to the item or function being analysed, and the cause-effect chain should be plausible in relation to the model context.

Traceability examines whether analysis outputs can be linked back to relevant model context, assumptions, and ontology concepts. A traceable row should make it possible to understand why it was generated and which part of the model supports it.

Table 4.8: Verification tests for the generated FMEA table

Test	Verification question	Purpose
T1	Are all relevant model parts represented in the FMEA table?	Checks whether the analysis covers the system elements extracted by the parser.
T2	Are parent-child relationships reflected correctly?	Checks whether failure modes are assigned to the correct component, subsystem, or parent group.
T3	Are functions linked to the correct items or parts?	Checks whether failure modes are derived from what each part is intended to do.
T4	Does each FMEA row contain a clear failure mode?	Checks whether the row states a specific way in which a function or item can fail.
T5	Are failure modes separated from causes and effects?	Checks whether the FMEA avoids mixing what fails, why it fails, and what happens afterward.
T6	Does each row include a plausible cause-effect chain?	Checks whether the stated cause can reasonably lead to the failure mode and whether the effect follows from it.
T7	Are interface-related failures considered where connections exist?	Checks whether connections, ports, flows, and bindings are used to identify possible interface failures.
T8	Are relevant requirements or constraints reflected in the analysis?	Checks whether requirement allocations, constraints, or performance limits influence the generated FMEA where applicable.
T9	Are severity, occurrence, and detection scores assigned for each row?	Checks whether every generated row includes the required risk-rating fields.
T10	Are risk scores internally consistent with the described effects, causes, and controls?	Checks whether high-severity effects and weak controls are not given unjustifiably low risk ratings.
T11	Are duplicate or near-duplicate failure modes removed or merged?	Checks whether global review has reduced repeated rows across parts or groups.
T12	Does each row include evidence, rationale, or a link to model context?	Checks whether generated outputs are connected back to source model information or ontology-guided reasoning.

Maintainability examines whether the analysis can be reviewed, revised, or updated when information changes. This is assessed by checking whether the table contains identifiable rows, explicit evidence or rationale, and clear fields that can be edited without rebuilding the whole analysis manually.

Usability examines whether the output is understandable and useful for engineering review. This is assessed by checking whether an engineer can read the row, understand the item-function-failure relationship, inspect the rationale, and identify what would need to be corrected if the row is weak.

Table 4.9 summarises the validation criteria and how pass, partial pass, and fail judgements are assigned.

Table 4.9: Validation criteria for the framework demonstration

Criterion	**Meaning**	**Pass / partial pass / fail guidance**	**Example question**
Reliability	The analysis is consistent with the available model information.	Pass if most rows are plausible and model-consistent; partial pass if several rows need correction; fail if rows are largely unsupported or inconsistent.	Does the failure mode fit the item, function, and context?
Traceability	The analysis output can be linked to model context and ontology-guided reasoning concepts.	Pass if rows provide clear evidence or rationale; partial pass if evidence exists but is incomplete; fail if rows cannot be traced back to model context.	Can the row be justified using model information and ontology concepts?
Maintainability	The analysis can be reviewed, revised, or updated after feedback or model changes.	Pass if rows are structured, identifiable, and revisable; partial pass if revisions are possible but require manual interpretation; fail if the output is too static or ambiguous to update reliably.	Can the table be revised without rebuilding the analysis manually?
Usability	The output is understandable and useful for engineering review.	Pass if rows are readable and reviewable; partial pass if some terminology or rationale is unclear; fail if engineers cannot interpret or challenge the result.	Can an engineer inspect, understand, and challenge the result?

The validation performed in this thesis is limited to the case study demonstration. The framework is not tested in a real industrial project, with practising engineers, or in an operational safety-critical environment. Therefore, the validation should be understood as an analytical and case-study-based validation, not as full real-world validation. The results can indicate whether the framework is promising and where it requires improvement, but they

cannot prove general industrial validity.

The evaluation also identifies weaknesses in the framework. These weaknesses are used to propose improvements in Chapter 6.

4.10 Methodological Boundaries

Several boundaries apply to this methodology. First, the framework is demonstrated using a compact toy flashlight model rather than a full industrial Digital Engineering environment. The case study is intentionally limited so that the framework can be inspected clearly within the scope of an MSc thesis. The consequences of this boundary are discussed further in Chapter 5.

Second, the parser is designed for the textual SysML v2-style model used in this thesis. It is not claimed to cover every possible SysML format, every modelling tool export, or the full SysML language specification. Its purpose is to demonstrate how structured model context can be extracted for downstream analysis.

Third, the ontology provides reasoning guidance but does not guarantee complete engineering correctness. It helps organise concepts and relationships, but domain expertise is still required to judge whether a generated failure mode, cause, effect, or score is valid.

Fourth, AI-supported outputs require human review. The methodology demonstrates assisted analysis, not full automation of engineering judgement. This is especially important because FMEA involves interpretation, prioritisation, and risk-related decisions.

Finally, the case study evaluates usefulness, traceability, and maintainability in a controlled demonstration. It does not prove universal industrial validity. Instead, it provides evidence for how the proposed framework can support higher model integrity in a concrete downstream engineering analysis task.

4.11 Summary

This chapter presented the methodology used to develop and demonstrate the model integrity framework. The framework consists of model representation, model parsing, ontology-based reasoning guidance, and AI-supported FMEA reasoning. The model representation provides the SysML v2-style system-specific engineering information. The parser transforms the raw textual model into bounded context records for AI use. The ontology provides a domain-agnostic reasoning structure for FMEA. The AI component combines the parser output and ontology guidance to support generation, review, scoring, explanation, and revision.

The framework is demonstrated in Chapter 5 using a toy flashlight system model and FMEA as the downstream engineering analysis task. The generated outputs are evaluated according to reliability, traceability, maintainability, and usability.

Chapter 5

Results

5.1 Case Study Setup

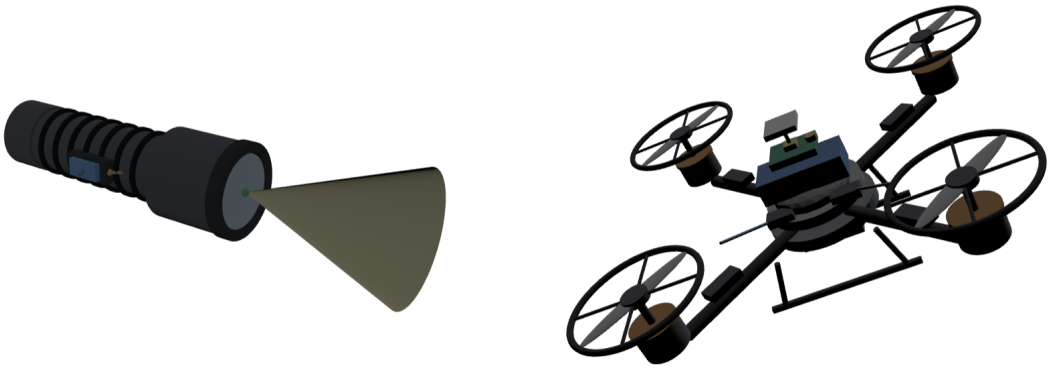
This chapter applies the methodology described in Chapter 4 to two SysML v2-style case study models. The first case study is the toy flashlight model, which is used as a compact and manually inspectable baseline case. The second case study is the toy quadcopter model, which is used as a richer model to examine how the framework behaves when the system contains more parts, interfaces, functions, requirements, repeated components, and safety-related concerns.

The purpose of using two case studies is to evaluate the proposed model integrity framework across two levels of model complexity. The flashlight model provides a small example in which the relationships between requirements, actions, parts, ports, connections, and allocations can be inspected manually. The quadcopter model provides a richer example with multiple subsystems, repeated components, control logic, telemetry, sensing, propulsion, and safety-related requirements. The comparison therefore gives insight into whether the framework remains useful when the model grows from a simple functional chain to a more interconnected system.

Both case studies are used to evaluate whether the framework can support traceable and reviewable Failure Mode and Effects Analysis (FMEA) generation from model-based engineering information. The evaluation is not intended to prove industrial validity. Instead, it examines whether the framework works in controlled engineering examples and identifies where the framework should be improved before it could be applied in an industrial Digital Engineering setting.

This chapter distinguishes three forms of review to avoid ambiguity. First, the generated FMEA rows are reviewed during the generation process at group level and global level. This review is used to merge duplicate rows, flag weak rows, and check consistency; it does not mean that unsupported rows are manually invented. Second, verification and validation assess the resulting FMEA outputs against predefined criteria. Third, interactive review refers to a later user interaction with the generated FMEA table, where a human reviewer asks questions or requests revisions.

Figure 5.1 shows the 3D model representations used for the two case studies. The flashlight model is used as the simple baseline model, while the quadcopter model is used as the more complex model.



(a) Flashlight 3D model **(b) Quadcopter 3D model**

Figure 5.1: 3D representations of the two case study models

Table 5.1 summarises the two case study models and their role in the evaluation.

Table 5.1: Overview of the two case study models

Case study	Purpose	Model characteristics
Flashlight model	Simple baseline case used to test the framework on a compact and manually inspectable SysML v2-style model.	Contains requirements, action flow, parts, ports, states, connections, bindings, and requirement allocations.
Quadcopter model	More complex case used to test the framework on a richer multi-subsystem SysML v2-style model.	Contains flight-performance, sensing, control, energy, safety, telemetry, propulsion, and physical design information.

The complete flashlight and quadcopter models are provided in Appendix A and Appendix B, respectively.

5.2 Case Study Models

5.2.1 Flashlight Model

The flashlight model represents a compact electromechanical system. It contains a requirements package, an action tree, a parts tree, and a requirement allocation package. The requirements describe expected properties such as field of view, light power, size, weight, reliability, and cost. The action tree describes the functional behaviour required to produce directed light. The main action, `produceDirectedLight`, is decomposed into actions for providing DC power, connecting DC power, generating light, and directing light.

The parts tree represents the physical structure of the flashlight. The main part is `flashlight`,

which contains attributes such as mass, field of view, and illumination level. It also contains ports, state behaviour, and internal parts including the battery, switch, lamp, optics, reflector, lens, and structure. Connections and bindings define how power, command, and light-related interactions move through the system.

The flashlight model is suitable as a simple case study because its functional chain is easy to follow: the battery provides power, the switch connects power, the lamp generates light, and the optics direct the light. At the same time, the model includes enough relationships to test the framework, including part hierarchy, actions, ports, connections, states, and requirement allocations.

5.2.2 Quadcopter Model

The quadcopter model represents a more complex multi-subsystem system. It contains requirements related to user interface, flight performance, physical constraints, energy, sensing and control, safety, reliability, and cost. Compared with the flashlight model, the quadcopter model contains a broader set of engineering concerns, including controlled flight, hover stability, flight time, payload, power distribution, telemetry, attitude sensing, altitude sensing, flight control, failsafe behaviour, and low-battery protection.

The action tree describes the behaviour required to produce controlled flight. It includes actions for providing electrical power, distributing electrical power, receiving flight commands, measuring vehicle state, computing flight control, modulating motor power, generating rotor thrust, combining thrust, and reporting telemetry. This creates a more complex functional chain than the flashlight model because sensing, command processing, control computation, power distribution, propulsion, and telemetry are all connected.

The parts tree includes the quadcopter, battery, power distribution board, receiver, flight controller, sensor suite, electronic speed controllers, motors, propellers, thrust mixer, telemetry radio, and frame body. Some parts have multiplicities, such as four electronic speed controllers, four motors, four propellers, and four arms. This makes the model useful for examining how the parser and FMEA generation procedure handle repeated elements and richer interconnections.

Table 5.2 summarises the main differences between the two models.

Table 5.2: Comparison of the flashlight and quadcopter case study models

Aspect	Flashlight model	Quadcopter model
System complexity	Simple compact system.	More complex multi-subsystem system.
Main function	Produce directed light.	Produce controlled flight and telemetry.
Main subsystems	Battery, switch, lamp, optics, structure.	Battery, power distribution, receiver, flight controller, sensor suite, ESCs, motors, propellers, telemetry, frame.
Behavioural content	Power and light-generation action chain.	Power, command, sensing, control, propulsion, thrust, and telemetry action chains.
Requirement types	Illumination and physical requirements.	User interface, flight performance, physical, energy, sensing/control, and safety requirements.
Purpose in evaluation	Baseline inspectable case.	Complexity and scalability stress test.

5.3 Model Parser Results

Both case study models were processed by the SysML v2-style model parser described in Chapter 4. The parser transforms the textual model representation into structured records that can be used by the AI-supported reasoning component. Each parsed record contains the part name, parent part, hierarchy level, original model fragment, and related context. Related context includes nearby requirements, actions, ports, attributes, connections, bindings, flows, allocations, and relevant state or control statements.

For the flashlight case, the SysML v2-style model contains four main modelling packages: requirements, actions, parts, and requirement allocations. The requirements package defines the flashlight specification, including illumination and physical requirements. The action tree defines the functional chain for producing directed light: providing DC power, connecting DC power, generating light, and directing light. The parts tree defines the physical components and their ports, connections, states, and bindings. The parser produced 13 raw records. Eleven of these were accepted as intended model-element records. Two records, `to` and `connect`, were parser artefacts caused by comment text around connection statements. This shows that comment masking and declaration parsing are important for maintaining parser reliability.

For the quadcopter case, the SysML v2-style model is larger and contains richer model information. It includes a requirements package with user-interface, flight-performance, physical, energy, sensing-and-control, safety, reliability, and cost requirements. The action tree defines the behaviour required to produce controlled flight, including electrical power provision, power distribution, command reception, vehicle-state measurement, flight-control computation, motor-power modulation, rotor-thrust generation, thrust combination, and telemetry reporting. The parts tree contains the `quadcopter`, battery, power distribution board, receiver, flight controller, sensor suite, ESCs, motors, propellers, thrust mixer, teleme-

try radio, and frame-body elements. The parser produced 20 model-element records for this case. Unlike the flashlight case, no false part records were observed in the parsed quadcopter output.

The quadcopter parser output also shows a scalability issue. Several records contain very long related-context fields because the parser attaches large portions of the action tree, requirement hierarchy, and allocation package to multiple parts. This is useful for traceability, but it can also make the AI input verbose. The quadcopter case therefore shows both the benefit and the challenge of the parser: it successfully extracts structured context from a richer model, but it also needs better context selection so that only the most relevant requirements, actions, and connections are passed to the downstream FMEA step.

Table 5.3 summarises the parser results for both case studies.

Table 5.3: Summary of parser results for the two case studies

Parser output measure	Flashlight model	Quadcopter model
Number of raw parsed part records	13 raw records	20 raw records
Number of accepted model-element records	11 accepted records	20 accepted records
Number of parser artefacts observed	2 artefacts: `to` and `connect`	0 false part records observed
Number of top-level parts	1 top-level part: `flashlight`	1 top-level part: `quadcopter`
Number of nested child parts	10 nested parts below `flashlight`	19 child parts below `quadcopter`; 11 direct children, 3 below `sensorSuite`, and 5 below `frameBody`
Number of extracted ports	10 port declarations	30 port declarations
Number of extracted attributes	7 attribute declarations, including requirement-related attributes	21 attribute declarations, including performance, physical, battery, sensing, and propulsion attributes
Number of extracted connections and bindings	3 connections and 2 bindings	11 connections and 3 bindings
Number of extracted actions or performed actions	5 action definitions and 4 performed-action links	10 action definitions and 10 performed-action links
Number of extracted requirements or allocations	11 requirement declarations and 4 allocation statements	26 requirement declarations and 16 allocation statements
Maximum hierarchy depth	3 logical levels: `flashlight` → subsystem/group → child part	3 logical levels: `quadcopter` → subsystem/group → child part
Main parser issues observed	Comment text around connection statements produced two false part records.	Context extraction was much more verbose; repeated parts such as `esc[4]`, `motor[4]`, `propeller[4]`, `arm[4]`, and `propellerGuard[4]` were represented as single part records with multiplicity.

The parser results are important for evaluating the framework because they determine the quality of the context supplied to the AI component. The flashlight case shows that even a small model can create parsing artefacts if comments are not handled carefully. The quadcopter case shows that a richer SysML v2-style model can be parsed into structured records, but that context selection becomes more important as the model grows. This directly affects downstream FMEA generation: if the generated analysis only receives a narrow subset of the parsed model, then the resulting FMEA will also have narrow coverage.

5.4 Ontology Use in the Case Studies

The same domain-agnostic FMEA ontology developed in Chapter 4 was used for both the flashlight and quadcopter case studies. The purpose of this section is only to summarise how the ontology was used in the results; the ontology itself is explained in detail in Chapter 4 and Appendix C.

The ontology provides general FMEA reasoning guidance rather than system-specific content. In the framework, the parser provides the system-specific context extracted from the SysML v2-style model, while the ontology provides the conceptual structure for FMEA reasoning. This means that the ontology does not state that a flashlight battery can fail or that a quadcopter motor can lose thrust. Instead, it provides general concepts such as function, failure mode, cause, effect, control, severity, occurrence, detection, recommended action, residual risk, and evidence. The AI-supported reasoning component then applies these concepts to the parsed model context.

For the flashlight model, the ontology helps transform simple functions such as providing DC power, connecting DC power, generating light, and directing light into possible failure-mode patterns. For the quadcopter model, the ontology guides reasoning over a wider set of functions, including power distribution, command reception, sensing, flight-control computation, motor-power modulation, rotor-thrust generation, thrust combination, and telemetry reporting.

The use of the same ontology in both models tests whether the ontology can support different levels of system complexity. The flashlight case tests whether the ontology supports a simple functional chain. The quadcopter case tests whether the ontology remains useful when the model contains more subsystems, interfaces, operating states, and safety-related requirements.

5.5 Generated FMEA Results

The FMEA generation procedure was applied to the parsed records using two review stages. First, group-level analysis generated candidate failure modes for local model groups. Second, global-level analysis inspected the combined outputs, normalised wording, split or merged selected rows, and prepared rows for scoring. In this workflow, *group-level review* refers to reviewing rows generated for one parent group or subsystem, while *global review* refers to reviewing the combined FMEA candidates across the case study. These review stages are part of the generation workflow and are different from the later interactive review discussed in Section 5.9.

For the flashlight, the group-level analysis produced 12 candidate rows. These rows focused mainly on the `dcPwrOutPort`, the optical parts, and the housing parts. The `dcPwrOutPort` rows covered open circuit, short circuit, corrosion, and loose connection. The reflector rows covered surface contamination, surface damage or scratching, deformation or warping, coating failure, and corrosion or oxidation. The lens and housing rows did not identify concrete failure modes; instead, they produced missing-information rows. These rows are still useful because they indicate where the available ontology and model context were not sufficient to generate a specific failure mode.

The flashlight global-level analysis produced 13 rows for scoring. Most group-level rows were retained, but the combined *corrosion or oxidation* row for the reflector was split into two separate rows: corrosion and oxidation. This increased the number of rows from 12 to 13. No duplicate rows were removed in this case. Some global-level outputs also required

light syntactic normalisation before scoring, because several rows were returned as array content wrapped inside invalid JSON-like braces. This observation is relevant for the framework evaluation because it shows that AI-generated intermediate artefacts need validation before they are treated as structured data.

For the quadcopter, the generation outputs show a different behaviour. The SysML parser extracted 20 model-element records from the quadcopter model, but the generated FMEA results supplied for this evaluation focused mainly on the `frameBody` group, especially the `propellerGuard`. The group-level outputs produced 24 candidate rows across three group-level review outputs. These rows covered `propellerGuard`, `arm`, `centerBody`, `landingGear`, and `payloadMount`. The strongest quadcopter rows concerned safety-related propeller-guard failures, such as structural damage, breakage, detachment, loose fitting, interference with propeller rotation, and material degradation. In contrast, `arm`, `centerBody`, `landingGear`, and `payloadMount` mostly produced missing-information rows.

The quadcopter global-level output also produced 24 rows for scoring. Several group-level rows were split into more specific entries. For example, *structural damage or breakage* was split into *structural damage* and *breakage*, and *detachment or loose fitting* was sometimes split into separate detachment and loose-fitting rows. However, the global review did not fully consolidate overlapping rows across all generated outputs. As a result, the scored quadcopter output contains repeated or near-duplicate entries for several `propellerGuard` failure modes. This is an important result because it shows that global review improves local wording but still requires stronger duplicate handling before the method can be treated as an industrial-strength FMEA generator.

Each scored FMEA row contains the analysed item, failure mode, effect, severity, occurrence, detection, RPN, risk class, recommended action, and assumptions or rationale. The flashlight scoring step produced 13 scored rows: six high-risk rows, four medium-risk rows, and three low-risk rows. The highest flashlight RPN was assigned to the `dcPwrOutPort` short-circuit failure mode, with an RPN of 400. The quadcopter scoring step produced 24 scored rows: 18 high-risk rows, one medium-risk row, and five low-risk rows. The highest quadcopter RPN was assigned to `propellerGuard` detachment, with an RPN of 400.

Table 5.4 summarises the generated FMEA results for both case studies.

Table 5.4: FMEA generation, review, and scoring summary

Result measure	Flashlight result	Quadcopter result
Rows generated during group-level analysis	12 candidate rows	24 candidate rows across three group-level outputs
Rows after global-level analysis	13 rows; reflector *corrosion or oxidation* was split into separate corrosion and oxidation rows	24 rows; several combined propeller-guard modes were split, but repeated or near-duplicate rows remained
Rows after scoring	13 scored rows	24 scored rows
Duplicate or overlapping rows	No exact duplicates removed; one combined row was split	Overlap remained; 24 scored rows correspond to 13 exact item–failure-mode labels, with repeated `propellerGuard` rows
Rows with complete S/O/D scoring	13 rows with severity, occurrence, detection, and RPN values	24 rows with severity, occurrence, detection, and RPN values
Rows with rationale or assumptions	13 rows include scoring reasons and assumptions; formal evidence links were not yet generated	24 rows include scoring reasons and assumptions; formal evidence links were not yet generated
Risk-class distribution	6 high-risk rows, 4 medium-risk rows, and 3 low-risk rows	18 high-risk rows, 1 medium-risk row, and 5 low-risk rows
Highest-risk row	`dcPwrOutPort` short circuit, RPN = 400	`propellerGuard` detachment, RPN = 400
Main limitation observed	Several rows relied on assumptions rather than explicit evidence, and missing-information rows appeared for `lens`, `backHousing`, and `middleHousing`.	FMEA coverage was narrow compared with the parsed model; most scored rows concern `propellerGuard` and frame-body elements, while major subsystems such as battery, power distribution, receiver, flight controller, sensor suite, ESCs, motors, propellers, thrust mixer, and telemetry radio were not represented in the scored output.

Table 5.5 shows representative examples of how group-level outputs changed during global review for the flashlight case. Table 5.6 provides the same type of example for the quadcopter case. These tables are not final FMEA tables; they illustrate intermediate review behaviour before final scoring.

Table 5.5: Examples of group-level and global-level outputs for the flashlight case

Item	Group-level failure mode	Group-level effect	Global-level change
`dcPwrOutPort`	open circuit	No power delivered to switch; flashlight does not turn on.	Wording was normalised to "Power is interrupted to the switch, causing the flashlight to remain off."
`dcPwrOutPort`	short circuit	Excessive current flow causing potential battery damage or safety hazard.	Effect was expanded to include battery damage, overheating, and safety hazards.
`reflector`	corrosion or oxidation	Surface degradation compromising reflectivity and long-term reliability.	Combined mode was split into two rows: corrosion and oxidation.
`lens`	no relevant failure modes found	No data available on failure modes and effects for lens in the ontology.	Row was retained as a missing-information indicator rather than a confirmed technical failure mode.
`backHousing` and `middleHousing`	no specific failure modes found	No specific effects identified due to lack of information.	Rows were retained as missing-context indicators and should be revisited with more detailed structural or material information.

Table 5.6: Examples of group-level and global-level outputs for the quadcopter case

Item	Group-level failure mode	Group-level effect	Global-level change
`propellerGuard`	structural damage or breakage	Reduced or lost ability to protect from rotating propellers, increasing injury risk.	Split into separate *structural damage* and *breakage* rows.
`propellerGuard`	detachment or loose fitting	Guard may detach or shift during flight, increasing the risk of contact with rotating propellers.	In some outputs this was split into separate *detachment* and *loose fitting* rows; in another output it remained combined, showing incomplete duplicate consolidation.
`propellerGuard`	interference with propeller rotation	Could cause propeller damage, loss of thrust, unstable flight, or crash risk.	Retained and scored as a high-risk row because it can affect flight stability and safety.
`propellerGuard`	material degradation	Weakening of guard structure over time leading to loss of protective function.	Retained and scored as a high-risk row; wording was normalised across outputs only partially.
`arm`, `centerBody`, `landingGear`, and `payloadMount`	no relevant failure modes found	No specific effects identified due to lack of data.	Rows were retained as missing-information indicators rather than validated technical failure modes.

Table 5.7 gives the generated and scored FMEA table for the flashlight case. The complete generated and scored FMEA table for the quadcopter case is placed in Appendix F, because the table is too large for the main results chapter. Rows such as *no relevant failure modes found* and *No specific failure modes found* should be interpreted as diagnostic outputs of the workflow. They indicate missing context or insufficient ontology coverage, not validated failure modes.

Table 5.7: Generated and scored FMEA rows for the flashlight case study

Item	Failure mode	Effect	S/O/D/RPN	Risk	Recommended action
`dcPwrOutPort`	open circuit	Power is interrupted to the switch, causing the flashlight to remain off.	6/4/7/168	Medium	Implement a basic continuity check or visual indicator on the power output port to detect connection issues before use.
`dcPwrOutPort`	short circuit	Excessive current flow leading to potential battery damage, overheating, or safety hazards.	10/5/8/400	High	Implement robust short circuit protection such as fuses or electronic current limiting and include continuous monitoring or automatic shutdown features.

Continued on next page

Table 5.7: Generated and scored FMEA rows for the flashlight case study (continued)

Item	Failure mode	Effect	S/O/D/RPN	Risk	Recommended action
dcPwrOutPort	corroded connection	Intermittent power delivery causing flickering or dim lighting.	5/7/8/280	High	Implement protective coatings or seals on the connection to prevent corrosion and schedule periodic maintenance inspections.
dcPwrOutPort	loose connection	Loss of power transmission causing the flashlight to turn off.	7/6/8/336	High	Implement a mechanical retention design or locking mechanism for the power output port connection to reduce loosening risk, and add visual or tactile inspection procedures during maintenance to detect looseness early.
lens	no relevant failure modes found	no data available on failure modes and effects for lens in the ontology	2/3/8/48	Low	Perform detailed component inspection and testing to identify potential failure modes and update FMEA accordingly.
reflector	surface contamination	Reduced reflectivity leading to decreased optical system performance	6/5/6/180	Medium	Implement regular cleaning protocols and visual inspection to maintain surface cleanliness and optical performance.
reflector	surface damage or scratching	distorted reflection causing image degradation or unwanted scattering	5/5/7/175	Medium	Implement protective coatings or covers to prevent surface scratches and improve inspection procedures to detect damage early.
reflector	deformation or warping	Improper focusing or altered reflection angle, resulting in optical misalignment	7/4/6/168	Medium	Implement regular maintenance inspections and environmental protection measures to prevent deformation; consider using more stable materials with less susceptibility to warping.
reflector	coating failure	loss of reflective coating causing increased absorption and reduced reflectivity	6/5/7/210	High	Implement routine inspection procedures or develop a simple reflectivity test to detect coating degradation early.
reflector	corrosion	surface degradation compromising reflectivity and long-term reliability	5/6/7/210	High	Apply protective coatings and implement regular inspections to detect corrosion early
reflector	oxidation	surface degradation compromising reflectivity and long-term reliability	5/6/7/210	High	Use corrosion-resistant materials or coatings and schedule regular visual inspections
backHousing	No specific failure modes found	No specific effects identified due to lack of information	2/3/8/48	Low	Conduct detailed examination to identify specific failure modes and effects for more accurate scoring; establish functional tests or inspections to improve detection.
middleHousing	No specific failure modes found	No specific effects identified due to lack of information	3/3/8/72	Low	Perform detailed failure mode analysis for middleHousing to identify specific failure modes and effects; implement inspections if wear or damage is possible.

The generated flashlight FMEA shows that the framework can transform bounded SysML model context and ontology guidance into a reviewable downstream analysis table. The strongest results are obtained for elements with explicit functional or interface context, such as the dcPwrOutPort and the reflector. Weaker results occur for passive parts with little or no contextual information, such as the lens and housing elements. The generated quadcopter FMEA shows that the same workflow can produce safety-relevant rows for a richer model, but the actual generated coverage is narrow: most rows concern the propellerGuard. This supports the thesis argument that model integrity affects downstream analysis quality. When the model context supplied to the AI is narrow or when the

review step does not consolidate repeated outputs, the resulting FMEA becomes less complete and less maintainable.

5.6 Comparison Between Simple and Complex Case Studies

The flashlight and quadcopter cases are compared to examine how the framework behaves when model complexity increases. The flashlight case checks whether the framework can produce inspectable FMEA output from a small and transparent SysML v2-style model. The quadcopter case checks whether the same framework remains useful when the model contains more elements, more interfaces, repeated components, more functions, and more safety-related requirements.

The comparison focuses on the aspects defined in Table 5.8. These definitions make clear how each aspect is assessed. The assessment is based on manual inspection by the author of the parser output, generated FMEA rows, review outputs, and links to model context.

Table 5.8: Definition and assessment of comparison aspects

Comparison aspect	**How it is assessed in this thesis**
Ease of manual inspection	Whether the model and generated FMEA rows can be checked manually without excessive cross-referencing.
Parser complexity	Number of parsed records, hierarchy depth, multiplicities, connections, actions, requirements, and context size.
Context extraction quality	Whether parsed records retain relevant actions, requirements, ports, connections, and allocations without excessive irrelevant text.
FMEA generation complexity	Number and diversity of generated rows, failure domains, repeated rows, and missing-information rows.
Duplicate or overlapping rows	Whether group-level and global-level review remove, merge, or split repeated failure modes consistently.
Requirement-to-analysis linkage	Whether generated rows can be related back to explicit requirements, constraints, or allocated model elements.
Interface failure coverage	Whether generated rows reflect failures at ports, connections, bindings, and interfaces present in the SysML v2-style model.
Usefulness of ontology guidance	Whether the ontology helps produce meaningful failure modes, effects, controls, and risk scores, and where it instead produces missing-information rows.

Table 5.9 summarises the comparison between the two case studies.

Table 5.9: Comparison of framework behaviour across the two case studies

Comparison aspect	**Flashlight model**	**Quadcopter model**
Ease of manual inspection	High; compact model, short functional chain, and 13 scored rows.	Lower; parser output is much richer, but the scored FMEA output is concentrated on a subset of the model.
Parser complexity	Low to moderate; 13 raw records, 11 accepted model-element records, 3 connections, and 2 bindings.	Higher; 20 parsed records, 30 ports, 21 attributes, 11 connections, 3 bindings, 26 requirements, and 16 allocations.
Context extraction quality	Partly successful; useful context was extracted, but comment text created two false part records.	Structurally successful; no false part records observed, but related-context fields became verbose and repeated.
FMEA generation complexity	Moderate; 12 group-level rows and 13 scored rows.	Higher; 24 group-level rows and 24 scored rows, but many rows were repeated or overlapping.
Duplicate or overlapping rows	Limited overlap; reflector corrosion or oxidation was split into two rows.	Substantial overlap; repeated `propellerGuard` rows remained after global review.
Requirement-to-analysis linkage	Indirect; illumination and functional-loss concerns are reflected, but explicit requirement identifiers are not carried into final rows.	Partial; propeller-guarding and safety concerns are reflected, but many other allocated requirements do not appear in the scored rows.
Interface failure coverage	Partial; power-port/interface failures are represented through `dcPwrOutPort`, but other connections are not fully covered.	Limited; the generated rows focus on propeller-guard safety and do not cover most power, sensing, control, propulsion, and telemetry interfaces.
Usefulness of ontology guidance	Useful for electrical and optical failure categories; weaker for passive parts with little context.	Useful for safety-related guard failures; weaker for broader quadcopter subsystems and passive frame-body elements.
Overall observation	Demonstrates that the framework can produce reviewable rows for a compact model, while revealing parser and evidence-linking limitations.	Demonstrates scalability pressure: the parser can handle a richer model, but the generation and review stages need stronger coverage control and duplicate consolidation.

The comparison shows that the framework does not fail simply because the model becomes larger. The parser successfully extracts a richer quadcopter model. However, downstream generation does not automatically use all available parsed context. The main challenge therefore shifts from merely extracting information to selecting, covering, and consolidating the extracted information in a controlled way.

5.7 Verification Results

The verification step evaluates the generated FMEA tables against the predefined tests defined in Chapter 4. Each test is assessed as pass, partial pass, or fail. A pass means that the test is clearly satisfied. A partial pass means that the test is satisfied for some rows or model elements but not consistently across the whole table. A fail means that the test is mostly absent or not demonstrated.

The flashlight verification results show that the generated FMEA table is structurally usable, but not yet complete as an engineering FMEA. The scoring structure is complete for all 13 generated rows, and the failure-mode, effect, and action fields are generally understandable. However, model coverage is incomplete because several valid parsed model elements, such as `switch`, `lamp`, `optics`, `structure`, and `frontHousing`, do not appear as analysed items in the final FMEA table. In addition, the final rows contain rationale and assumptions, but they do not yet contain formal evidence links to specific model elements or requirement identifiers.

The quadcopter verification results show a stronger contrast between parser coverage and FMEA coverage. The parser extracted 20 model-element records from the quadcopter model, but the scored FMEA output covers only five item names: `propellerGuard`, `arm`, `centerBody`, `landingGear`, and `payloadMount`. Most high-risk rows concern `propellerGuard`. This means that the generated FMEA captures an important safety-related area, but it does not cover many central flight-related elements such as the battery, power distribution board, receiver, flight controller, sensor suite, ESCs, motors, propellers, thrust mixer, and telemetry radio. The quadcopter output is therefore useful as a demonstration of local safety-related generation, but it is not a complete FMEA for the full quadcopter model.

Table 5.10 provides the verification results for both case studies.

Table 5.10: Verification results for the generated FMEA tables

Test	Verification question	Flashlight result	Quadcopter result	Notes
T1	Are all relevant model parts represented in the FMEA table?	Partial pass	Partial pass	Flashlight rows cover some electrical, optical, and housing elements. Quadcopter rows cover only frame-body/propeller-guard-related elements despite 20 parsed records.
T2	Are parent-child relationships reflected correctly?	Partial pass	Partial pass	Parent-child relations exist in the parser output, but the final FMEA rows do not explicitly retain the parent group.
T3	Are functions linked to the correct items or parts?	Partial pass	Partial pass	Stronger for explicit functional/interface elements; weaker for passive or missing-information rows. In the quadcopter case, propeller-guard safety is represented, but flight-control and propulsion functions are not covered.
T4	Does each FMEA row contain a clear failure mode?	Partial pass	Partial pass	Most technical rows contain clear modes; missing-information rows are diagnostic outputs rather than validated failure modes.
T5	Are failure modes separated from causes and effects?	Pass	Pass	Failure mode and effect are represented in separate fields throughout the scored outputs.
T6	Does each row include a plausible cause-effect chain?	Partial pass	Partial pass	Effects are generally plausible, but causes are mostly implicit in scoring reasons and assumptions rather than explicit FMEA fields.
T7	Are interface-related failures considered where connections exist?	Partial pass	Partial pass	Flashlight covers one electrical interface. Quadcopter output does not cover most power, sensing, control, propulsion, and telemetry interfaces.
T8	Are relevant requirements or constraints reflected in the analysis?	Partial pass	Partial pass	Requirement concerns are reflected indirectly, but final rows do not carry explicit requirement identifiers.
T9	Are severity, occurrence, and detection scores assigned for each row?	Pass	Pass	All 13 flashlight rows and all 24 quadcopter rows include S/O/D values and RPN.
T10	Are risk scores internally consistent with the described effects, causes, and controls?	Pass	Partial pass	RPN values are mathematically consistent with S/O/D values. In the quadcopter output, overlapping rows sometimes receive different scores, indicating a need for score harmonisation.

The verification results show that the framework produces structurally usable FMEA rows, but that completeness and traceability remain the main weaknesses. The results also show that parser success is not sufficient on its own. Even when many model elements are extracted, the generation procedure must still ensure systematic coverage of those elements.

5.8 Validation Results

The validation step evaluates whether the generated FMEA outputs are useful for the intended engineering review purpose. In this thesis, validation is analytical and case-study-based. The framework is not tested in a real industrial project, with practising engineers, or in an operational safety-critical environment. Therefore, the validation results should be interpreted as evidence of usefulness within controlled examples, not as proof of full real-world validity.

Validation is assessed using four criteria: reliability, traceability, maintainability, and usability. Reliability examines whether generated rows are consistent with the available model information. Traceability examines whether rows can be linked back to model context and ontology concepts. Maintainability examines whether the table can be reviewed, revised, or updated after feedback or model changes. Usability examines whether the output is understandable and useful for engineering review.

For this validation, a pass means that the criterion is clearly demonstrated for most generated rows. A partial pass means that the criterion is demonstrated for some rows, but important limitations remain. A fail means that the criterion is not demonstrated sufficiently for the generated output. The assessment is based on manual inspection by the author.

Table 5.11 summarises the validation results.

Table 5.11: Validation results for the generated FMEA outputs

Validation criterion	Flashlight assessment	Quadcopter assessment	Rationale
Reliability	Partial pass	Partial pass	The generated technical rows are mostly plausible. However, both cases include missing-information rows and assumptions. The quadcopter output also contains repeated rows with varying scores.
Traceability	Partial pass	Partial pass	Rows can be interpreted in relation to the parsed model context and ontology concepts, but explicit trace links to model elements, parent groups, requirements, and evidence are not yet carried into the final table.
Maintainability	Partial pass	Partial pass	The rows are structured and can be reviewed or revised. However, duplicate consolidation, score harmonisation, and coverage checking would be needed before the tables are maintainable in a larger engineering process.
Usability	Partial pass	Partial pass	The tables are understandable and useful for identifying candidate risks. The flashlight table is easier to inspect. The quadcopter table reveals safety-relevant propeller-guard risks but is not representative of the full quadcopter model.

The validation results show that the framework is useful as an analysis-support workflow rather than as a fully automatic FMEA generator. The strongest validation result is reviewability: the generated rows are concrete enough for a human reviewer to inspect, challenge, and improve. The weakest result is completeness: neither case produces a full FMEA covering all model elements and all relevant requirements. This is especially clear in the quadcopter case, where the parser extracts a rich model but the generated FMEA output focuses on a narrow subset.

5.9 Interactive Review and Revision Results

The generated FMEA tables were also evaluated through example review and revision scenarios. This step is different from the group-level and global review described in Section 5.5. Group-level and global review are part of the generation workflow and are used to clean, split, or check generated rows before scoring. Interactive review happens after a generated FMEA table exists. It examines whether a human reviewer can inspect, question, and revise the table through the framework.

Two types of interaction are considered. The first is *elicitation*, where the user asks questions about the existing FMEA table without changing it. Example elicitation questions in-

clude asking why a failure mode was generated, which model element supports a row, why a severity score was assigned, or which rows belong to a specific subsystem. The second type is *revision*, where the user asks for changes to the FMEA table. Example revision requests include removing duplicate rows, adding missing interface failures, changing scores, rewriting causes, or splitting one broad failure mode into more specific rows.

The flashlight and quadcopter outputs show why this interactive step is necessary. In the flashlight case, a reviewer would need to challenge missing-information rows such as `lens` and housing failures. In the quadcopter case, a reviewer would need to consolidate repeated `propellerGuard` rows and ask why major subsystems such as `flightController`, `sensorSuite`, `esc`, `motor`, and `propeller` are absent from the scored FMEA table.

Table 5.12 summarises representative interaction examples.

Table 5.12: Interactive review and revision examples

Case study	Interaction type	Example user request	Expected or observed result
Flashlight	Elicitation	"Why is `dcPwrOutPort` short circuit rated as high risk?"	The system can point to the severe effect: excessive current, overheating, possible battery damage, and safety hazard.
Flashlight	Revision	"Replace missing-information rows for the housing with concrete mechanical failure modes."	The table should be revised using additional model context or user-provided knowledge about material, loading, impact, ingress, and assembly.
Quadcopter	Elicitation	"Which rows relate to propeller-guard safety?"	The system can retrieve the high-risk `propellerGuard` rows, including structural damage, breakage, detachment, loose fitting, interference, and material degradation.
Quadcopter	Revision	"Merge duplicate propeller-guard rows and keep the highest justified score."	The table should consolidate overlapping rows, harmonise scores, and preserve the rationale for the selected final row.
Quadcopter	Revision	"Generate missing FMEA rows for the flight controller, sensor suite, ESCs, motors, propellers, and telemetry radio."	The framework should reuse the existing parser output and ontology guidance to expand coverage beyond the frame-body subset.

The interaction results are important because they show whether the generated FMEA table is a static document or a maintainable analysis artefact. A maintainable output should allow the user to inspect the reasoning, challenge weak rows, correct mistakes, merge duplicates, add missing coverage, and update the table without rebuilding the entire analysis manually.

5.10 Identified Framework Improvements

The flashlight and quadcopter results identify several concrete improvements to the framework. These improvements are grouped into parser improvements, ontology improvements, AI reasoning improvements, evidence-linking improvements, interactive-revision improvements, and evaluation improvements.

The most visible parser issue in the flashlight case is that two false part records, `to` and `connect`, were created from comment text around connection statements. This indicates that comment masking and declaration parsing should be strengthened. In the quadcopter case, no false part records were observed, but the related-context fields became long and repetitive. This suggests that the parser should not only extract context, but also rank and filter it according to the analysis task. For example, a part-level FMEA row for `propellerGuard` should receive the propeller-guarding requirement and nearby frame-body context, but it does not need the full requirement hierarchy repeated in every prompt.

The ontology was useful for guiding general failure-mode generation, especially for electrical, optical, and safety-related failure categories. However, both cases also show that ontology guidance alone is not enough. Missing-information rows appeared for passive or weakly described parts such as `lens`, `backHousing`, `middleHousing`, `arm`, `centerBody`, `landingGear`, and `payloadMount`. This indicates that the ontology needs stronger domain-specific extensions or better fallback question templates for passive structural components, optical components, and mechanical frame elements. For the quadcopter case, more specific control, sensing, propulsion, battery, and telemetry failure knowledge would be needed to produce a more complete FMEA.

The AI reasoning stage also requires improvement. The quadcopter results show that group-level and global-level review can split broad rows into more specific rows, but the workflow did not fully remove near-duplicates. Several `propellerGuard` rows reappeared with slightly different wording and sometimes different scores. This creates maintainability problems because the reviewer must decide which row to keep and which score is justified. Future prompts should therefore include explicit duplicate-checking, score-harmonisation, and row-consolidation instructions.

Evidence linking is another important improvement area. Both case studies include rationale and assumptions, but they do not yet include formal evidence references, requirement identifiers, parser-record identifiers, or ontology-node identifiers in the final FMEA rows. This limits traceability. Future versions should include fields such as `source_part`, `source_requirement`, `source_connection`, `ontology_concept`, and `evidence_status`. This would make it easier to inspect why a row was generated and whether it is supported by the model or by an assumption.

Interactive revision should also become more systematic. The results show that human review is not optional: the generated tables contain missing-information rows, incomplete coverage, and repeated rows. A practical implementation should allow the reviewer to merge duplicates, request missing subsystem coverage, change scores, attach evidence, and mark rows as accepted, rejected, or requiring further information.

Finally, the evaluation method should be extended. The current evaluation is based on controlled toy models and manual inspection by the author. This is sufficient for demonstrating the framework logic, but not for proving industrial validity. Future work should include expert review, larger case studies, comparison with human-created FMEA tables, and measurement of time saved or coverage improved.

Table 5.13 summarises the identified improvements.

Table 5.13: Identified framework improvements

Improvement area	Observed issue	Possible improvement
Model parser	Flashlight parsing produced two false records from comment text; quadcopter context extraction became verbose and repetitive.	Improve comment masking, declaration parsing, context filtering, and task-specific context ranking.
Ontology	Missing-information rows appeared for passive optical, housing, frame, landing, and payload-mount parts.	Add domain-specific extensions and fallback question templates for mechanical, optical, frame, landing, propulsion, sensing, and control elements.
AI-supported reasoning	Quadcopter global review split some broad rows but did not fully consolidate duplicates.	Add explicit duplicate detection, row merging, score harmonisation, and coverage-checking instructions.
Evidence linking	Rows contain rationale and assumptions, but not formal links to source requirements, connections, parser records, or ontology nodes.	Add explicit trace fields for source part, source requirement, source connection, ontology concept, evidence item, and evidence status.
Interactive revision	Human review is needed to remove duplicates, correct scores, and add missing subsystem coverage.	Support reviewer commands for merging, accepting, rejecting, revising, and expanding FMEA rows.
Evaluation method	The current evaluation uses controlled toy models and author inspection.	Add expert review, larger models, comparison with human FMEA tables, and quantitative measures of coverage and revision effort.

These improvements provide a bridge from the results chapter to the discussion and conclusion. They show that the framework is not presented as a finished industrial tool, but as a research artefact that demonstrates how model representation, parsing, ontology-based guidance, AI-supported reasoning, and human review can be combined to support model integrity.

5.11 Summary of Results

This chapter applied the proposed framework to two SysML v2-style case study models. The flashlight model was used as a compact baseline case, while the quadcopter model was used as a richer case with more requirements, actions, ports, connections, repeated components, and safety-related concerns.

The flashlight parser produced 13 raw records, of which 11 were accepted as intended SysML model elements. The two false records, `to` and `connect`, show that parser robustness remains an important issue. Nevertheless, the parser successfully extracted the main parts, actions, ports, connections, bindings, requirements, and allocations needed to create bounded model context for downstream analysis. The flashlight FMEA workflow then produced 12 group-level candidate rows and 13 globally reviewed and scored rows. The strongest rows concerned the `dcPwrOutPort` and reflector. The weaker rows concerned passive or weakly described elements such as the lens and housing parts.

The quadcopter parser produced 20 accepted model-element records and no false part records. This shows that the parser can handle a richer SysML v2-style model with more elements and relationships. However, the generated FMEA output did not cover the full parsed model. The quadcopter FMEA workflow produced 24 scored rows, but these rows focused mainly on the `propellerGuard` and other frame-body elements. The strongest rows were safety-related, especially propeller-guard detachment, breakage, structural damage, loose fitting, interference with propeller rotation, and material degradation. The highest quadcopter RPN was 400 for propeller-guard detachment. At the same time, major model elements such as the battery, power distribution board, receiver, flight controller, sensor suite, ESCs, motors, propellers, thrust mixer, and telemetry radio were not represented in the scored output.

The verification and validation results therefore support a balanced conclusion. The framework can transform parsed SysML model context and ontology guidance into structured, reviewable FMEA rows with complete S/O/D scoring. However, the results also show that this is not yet sufficient for a complete engineering FMEA. The main limitations are incomplete model coverage, lack of formal evidence links, missing explicit requirement identifiers, repeated or overlapping rows, and dependence on assumptions where model context is weak.

Overall, the results support the thesis argument that model integrity matters for downstream analysis. When model elements, functions, interfaces, and requirements are explicit and passed into the reasoning workflow, the generated FMEA rows become more specific and reviewable. When context is missing, too narrow, or not consolidated properly, the resulting FMEA becomes incomplete, repetitive, or assumption-heavy. The findings from this chapter provide the basis for the final discussion and conclusion in Chapter 6.

Chapter 6

Conclusion

This thesis addressed the problem of poor model integrity in Digital Engineering environments. The central motivation was that engineering models are increasingly expected to support downstream analysis, decision-making, verification, and review. However, the existence of models alone does not guarantee that the information inside them is reliable, traceable, current, consistent, or usable. When model information is incomplete, weakly connected, or difficult to inspect, downstream analysis becomes slower, harder to justify, and more difficult to maintain after changes.

To address this problem, the thesis developed a model integrity framework that combines four main elements: SysML v2-style model representation, model parsing, ontology-based reasoning guidance, and AI-supported reasoning. The SysML v2-style model representation provides the system-specific engineering information. The model parser transforms the textual model representation into structured and bounded context. The ontology provides domain-agnostic reasoning guidance for Failure Mode and Effects Analysis (FMEA). The AI component then uses both the parser output and the ontology to support the generation, review, scoring, explanation, and revision of the failure analysis.

The framework was demonstrated through two SysML v2-style case studies. The flashlight model was used as a simple and inspectable baseline case, while the quadcopter model was used as a more complex case with richer subsystem structure, repeated elements, sensing, control, propulsion, safety, and telemetry concerns. FMEA was used as the downstream failure analysis task because it depends strongly on connected engineering information and exposes weaknesses in model coverage, traceability, and reasoning structure. FMEA is also a labour-intensive engineering activity, so it is a suitable example of a downstream task that could benefit from structured model context, semantic guidance, and controlled AI support.

The thesis does not claim that the proposed framework solves all model integrity problems in Digital Engineering. It also does not claim that AI can replace engineering judgement. Instead, the thesis shows how AI can be positioned as one supporting method within a broader model integrity workflow. The main result is the demonstration of how model-based context, ontology-guided reasoning, AI-supported assistance, and human review can be combined to make downstream failure analysis more structured, reviewable, and traceable.

6.1 Answer to the Research Questions

This thesis was guided by two research questions.

RQ1: How can existing tools and methods, including descriptive modelling notations, semantic representations, and artificial intelligence, be organised into a practical framework for improving model integrity, with Failure Mode and Effects Analysis used as the demonstration case?

The thesis answered this question by proposing a framework in which each method has a distinct role. The SysML v2-style model representation provides the system-specific engineering information. The model parser extracts relevant information from the model and turns it into structured context. The ontology provides a domain-agnostic reasoning structure for FMEA. The AI component combines the parsed model context and ontology guidance to support downstream failure analysis.

This organisation is important because it separates two different types of knowledge. The parser provides what the AI should reason about: the system-specific parts, functions, ports, connections, actions, requirements, and allocations. The ontology provides how the AI should reason: the FMEA concepts, failure logic, cause–effect relations, controls, ratings, evidence expectations, and review questions. This separation reduces the risk of asking the AI to reason directly from one large model text without task-specific structure.

The framework therefore organises existing methods into complementary roles rather than treating them as competing solutions. Descriptive modelling provides the source information. Parsing makes that information usable for the AI-supported reasoning task. The ontology provides semantic and procedural guidance. AI supports selected reasoning tasks. Human review preserves engineering judgement and accountability.

RQ2: To what extent does the proposed framework support the reliability, traceability, and maintainability of downstream engineering analysis, and how can the framework be improved, evaluated through FMEA as a case study?

The case-study results show that the framework can support downstream failure analysis, but only partially and under controlled conditions. In the flashlight case, the parser produced 13 raw records, of which 11 were accepted as intended model-element records. The two rejected records were parser artefacts caused by comment text around connection statements. The FMEA generation and scoring process then produced a set of scored rows that could be inspected, including failures related to the power output port and optical reflector. This indicates that the framework can produce structured and reviewable FMEA content for a compact model.

The quadcopter case showed both the potential and the limitations of the approach. The parser extracted 20 accepted model-element records without false part records, which indicates that the parser can handle a larger SysML v2-style model. However, the generated and scored FMEA output mainly covered the frame-body and propeller-guard subset rather than the full quadcopter model. This means that the framework supported detailed reasoning for some safety-relevant elements, but did not yet provide sufficient coverage across all important subsystems, such as sensing, flight control, power distribution, propulsion, and telemetry. This is an important result because it shows that model parsing alone is not enough; coverage control and task orchestration are also needed.

The verification and validation results therefore lead to a mixed but useful conclusion. The framework performed well in making generated rows structurally explicit, separating failure modes from effects, assigning severity–occurrence–detection scores, and producing reviewable assumptions and recommended actions for the generated rows. It performed less well in guaranteeing complete model coverage, avoiding repeated or near-duplicate candidate rows before review, linking every claim to strong evidence, and ensuring that all model elements were carried through to the final FMEA output. The best-supported aspects were structure, reviewability, and scoring completeness for covered rows. The weakest aspects

were coverage, evidence binding, and scalability of the generation process.

The framework should therefore be understood as a research-grade demonstration of how model integrity can be supported, not as a finished industrial tool. The results support the claim that the combination of SysML v2-style model context, parsing, ontology guidance, and AI-supported reasoning can improve the structure and reviewability of downstream failure analysis. They do not yet prove full industrial reliability or completeness.

6.2 Main Findings

The first main finding is that model representation alone is not sufficient for downstream failure analysis. A SysML v2-style model can contain useful engineering information, but this information must be extracted, bounded, and organised before it can effectively support AI-assisted FMEA generation. In the case studies, the parser made parts, parent–child relations, actions, connections, requirements, and allocations easier to inspect and reuse. However, the study did not include a separate no-parser baseline experiment. Therefore, the conclusion is not that parser use has been quantitatively proven against a raw-model baseline, but that the observed results depend strongly on having structured parser output available to the AI component.

The second main finding is that parser quality directly affects downstream analysis quality. In the flashlight case, two false parser records were produced from comment text around connection statements. These artefacts did not represent real model elements and therefore had to be excluded from the accepted parser output. This shows that even a small model can create integrity problems if comment handling and declaration parsing are not robust. In the quadcopter case, no false part records were observed, but the parser produced very long related-context fields for some records. This made the context traceable but also verbose, showing a trade-off between completeness and boundedness.

The third main finding is that the ontology provides a useful reasoning frame, but it does not replace system-specific or domain-specific knowledge. The ontology helped structure the FMEA task around concepts such as function, failure mode, effect, cause, control, risk rating, recommended action, and evidence. It also supported the generation of safety-relevant failure modes for the propeller guard in the quadcopter case. However, several outputs also showed “no relevant failure modes found” or similar placeholder results for components such as housings, arms, landing gear, or payload mount. This indicates that a domain-agnostic ontology is useful for general reasoning structure, but may need domain-specific extensions to improve coverage and technical precision.

The fourth main finding is that review and revision are essential parts of the workflow. The group-level and global reviews helped normalise candidate rows, split combined failure modes where appropriate, and expose duplicated or weak outputs. These reviews should not be understood as manual invention of new rows, but as structured post-processing of generated candidates. The interactive review concept adds a further layer in which a human reviewer can ask why a row was produced, request score changes, challenge assumptions, or ask for missing failure modes to be considered. This supports maintainability because the FMEA table can be treated as a revisable analysis artefact rather than a static AI output.

The fifth main finding is that model complexity increases the difficulty of maintaining coverage. The flashlight model is compact and easier to inspect manually. The quadcopter model introduces more subsystems, repeated components, more interfaces, more safety-related requirements, and more functional dependencies. The parser handled the richer model structure, but the FMEA generation did not cover the full model evenly. This shows that fu-

ture versions of the framework require stronger coverage tracking, such as explicit checks that each relevant part, function, interface, requirement, and safety concern has been considered.

The final finding is that human engineering judgement remains necessary. AI-supported reasoning can help generate, score, explain, and revise candidate FMEA rows, but it cannot be treated as automatically correct. Risk ratings, assumptions, recommended actions, and evidence links still require human inspection. This is especially important in FMEA because severity, occurrence, detection, and residual risk involve engineering judgement and may have safety or regulatory implications.

6.3 Implications for Digital Engineering

The findings have several implications for Digital Engineering.

First, Digital Engineering requires more than digitised documents or isolated models. The value of a Digital Engineering environment depends on whether model information can be connected, interpreted, reused, and maintained over time. Model integrity is therefore not a minor modelling concern. It is central to whether models can support downstream analysis and decision-making.

Second, AI should not be treated as a shortcut around model quality. If the model information is incomplete, ambiguous, or weakly connected, AI may produce outputs that appear convincing but are poorly grounded. This thesis supports the view that AI is most useful when it is bounded by structured model context, guided by explicit semantic reasoning, linked to evidence, and reviewed by humans.

Third, ontologies and graph-based representations can help make reasoning structures more explicit. They are especially useful when AI systems need to reason consistently across concepts such as function, failure mode, cause, effect, control, risk, action, and evidence. However, ontologies also require maintenance, governance, and domain adaptation. They should be treated as engineering artefacts that must be designed, reviewed, and updated, not as automatic solutions to model integrity problems.

Fourth, FMEA is a useful evaluation surface for model integrity because it turns hidden model weaknesses into visible analysis problems. If functions, interfaces, requirements, or evidence links are missing, the FMEA table will often expose those weaknesses through incomplete rows, unsupported causes, weak effects, unclear scores, or placeholder entries. This makes FMEA useful not only as a safety and reliability method, but also as a diagnostic task for evaluating whether model information is suitable for downstream reasoning.

6.4 Limitations

This thesis has several limitations.

First, the framework was demonstrated only on two toy SysML v2-style models. The flashlight and quadcopter models are useful for controlled evaluation, but they do not represent the full complexity of industrial Digital Engineering environments.

Second, the parser is designed for the textual SysML v2-style model structure used in this thesis. It is not claimed to support every possible SysML v2 syntax, SysML v2 implementation, modelling convention, or tool export format. More robust parsing would be required

for industrial deployment.

Third, the framework was not evaluated against a separate no-parser baseline. The results show that the parser output was useful for the demonstrated workflow, but they do not quantify how much better the framework performs compared with direct AI processing of raw model text.

Fourth, the ontology is domain-agnostic. This makes it reusable across different systems, but it also means that it does not contain detailed domain-specific failure physics. For example, the quadcopter case could benefit from more specialised knowledge about batteries, flight control, propulsion, sensors, frame structures, and safety-critical control systems.

Fifth, AI-supported outputs are not guaranteed to be correct. The AI can support generation, review, explanation, scoring, and revision, but its outputs still require human review. This is especially important for FMEA because risk assessment involves engineering judgement.

Sixth, the validation in this thesis is analytical and case-study-based. The framework was not validated in a real industrial project, with practising engineers, or in an operational safety-critical environment. Therefore, the results should be interpreted as evidence that the framework is promising, not as proof of general industrial validity.

Finally, the evaluation does not compare the generated FMEA tables against a full expert-created reference FMEA. Such a comparison would be valuable for assessing completeness, correctness, and practical usefulness more rigorously.

6.5 Future Work

Several directions for future work follow from these limitations.

First, the framework should be tested on larger and more realistic industrial models. This would help evaluate whether the approach remains useful when models contain many requirements, subsystems, variants, interfaces, simulations, and verification artefacts.

Second, the model parser should be extended to handle more complete SysML v2 syntax and different modelling tool exports. A stronger parser could support more robust extraction of actions, states, allocations, constraints, ports, flows, requirements, comments, and nested declarations.

Third, future work should include an ablation study comparing the proposed parser-based approach with direct AI processing of raw model text. This would make it possible to evaluate the specific contribution of the parser to output quality, coverage, traceability, and reviewability.

Fourth, the ontology should be refined into a cleaner and more formal representation. Future work could distinguish more clearly between classes, instances, relations, rules, question templates, and evidence types. A more formal ontology could also support stronger consistency checking.

Fifth, domain-specific ontology extensions should be developed. For example, separate extensions could be created for electrical systems, control systems, software-intensive systems, autonomous systems, and safety-critical systems. These extensions would allow the framework to combine domain-agnostic reasoning with domain-specific failure knowledge.

Sixth, coverage control should be strengthened. Future versions of the framework should explicitly track whether each relevant part, function, interface, requirement, operating mode, and safety concern has been considered in the generated FMEA. This is especially important

for larger systems, as shown by the quadcopter case.

Seventh, evidence linking should be strengthened. Future versions of the framework should connect each generated FMEA row to specific model elements, assumptions, requirements, test results, simulation outputs, or expert rationale. This would improve traceability and reduce unsupported claims.

Eighth, the AI prompting and retrieval strategy should be improved. Future work should investigate stricter prompts, better GraphRAG retrieval, uncertainty marking, assumption tracking, duplicate detection, and automated checks for unsupported statements.

Ninth, the framework should be evaluated with domain experts or practising engineers. Expert review would help assess whether the generated outputs are useful, understandable, and trustworthy in real engineering workflows.

Tenth, future work should compare framework-generated FMEA tables against human-created FMEA tables. This would allow a more rigorous assessment of completeness, quality, effort reduction, and practical value.

Finally, the framework could be extended beyond FMEA. The same model integrity approach may support other downstream engineering tasks, such as verification planning, impact analysis, compliance checking, trade-off analysis, safety analysis, and design review.

6.6 Final Reflection

This thesis shows that improving model integrity requires coordination between modelling, semantic structure, AI support, and human judgement. A model alone is not enough. An ontology alone is not enough. AI alone is not enough. The value lies in the controlled combination of these elements.

The proposed framework demonstrates one practical way to make model-based information more structured, bounded, explainable, and reviewable for downstream failure analysis. By combining SysML v2-style model representation, parsing, ontology-based guidance, AI-supported reasoning, and human review, the framework provides a step toward more trustworthy use of model information in Digital Engineering.

The main lesson is that AI can support model integrity only when it is grounded in structured model context and guided by explicit reasoning structures. In this thesis, FMEA served as the demonstration task, but the broader contribution is the framework logic: system-specific model context tells the AI what to reason about, while the ontology tells it how to reason. This separation provides a foundation for more reliable, traceable, maintainable, and usable downstream engineering analysis.

Bibliography

[1] João Paulo A. Almeida, Luís Ferreira Pires, Giancarlo Guizzardi, and Gerd Wagner. An analysis of the semantic foundation of KerML and SysML v2. In *Conceptual Modeling*, pages 133–151. Springer, Cham, 2024. 11

[2] Automotive Industry Action Group and Verband der Automobilindustrie. *AIAG & VDA FMEA Handbook: Failure Mode and Effects Analysis*. AIAG and VDA, 1 edition, 2019. 22, 25, 26, 27, 29, 35

[3] Manas Bajaj, Sanford Friedenthal, and Ed Seidewitz. Systems modeling language SysML v2 support for digital engineering. *INSIGHT*, 25(1):19–24, 2022. 11

[4] Mary Bone, Mark Blackburn, Benjamin Kruse, John Dzielski, Thomas Hagedorn, and Ian Grosse. Toward an interoperability and integration framework to enable the digital thread. *Systems*, 6(4):46, 2018. 13

[5] Robert Buchmann, Johann Eder, Hans-Georg Fill, Ulrich Frank, Dimitris Karagiannis, Emanuele Laurenzi, John Mylopoulos, Dimitris Plexousakis, and Maribel Yasmina Santos. Large language models: Expectations for semantics-driven systems engineering. *Data & Knowledge Engineering*, 152:102324, 2024. 17, 19

[6] Pierre de Saqui-Sannes, Rob A. Vingerhoeds, Christophe Garion, and Xavier Thirioux. A taxonomy of MBSE approaches by languages, tools and methods. *IEEE Access*, 10:120936–120950, 2022. 7, 9, 10, 11, 19

[7] Daniel Dunbar, Thomas Hagedorn, Mark Blackburn, John Dzielski, Steven Hespelt, Benjamin Kruse, Dinesh Verma, and Zhongyuan Yu. Driving digital engineering integration and interoperability through semantic integration of models with ontologies. *Systems Engineering*, 26(4):365–378, 2023. 13, 15, 16, 17, 19

[8] Daniel Dunbar, Thomas Hagedorn, Mark Blackburn, and Dinesh Verma. A three-pronged verification approach to higher-level verification using graph data structures. *Systems*, 12(1):27, 2024. 13, 15, 19

[9] Christian Ellsel, Andreas Vogelsang, Daniel Mendez, Henning Femmer, and Eric Knauss. Advancing requirements engineering with large language models. *Procedia Computer Science*, 2025. Publisher metadata indicates an article on LLM-supported requirement reformulation and quality assessment in practically relevant workflows. 17

[10] Jeff A. Estefan. Survey of model-based systems engineering methodologies. Technical report, INCOSE, 2008. Rev. B. 7, 9, 10, 18

[11] Alessio Ferrari et al. Formal requirements engineering and large language models: A two-way roadmap. *Information and Software Technology*, 177:107697, 2025. 17, 19

[12] Sanford Friedenthal, Alan Moore, and Rick Steiner. *Systems Modeling Language (SysML) Tutorial*. Object Management Group and International Council on Systems Engineering, 2008. Revision B, tutorial dated 15 April 2008. 10, 11

[13] Thomas R. Gruber. A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2):199–220, June 1993. 14, 19

[14] Nicola Guarino, Daniel Oberle, and Steffen Staab. What is an ontology? In Steffen Staab and Rudi Studer, editors, *Handbook on Ontologies*, pages 1–17. Springer, Berlin, Heidelberg, 2009. 15, 16, 19

[15] Thomas D. Jr. Hedberg, Manas Bajaj, and Jaime A. Camelio. Using graphs to link data across the product lifecycle for enabling smart manufacturing digital threads. *Journal of Computing and Information Science in Engineering*, 20(1):011011, 2020. 8, 15, 16

[16] Melinda Hodkiewicz, Gloria Ho, Christiane Howard, Suzanne Robinson, and Thomas Kelly. An ontology for reasoning over engineering textual data stored in fmea spreadsheet tables. *Information Processing & Management*, 58(6):102701, 2021. 31

[17] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d'Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, Jose Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan F. Sequeda, Steffen Staab, and Antoine Zimmermann. Knowledge graphs. *ACM Computing Surveys*, 54(4):1–37, 2021. 15, 17, 19

[18] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Computing Surveys*, 2025. 17, 19

[19] Zhaofeng Huang, Robert Hansen, and Zhaofeng Carl Huang. Toward fmea and mbse integration. In *Proceedings of the 2018 Annual Reliability and Maintainability Symposium*, pages 1–7, 2018. 31

[20] Zhaofeng Huang, Steve Swalgen, Heidi Davidz, and John Murray. MBSE-assisted FMEA approach—challenges and opportunities. In *Proceedings of the Annual Reliability and Maintainability Symposium*, pages 1–8, 2017. 31

[21] Tomas Huldt and Ivan Stenius. State-of-practice survey of model-based systems engineering. *Systems Engineering*, 22(2):134–145, 2019. 9, 10, 19

[22] International Electrotechnical Commission. IEC 60812:2018 failure modes and effects analysis FMEA and FMECA, 2018. 19, 21, 22, 23, 25, 35

[23] Artjoms Korsunovs, Irem Y. Tumer, Felician Campean, Jakub Burek, and Michael Henshaw. Towards a model-based systems engineering approach for robotic manufacturing process modelling with automatic fmea generation. In *Proceedings of the Design Society*, volume 2, pages 853–862, 2022. 31

[24] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020. 17, 19

[25] Junda Ma, Guoxin Wang, Jinzhi Lu, Hans Vangheluwe, Dimitris Kiritsis, and Yan Yan. Systematic literature review of MBSE tool-chains. *Applied Sciences*, 12(7):3431, 2022. 9, 10, 19

[26] Azad M. Madni and Michael Sievers. Model-based systems engineering: Motivation, current status, and research opportunities. *Systems Engineering*, 21(3):172–190, 2018. 1, 7, 9, 10, 11, 13, 18

[27] Natalya F. Noy and Deborah L. McGuinness. Ontology development 101: A guide to creating your first ontology. Technical Report KSL-01-05, Stanford Knowledge Systems Laboratory, 2001. 15, 16

[28] Object Management Group. OMG Systems Modeling Language (SysML) Version 1.7. Formal specification, June 2024. 10, 11, 18

[29] Object Management Group. Kernel modeling language KerML version 1.0 specification. Technical report, Object Management Group, Needham, MA, 2025. 11, 18

[30] Object Management Group. OMG systems modeling language SysML version 2.0 specification. Technical report, Object Management Group, Needham, MA, 2025. 10, 11, 18

[31] Object Management Group. Systems Modeling API and Services Version 1.0. Formal specification, September 2025. 11, 18

[32] SAE International. ARP5580: Recommended Failure Modes and Effects Analysis (FMEA) Practices for Non-Automobile Applications, 2021. 25, 35

[33] SAE International. J1739: Potential Failure Mode and Effects Analysis in Design, Manufacturing and Assembly Processes, and Machinery, 2021. 25

[34] Victor Singh and Karen E. Willcox. Engineering design with digital thread. *AIAA Journal*, 56(11):4515–4528, 2018. 7, 8, 9, 18

[35] Claudio Spreafico, Davide Russo, and Catia Rizzi. A state-of-the-art review of FMEA/FMECA including patents. *Computer Science Review*, 25:19–28, 2017. 19, 31

[36] Fei Tao, Xiaoyu Ma, Wei Liu, and Cheng Zhang. Digital engineering: State-of-the-art and perspectives. *Digital Engineering*, 1:100007, 2024. 1

[37] U.S. Department of Defense. MIL-STD-1629A: Procedures for performing a failure mode, effects and criticality analysis. Technical report, Department of Defense, Washington, DC, 1980. 19, 21, 22, 23, 25, 29, 30, 35

[38] U.S. Department of Defense. Digital engineering strategy. Technical report, Department of Defense, Washington, DC, 2018. 1, 7, 8, 9, 18

[39] U.S. Department of Defense. DoD Data Strategy: Unleashing Data to Advance the National Defense Strategy. Technical report, U.S. Department of Defense, October 2020. 8, 9, 13

[40] U.S. Department of Defense. DoD Instruction 5000.97: Digital Engineering. Technical report, Department of Defense, December 2023. Issued 21 December 2023. 7, 8, 9, 13, 18

[41] Haytham Younus, Felician Campean, Sohag Kabir, Pascal Bonnaud, David Delaux, and Unal Yildirim. An ontological framework for the integration of system design and FMEA. *AI EDAM*, 2025. Advance online / issue details may vary. 31

[42] Haytham Younus, Sohag Kabir, Felician Campean, Pascal Bonnaud, and David Delaux. AI- and ontology-based enhancements to FMEA for advanced systems engineering: Current developments and future directions. *Applied Sciences*, 16(5):2464, 2026. 31

Appendix A

Toy Flashlight SysML Model

This appendix provides the textual SysML-style toy flashlight model used as the demonstration case in this thesis. The model includes requirements, actions, parts, ports, attributes, connections, states, and requirement allocations. It is used as the model representation input for the methodology described in Chapter 4.

```
package FlashlightSpecificationAndDesign {
    public import ScalarValues::*;
    public import SI::*;
    public import USCustomaryUnits::*;

    // create a Requirements package that contains the hierarchy of requirements that compose the
     flashlight specification
    package Requirements{
        requirement flashlightSpecification{
            requirement userInterface;
            requirement illumination{
                // the following is a simple text-based requirement
                requirement fieldOfView{
                    doc /* The flashlight field of view shall be 0 - 20 degrees.*/
                }
                requirement lightPower{
                    doc /* The light power shall be a minumum of TBD lumens.*/
                }
            }
            requirement physical{
                requirement portability;
                requirement size{
                    doc /* The flashlight shall be less than 6 inches in length.*/
                }
                // the following is a more precisely specified requirement using constraints
                requirement weight{
                    doc /* The weight shall be less than 0.25 lbs.*/
                    attribute actualWeight :> ISQ::mass;
                    require constraint {actualWeight <= 0.25 [lb]}
                }
            }
            requirement reliability;
            requirement cost;
        }
    }

    // create a package called ActionTree that contains the actions and flows to produceDirectedLight
    package ActionTree{
        action produceDirectedLight{
            in item onOffCmd; //input
            // the following output (i.e., lightOut) from produceDirectedLight is equal to (i.e.,
     bound to) the lightOut from the action directLight;
            out item lightOut = directLight.lightOut;

            // declare each nested action and their inputs and outputs
            action provideDCPwr{
                out item dcPwr;
            }
            action connectDCPwr {
```

```
            in item onOffCmd = produceDirectedLight.onOffCmd;
            in item dcPwrIn;
            out item dcPwrOut;
        }
        action generateLight{
            in item dcPwrIn;
            out item light;
        }
        action directLight{
            in item lightIn;
            out item lightOut;
        }

        // the following declares a fork node named fork1 that is succeeded by two parallel
 actions
        fork fork1;
            then provideDCPwr; // succession
            then connectDCPwr; // succession

        // define a control flow beginning with a start node and then fork1
        first start then fork1;

        // define input/output flow from the output of one action to the input of another
        flow from provideDCPwr.dcPwr to connectDCPwr.dcPwrIn;
        flow from connectDCPwr.dcPwrOut to generateLight.dcPwrIn;
        flow from generateLight.light to directLight.lightIn;
    }
}

// create a PartsTree package that contains the flashlight,
// along with its key features such as attributes, ports, actions, parts, and interconnections
package PartsTree{
    part flashlight {
        // flashlight attributes
        attribute mass :> ISQ::mass;
        attribute fov:Real;
        attribute illuminationLevel:Real;

        // flashlight ports
        port cmdPort;
        port lightOutPort;

        // flashlight perform action. The flashlight performs the action called
 produceDirectedLight
        perform ActionTree::produceDirectedLight;

        // the flashlight exhibits states: initial, off, and on
        exhibit state flashlightStates{
            // declare the states
            state initial;
            state off;
            state on{
                // define the do action by referring to the action above
                do produceDirectedLight;
            }

            // each state transition specifies start and end states
            transition initial then off;
            transition off_To_on
                first off
                then on;
            transition on_To_off
                first on
                then off;
        }

        // the battery is a referential part of the flashlight
        ref part battery [2]{
            attribute power:> ISQ::power;
            port dcPwrOutPort;
            perform produceDirectedLight.provideDCPwr;
        }

        // the following are composite parts of the flashlight
        part switch{
            port cmdPort;
            port inPort;
            port outPort;
            perform produceDirectedLight.connectDCPwr;
        }
```

```
            part lamp{
                attribute efficiency:Real;
                port dcPwrInPort;
                port lightOutPort;
                perform produceDirectedLight.generateLight;
            }

            part optics{
                port lightInPort;
                port lightOutPort;
                perform produceDirectedLight.directLight;

                part reflector{
                    attribute radius:> ISQ::length;
                }

                part lens;
            }

            part structure{
                part frontHousing;
                part middleHousing;
                part backHousing;
            }

            // binding connections
            bind cmdPort = switch.cmdPort;
            bind lightOutPort = optics.lightOutPort;

            // port connections
            connect battery.dcPwrOutPort to switch.inPort;
            connect switch.outPort to lamp.dcPwrInPort;
            connect lamp.lightOutPort to optics.lightInPort;
        }
    }

    // create a RequirementsAllocation package that contains the allocation of requirements to design
      features
    package RequirementsAllocation{
        public import Requirements::*;
        public import PartsTree::*;

        allocate flashlightSpecification to flashlight{
            allocate flashlightSpecification.physical.weight to flashlight.mass;
            allocate flashlightSpecification.illumination.fieldOfView to flashlight.fov;
            allocate flashlightSpecification.illumination.lightPower to flashlight.illuminationLevel;
        }
    }
}
```

Listing A.1: Toy flashlight SysML-style model used in the case study

Appendix B

Toy Quadcopter SysML Model

This appendix provides the textual SysML-style toy quadcopter model used as an additional demonstration model. The model includes requirements, actions, parts, ports, attributes, connections, states, and requirement allocations. It can be used to test whether the proposed framework scales from a simple flashlight system to a richer multi-subsystem model.

```
package QuadcopterSpecificationAndDesign {
    public import ScalarValues::*;
    public import SI::*;
    public import USCustomaryUnits::*;

    // create a Requirements package that contains the hierarchy of requirements
    // that compose the quadcopter specification
    package Requirements{
        requirement quadcopterSpecification{
            requirement userInterface{
                requirement commandInput{
                    doc /* The quadcopter shall accept pilot or autonomous flight commands through a
    command input interface.*/
                }
                requirement telemetry{
                    doc /* The quadcopter shall provide telemetry including battery state, attitude,
    altitude, and flight mode.*/
                }
            }

            requirement flightPerformance{
                requirement controlledFlight{
                    doc /* The quadcopter shall produce controlled vertical lift and directional
    motion.*/
                }
                requirement hover{
                    doc /* The quadcopter shall maintain stable hover in nominal operating conditions
    .*/
                }
                requirement flightTime{
                    doc /* The quadcopter shall operate for a minimum of TBD minutes on a fully
    charged battery.*/
                    attribute actualFlightTime :> ISQ::time;
                    require constraint {actualFlightTime >= 10 [min]}
                }
                requirement payload{
                    doc /* The quadcopter shall carry a payload of at least TBD mass.*/
                    attribute actualPayloadMass :> ISQ::mass;
                    require constraint {actualPayloadMass >= 0.10 [kg]}
                }
            }

            requirement physical{
                requirement portability;
                requirement size{
                    doc /* The quadcopter shall have a maximum diagonal motor-to-motor size of less
    than 45 centimeters.*/
                    attribute actualDiagonalSize :> ISQ::length;
                    require constraint {actualDiagonalSize <= 45 [cm]}
```

```
            }
            requirement weight{
                doc /* The quadcopter takeoff mass shall be less than 2.0 kg.*/
                attribute actualMass :> ISQ::mass;
                require constraint {actualMass <= 2.0 [kg]}
            }
        }

        requirement energy{
            requirement batteryVoltage{
                doc /* The quadcopter shall use a battery pack with nominal voltage suitable for
the motor and ESC system.*/
            }
            requirement powerDistribution{
                doc /* The quadcopter shall distribute electrical power from the battery to all
propulsion and control components.*/
            }
        }

        requirement sensingAndControl{
            requirement attitudeSensing{
                doc /* The quadcopter shall sense roll, pitch, and yaw attitude for flight
stabilization.*/
            }
            requirement altitudeSensing{
                doc /* The quadcopter shall estimate altitude for takeoff, landing, and hover
control.*/
            }
            requirement flightControl{
                doc /* The quadcopter shall compute motor commands from desired motion and sensed
 vehicle state.*/
            }
        }

        requirement safety{
            requirement failsafe{
                doc /* The quadcopter shall enter a safe state when communication, power, or
critical sensor faults are detected.*/
            }
            requirement lowBatteryProtection{
                doc /* The quadcopter shall warn the operator or initiate landing when battery
state is below a defined threshold.*/
            }
            requirement propellerGuarding{
                doc /* The quadcopter should reduce risk of injury from rotating propellers where
 practical.*/
            }
        }

        requirement reliability;
        requirement cost;
    }
}

// create a package called ActionTree that contains the actions and flows
// required to produce controlled flight
package ActionTree{
    action produceControlledFlight{
        in item flightCmd;
        out item telemetryOut;
        out item thrustOut = combineThrust.thrustOut;

        action provideElectricalPower{
            out item dcPwr;
        }

        action distributeElectricalPower{
            in item dcPwrIn;
            out item propulsionPwr;
            out item controlPwr;
        }

        action receiveFlightCommand{
            in item flightCmdIn = produceControlledFlight.flightCmd;
            out item desiredMotion;
        }

        action measureVehicleState{
            in item controlPwrIn;
            out item sensedState;
```

```
        }

        action computeFlightControl{
            in item desiredMotionIn;
            in item sensedStateIn;
            in item controlPwrIn;
            out item motorCmd;
            out item telemetry;
        }

        action modulateMotorPower{
            in item motorCmdIn;
            in item propulsionPwrIn;
            out item motorPwr;
        }

        action generateRotorThrust{
            in item motorPwrIn;
            out item rotorThrust;
        }

        action combineThrust{
            in item rotorThrustIn;
            out item thrustOut;
        }

        action reportTelemetry{
            in item telemetryIn = computeFlightControl.telemetry;
            out item telemetryOut = produceControlledFlight.telemetryOut;
        }

        fork fork1;
            then provideElectricalPower;
            then receiveFlightCommand;

        first start then fork1;

        flow from provideElectricalPower.dcPwr to distributeElectricalPower.dcPwrIn;
        flow from distributeElectricalPower.controlPwr to measureVehicleState.controlPwrIn;
        flow from distributeElectricalPower.controlPwr to computeFlightControl.controlPwrIn;
        flow from distributeElectricalPower.propulsionPwr to modulateMotorPower.propulsionPwrIn;

        flow from receiveFlightCommand.desiredMotion to computeFlightControl.desiredMotionIn;
        flow from measureVehicleState.sensedState to computeFlightControl.sensedStateIn;

        flow from computeFlightControl.motorCmd to modulateMotorPower.motorCmdIn;
        flow from modulateMotorPower.motorPwr to generateRotorThrust.motorPwrIn;
        flow from generateRotorThrust.rotorThrust to combineThrust.rotorThrustIn;

        flow from computeFlightControl.telemetry to reportTelemetry.telemetryIn;
    }
}

// create a PartsTree package that contains the quadcopter,
// its key features such as attributes, ports, and actions,
// its parts, and its part interconnections
package PartsTree{
    part quadcopter {
        // quadcopter attributes
        attribute mass :> ISQ::mass;
        attribute diagonalSize :> ISQ::length;
        attribute payloadMass :> ISQ::mass;
        attribute flightTime :> ISQ::time;
        attribute batteryStateOfCharge : Real;
        attribute hoverStability : Real;

        // quadcopter ports
        port cmdPort;
        port telemetryPort;
        port thrustOutPort;

        perform ActionTree::produceControlledFlight;

        // the quadcopter exhibits states for basic operating modes
        exhibit state quadcopterStates{
            state initial;
            state off;
            state armed;
            state flying{
                do produceControlledFlight;
```

```
        }
        state landing{
            do produceControlledFlight;
        }
        state fault;

        transition initial then off;

        transition off_To_armed
            first off
            then armed;

        transition armed_To_flying
            first armed
            then flying;

        transition flying_To_landing
            first flying
            then landing;

        transition landing_To_off
            first landing
            then off;

        transition flying_To_fault
            first flying
            then fault;

        transition fault_To_off
            first fault
            then off;
    }

    // the battery is a referential part of the quadcopter
    ref part battery [1]{
        attribute capacity :> ISQ::electricCharge;
        attribute voltage :> ISQ::electricPotential;
        attribute power :> ISQ::power;
        port dcPwrOutPort;
        perform produceControlledFlight.provideElectricalPower;
    }

    // composite parts of the quadcopter
    part powerDistributionBoard{
        port dcPwrInPort;
        port propulsionPwrOutPort;
        port controlPwrOutPort;
        perform produceControlledFlight.distributeElectricalPower;
    }

    part receiver{
        port cmdInPort;
        port desiredMotionOutPort;
        perform produceControlledFlight.receiveFlightCommand;
    }

    part flightController{
        attribute controllerRate : Real;
        port desiredMotionInPort;
        port sensedStateInPort;
        port controlPwrInPort;
        port motorCmdOutPort;
        port telemetryOutPort;
        perform produceControlledFlight.computeFlightControl;
    }

    part sensorSuite{
        port controlPwrInPort;
        port sensedStateOutPort;
        perform produceControlledFlight.measureVehicleState;

        part imu{
            attribute sampleRate : Real;
            port attitudeOutPort;
        }

        part barometer{
            attribute altitudeEstimate :> ISQ::length;
            port altitudeOutPort;
        }
```

```
            part gps{
                attribute positionAccuracy :> ISQ::length;
                port positionOutPort;
            }
        }

        // four electronic speed controllers
        part esc [4]{
            port motorCmdInPort;
            port propulsionPwrInPort;
            port motorPwrOutPort;
            perform produceControlledFlight.modulateMotorPower;
        }

        // four motors
        part motor [4]{
            attribute maxPower :> ISQ::power;
            attribute rotationalSpeed : Real;
            port motorPwrInPort;
            port shaftOutPort;
        }

        // four propellers generate rotor thrust
        part propeller [4]{
            attribute diameter :> ISQ::length;
            attribute pitch :> ISQ::length;
            port shaftInPort;
            port rotorThrustOutPort;
            perform produceControlledFlight.generateRotorThrust;
        }

        part thrustMixer{
            port rotorThrustInPort;
            port thrustOutPort;
            perform produceControlledFlight.combineThrust;
        }

        part telemetryRadio{
            port telemetryInPort;
            port telemetryOutPort;
            perform produceControlledFlight.reportTelemetry;
        }

        part frameBody{
            part centerBody;
            part arm [4];
            part landingGear;
            part payloadMount;
            part propellerGuard [4];
        }

        // binding connections assert that external quadcopter ports
        // are the same as selected internal subsystem ports
        bind cmdPort = receiver.cmdInPort;
        bind telemetryPort = telemetryRadio.telemetryOutPort;
        bind thrustOutPort = thrustMixer.thrustOutPort;

        // power connections
        connect battery.dcPwrOutPort to powerDistributionBoard.dcPwrInPort;
        connect powerDistributionBoard.controlPwrOutPort to sensorSuite.controlPwrInPort;
        connect powerDistributionBoard.controlPwrOutPort to flightController.controlPwrInPort;
        connect powerDistributionBoard.propulsionPwrOutPort to esc.propulsionPwrInPort;

        // command and control connections
        connect receiver.desiredMotionOutPort to flightController.desiredMotionInPort;
        connect sensorSuite.sensedStateOutPort to flightController.sensedStateInPort;
        connect flightController.motorCmdOutPort to esc.motorCmdInPort;

        // propulsion chain connections
        connect esc.motorPwrOutPort to motor.motorPwrInPort;
        connect motor.shaftOutPort to propeller.shaftInPort;
        connect propeller.rotorThrustOutPort to thrustMixer.rotorThrustInPort;

        // telemetry connections
        connect flightController.telemetryOutPort to telemetryRadio.telemetryInPort;
    }
}

// create a RequirementsAllocation package that contains the allocation of
```

```
    // requirements to the quadcopter design features
    package RequirementsAllocation{
        public import Requirements::*;
        public import PartsTree::*;

        allocate quadcopterSpecification to quadcopter{
            allocate quadcopterSpecification.physical.weight to quadcopter.mass;
            allocate quadcopterSpecification.physical.size to quadcopter.diagonalSize;
            allocate quadcopterSpecification.flightPerformance.flightTime to quadcopter.flightTime;
            allocate quadcopterSpecification.flightPerformance.payload to quadcopter.payloadMass;
            allocate quadcopterSpecification.flightPerformance.hover to quadcopter.hoverStability;

            allocate quadcopterSpecification.userInterface.commandInput to quadcopter.cmdPort;
            allocate quadcopterSpecification.userInterface.telemetry to quadcopter.telemetryPort;

            allocate quadcopterSpecification.energy.batteryVoltage to quadcopter.battery.voltage;
            allocate quadcopterSpecification.energy.powerDistribution to quadcopter.
    powerDistributionBoard;

            allocate quadcopterSpecification.sensingAndControl.attitudeSensing to quadcopter.
    sensorSuite.imu;
            allocate quadcopterSpecification.sensingAndControl.altitudeSensing to quadcopter.
    sensorSuite.barometer;
            allocate quadcopterSpecification.sensingAndControl.flightControl to quadcopter.
    flightController;

            allocate quadcopterSpecification.safety.failsafe to quadcopter.quadcopterStates.fault;
            allocate quadcopterSpecification.safety.lowBatteryProtection to quadcopter.
    batteryStateOfCharge;
            allocate quadcopterSpecification.safety.propellerGuarding to quadcopter.frameBody.
    propellerGuard;
        }
    }
}
```

Listing B.1: Toy quadcopter SysML-style model

Appendix C

Domain-Agnostic FMEA Ontology

This appendix describes the domain-agnostic Failure Mode and Effects Analysis (FMEA) ontology used in this thesis. The ontology is represented as a JSON-based graph structure and is used as a semantic reasoning input for the AI-supported failure analysis component described in Chapter 4. Its purpose is to provide the AI component with a reusable reasoning structure for FMEA, while the parsed SysML model provides the system-specific context.

The ontology should not be interpreted as a complete formal ontology in the strict OWL sense. Instead, it is a GraphRAG-ready ontology representation. It contains concepts, relationships, reasoning rules, question templates, and standards-based references that can be retrieved and supplied to the AI during FMEA generation, review, explanation, and revision.

C.1 Purpose of the Ontology

The ontology was created to support domain-agnostic FMEA reasoning. This means that it is not specific to the flashlight model, the quadcopter model, or any single engineering domain. Instead, it provides general concepts and reasoning structures that can be applied across mechanical, electrical, electronic, software, control, thermal, fluidic, chemical, material, human, process, interface, environmental, cybersecurity, data/AI, maintenance, and supply-chain failure domains.

The ontology has three main purposes in the methodology:

1. to guide AI-supported generation of candidate failure modes;
2. to support review of generated FMEA rows by checking conceptual consistency;
3. to provide a shared semantic structure for explanation, elicitation, and revision.

In the proposed framework, the ontology works in parallel with the model parser. The parser provides system-specific context extracted from the SysML model. The ontology provides domain-agnostic reasoning guidance. Therefore, the parser tells the AI what system information to reason about, while the ontology guides how the AI should structure its reasoning.

C.2 Ontology Creation Process

The ontology was created through a four-stage process, shown conceptually in Figure C.1. The process begins with standards and guidance documents, then moves through a human-readable outline, then ontology formalisation, and finally transformation into a GraphRAG-ready representation.

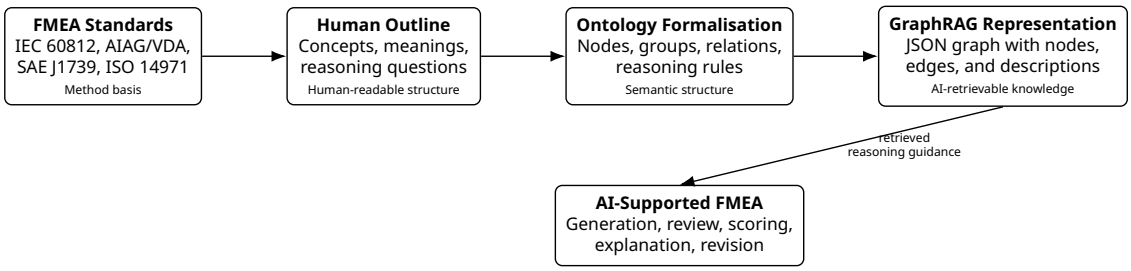


Figure C.1: Ontology creation process from standards-based guidance to a GraphRAG-ready ontology representation.

C.2.1 Stage 1: Standards-Based Concept Extraction

The first stage was to identify the main concepts and reasoning steps used in established FMEA and risk-analysis practice. The ontology uses four standards or guidance sources as its foundation:

- IEC 60812:2018, used as a general FMEA and FMECA method reference;
- AIAG/VDA FMEA Handbook, used for its seven-step FMEA workflow;
- SAE J1739, used for design FMEA, process FMEA, and FMEA-MSR concepts;
- ISO 14971 risk-management thinking, used for hazard, hazardous situation, harm, and risk-control logic.

These sources provide the methodological basis for the ontology. They help identify the required reasoning concepts, such as item, function, failure mode, effect, cause, control, severity, occurrence, detection, recommended action, evidence, and residual risk.

C.2.2 Stage 2: Human-Readable Ontology Outline

The second stage was to translate the standards-based concepts into a human-readable outline. This step was necessary because standards describe FMEA methods, but they do not directly provide an AI-usable ontology. The outline organised the concepts into groups, clarified their meanings, and identified relationships between them.

For example, the outline separates the following ideas:

- an item or component performs a function;
- a function can fail through a failure mode;

- a failure mode may have one or more causes;
- a failure mode produces local, next-higher-level, system, or end-user effects;
- current controls may prevent causes, detect failures, or mitigate effects;
- risk ratings depend on severity, occurrence, and detection;
- recommended actions should reduce risk, uncertainty, or weak traceability.

This human outline provided the bridge between standards-based FMEA knowledge and the structured JSON ontology.

C.2.3 Stage 3: Ontology Formalisation

The third stage was to formalise the outline into a graph-like ontology structure. Concepts were represented as nodes, and their relationships were represented as edges. Each node includes an identifier, label, group, title, and description. Some nodes also include AI question prompts, which are used to guide the AI when information is missing or when deeper reasoning is required.

The formalisation produced concepts across several categories, including FMEA process steps, context concepts, structure concepts, function concepts, failure-logic concepts, risk concepts, rating concepts, control concepts, action concepts, evidence concepts, FMEA types, failure domains, failure-mode categories, cause categories, and AI question templates.

C.2.4 Stage 4: GraphRAG-Ready Transformation

The final stage was to transform the ontology into a GraphRAG-ready JSON representation. In this representation, concepts are stored as retrievable knowledge nodes, and semantic relations are stored as edges. This enables the AI component to retrieve relevant ontology fragments during analysis instead of relying only on its internal model knowledge.

This GraphRAG-ready form supports AI reasoning in three ways:

1. it makes relevant concepts retrievable;
2. it makes relationships between concepts explicit;
3. it provides reasoning rules and question templates that can be injected into AI prompts.

This is important for model integrity because it reduces unbounded AI reasoning. The AI is guided by explicit concepts and relations rather than being asked to invent an FMEA structure independently.

C.3 JSON Structure of the Ontology

The ontology JSON contains five main top-level elements: name, description, references, reasoning rules, nodes, and edges. Table C.1 summarises these elements.

Table C.1: Top-level JSON structure of the domain-agnostic FMEA ontology

Element	Type	Purpose
`name`	String	Provides the name of the ontology.
`description`	String	Describes the purpose and domain coverage of the ontology.
`references`	Array	Lists standards and guidance sources used to ground the ontology.
`reasoning_rules`	Array	Defines rules that guide AI-supported FMEA reasoning.
`nodes`	Array	Contains ontology concepts, grouped by semantic category.
`edges`	Array	Contains graph relationships between ontology concepts.

The current ontology contains 4 references, 10 reasoning rules, 168 nodes, and 355 edges. Table C.2 summarises the size of the ontology artefact.

Table C.2: Size of the ontology artefact

Ontology element	Count
References	4
Reasoning rules	10
Nodes	168
Edges	355

C.4 Standards Foundation

The ontology is grounded in established FMEA and risk-analysis sources. These sources are not used as direct executable rules, but as methodological anchors for the ontology. Table C.3 summarises the four reference sources represented in the JSON file.

Table C.3: Standards and guidance sources represented in the ontology

Source	Role in the ontology
IEC 60812:2018	Provides the general FMEA/FMECA method basis for planning, performing, documenting, maintaining, and identifying how items or processes fail to perform functions.
AIAG/VDA FMEA Handbook	Provides the seven-step FMEA workflow: planning and preparation, structure analysis, function analysis, failure analysis, risk analysis, optimisation, and results documentation.
SAE J1739	Provides concepts relevant to Design FMEA, Process FMEA, and monitoring/system-response FMEA.
ISO 14971 risk-management thinking	Provides concepts for connecting technical failure modes to hazards, hazardous situations, harm, and risk controls.

C.5 Reasoning Rules

The ontology includes ten reasoning rules. These rules are not ordinary FMEA table columns. Instead, they guide the AI in how to reason when generating or reviewing FMEA content. Table C.4 summarises the rules.

Table C.4: Reasoning rules included in the ontology

Rule ID	Name	Purpose
rule-01	Function-first reasoning	Before proposing failure modes, identify the intended function, operating mode, environment, input, output, performance parameter, and requirement.
rule-02	Separate mode, cause, and effect	Prevents the AI from mixing what goes wrong, why it happens, and what happens because of it.
rule-03	Multiple levels of effect	Encourages local, next-higher-level, system, end-user, safety, regulatory, or business effects where relevant.
rule-04	Cross-domain sweep	Forces consideration across mechanical, electrical, software, control, thermal, interface, human, environmental, cybersecurity, data/AI, maintenance, and supply-chain domains.
rule-05	Interface suspicion	Treats interfaces as high-risk zones because mismatched assumptions, timing, units, tolerances, protocols, and responsibilities often create hidden failures.
rule-06	Evidence over confidence	Requires ratings and claims about controls to be supported by evidence such as tests, simulations, inspections, process capability, supplier records, field data, or expert rationale.
rule-07	Current-control specificity	Requires current controls to specify whether they prevent causes, detect failures, mitigate effects, monitor diagnostics, or trigger responses.
rule-08	Action target	Requires recommended actions to explicitly target severity, occurrence, detection, diagnostic coverage, uncertainty, or traceability.
rule-09	Residual risk	Requires reassessment after actions and records remaining risk with acceptance rationale.
rule-10	AI follow-up	Requires targeted follow-up questions when key context is missing, and explicit assumption marking when the analysis must proceed with incomplete information.

These reasoning rules are central to the use of the ontology. They reduce the risk of unsupported or vague AI-generated FMEA rows by forcing the reasoning process to remain

function-based, evidence-aware, and reviewable.

C.6 Node Groups

The ontology contains 168 nodes organised into conceptual groups. Each node represents a concept that may be used during FMEA reasoning. The node groups are summarised in Table C.5.

Table C.5: Main node groups in the domain-agnostic FMEA ontology

Node group	Count	Purpose
Control Method	18	Represents methods such as design review, simulation, inspection, redundancy, derating, fail-safe states, maintenance plans, and change control.
Failure Domain	17	Represents broad domains where failures may arise, including mechanical, electrical, software, control, thermal, process, interface, cybersecurity, data/AI, maintenance, and supply chain.
Cause Category	13	Represents typical classes of causes, such as requirement ambiguity, design weakness, manufacturing defect, assembly error, contamination, overload, supplier variation, misuse, and software defect.
Failure Mode Category	12	Represents generic patterns of functional deviation, such as loss of function, partial function, degraded function, intermittent function, unintended function, wrong function, early function, late function, unstable function, out-of-tolerance, leakage, and blockage.
AI Question Template	12	Provides question prompts for context, function, interface, failure mode, effect, cause, control, evidence, missing information, cross-domain review, and prioritisation.
FMEA Type	9	Represents variants such as System FMEA, Design FMEA, Process FMEA, FMEA-MSR, Software FMEA, Hardware FMEA, FMECA, FMEDA, and Human Factors FMEA.
Context	8	Represents scope, boundary, environment, lifecycle phase, operating mode, use scenario, stakeholder, and assumption.
Failure Logic	8	Represents failure mode, failure effect, failure cause, root cause, mechanism, local effect, next-higher-level effect, and end-user effect.
FMEA Step	7	Represents the workflow steps from planning and preparation to results documentation.
Structure	7	Represents item, system, subsystem, component, interface, input, and output.
Function	6	Represents function, primary function, secondary function, safety function, diagnostic function, and functional chain.

Node group	Count	Purpose
Rating	6	Represents severity, occurrence, detection rating, RPN, action priority, and criticality.
Risk Concept	5	Represents hazard, hazardous situation, harm, risk, and residual risk.
Control	4	Represents current control, prevention control, detection control, and diagnostic monitoring.
Action	3	Represents recommended action, action owner, and action deadline.
Standard	4	Represents the standards and guidance sources used to ground the ontology.
Other specialised groups	31	Include requirement, metric, decision rule, evidence, knowledge, and specific mechanical, electrical, software, thermal, data, AI, and interface failure-mode concepts.

The node groups show that the ontology is broader than a simple FMEA table schema. It includes not only output fields, but also reasoning steps, failure domains, cause categories, control methods, rating logic, and AI question prompts.

C.7 FMEA Process Concepts

The ontology includes a process structure aligned with a staged FMEA workflow. The main FMEA step nodes are:

1. planning and preparation;
2. structure analysis;
3. function analysis;
4. failure analysis;
5. risk analysis;
6. optimisation;
7. results documentation.

These steps are linked through precedence relations. This means that the ontology does not only describe FMEA concepts; it also represents an expected reasoning sequence. Planning and preparation define the scope, boundary, assumptions, stakeholders, lifecycle phase, and risk acceptance criteria. Structure analysis identifies the item, system, subsystem, component, interface, and environment. Function analysis identifies functions, requirements, inputs, outputs, constraints, operating modes, and performance parameters. Failure analysis identifies failure modes, effects, causes, mechanisms, hazards, and harms. Risk analysis evaluates severity, occurrence, detection, RPN, action priority, criticality, and risk. Optimisation improves actions and controls. Results documentation captures evidence, traceability, residual risk, and lessons learned.

This process structure is useful for AI-supported analysis because it discourages the AI from jumping directly to failure modes without first considering structure and function.

C.8 Failure Logic Concepts

The ontology contains a detailed failure-logic structure. At the centre is the relationship between function, failure mode, cause, effect, and control. The key idea is that failure modes should be derived from functions. A failure mode is not simply a bad event; it is a deviation from what the item or function is supposed to do.

The ontology distinguishes between:

- **failure mode**: the way the function fails;
- **failure cause**: the reason the failure mode occurs;
- **failure mechanism**: the physical, logical, chemical, human, cyber, or process mechanism behind the cause;
- **failure effect**: the consequence of the failure mode;
- **local effect**: the consequence on the same item or process step;
- **next-higher-level effect**: the consequence on a subsystem or downstream function;
- **end-user effect**: the consequence experienced by the user, operator, maintainer, customer, or other stakeholder.

This distinction is important because AI-generated FMEA rows often risk mixing mode, cause, and effect. The ontology provides a conceptual structure that helps prevent this confusion.

C.9 Failure Domains and Failure Mode Categories

The ontology includes a cross-domain failure sweep. This is useful because failures can arise from many domains, not only from physical component breakage. The failure domain nodes include mechanical, electrical, electronic, software, control, thermal, fluidic, chemical, material, human, process, interface, environmental, cybersecurity, data/AI, maintenance, and supply-chain failures.

The ontology also includes generic failure mode categories. These categories help transform functions into possible deviations. Examples include:

- loss of function;
- partial function;
- degraded function;
- intermittent function;
- unintended function;
- wrong function;
- early function;
- late function;

- unstable or oscillatory function;
- out-of-tolerance output;
- leakage;
- blockage.

These categories support systematic failure-mode generation. Instead of asking the AI to guess random failures, the ontology guides the AI to ask how each function might be absent, partial, degraded, intermittent, unintended, wrong, early, late, unstable, or outside tolerance.

C.10 Risk, Rating, and Control Concepts

The ontology includes concepts for risk analysis and prioritisation. These include severity, occurrence, detection rating, RPN, action priority, criticality, risk, residual risk, and risk acceptance criteria. These concepts help the AI distinguish between identifying a failure and prioritising it.

Severity is associated with the seriousness of the effect. Occurrence is associated with the likelihood or frequency of the cause or failure mode. Detection rating is associated with the ability of existing controls to detect the cause or failure before the effect reaches the user. RPN is represented as the traditional product of severity, occurrence, and detection, while action priority and criticality are included as additional prioritisation concepts.

The ontology also distinguishes between different types of controls:

- current controls;
- prevention controls;
- detection controls;
- diagnostic monitoring;
- recommended actions.

This distinction supports more precise FMEA rows. For example, a control should not be written vaguely as “testing” without explaining whether it prevents the cause, detects the failure, mitigates the effect, or monitors the system during operation.

C.11 Evidence, Traceability, and Documentation Concepts

A major strength of the ontology is that it includes evidence and traceability concepts. Verification evidence is represented as proof that an action or control works. Examples include test reports, simulation results, inspection data, requirement traces, field data, audit records, or expert rationale.

The ontology also includes traceability as a control method. Traceability links requirements, functions, design elements, tests, risks, actions, and evidence. In the context of this thesis,

this is directly connected to model integrity. An FMEA row is more useful when it can be connected back to the model information that supports it.

The ontology therefore supports evidence-aware AI reasoning. It encourages the AI not only to generate plausible failure modes, but also to consider what evidence or rationale supports the generated row.

C.12 AI Question Templates

The ontology includes a set of AI question templates. These templates are designed to guide the AI when context is missing, when reasoning needs to be expanded, or when the user asks for explanation. The question templates include:

- context questions;
- function questions;
- interface questions;
- failure mode questions;
- effect questions;
- cause questions;
- control questions;
- evidence questions;
- counterfactual questions;
- cross-domain questions;
- missing-information questions;
- prioritisation questions.

These templates are useful because they make the ontology interactive. The ontology does not only store concepts; it also suggests what questions should be asked to improve the analysis. This is especially relevant for the chat-based elicitation and revision interface described in Chapter 4.

C.13 Graph Structure and Edge Relations

The ontology contains 355 edges. These edges connect the ontology concepts into a reasoning graph. The edges describe relationships such as:

- standards inform the ontology;
- the ontology enables the AI FMEA facilitator;
- an FMEA study contains FMEA records;

- FMEA process steps precede one another;
- planning defines scope, boundary, assumptions, stakeholders, lifecycle phase, and risk criteria;
- structure analysis identifies items, systems, subsystems, components, interfaces, and environments;
- function analysis identifies functions, requirements, inputs, outputs, constraints, operating modes, and performance parameters;
- failure analysis identifies failure modes, effects, causes, mechanisms, hazards, and harms;
- risk analysis evaluates severity, occurrence, detection, RPN, action priority, criticality, and risk;
- optimisation improves recommended actions, prevention controls, detection controls, diagnostic monitoring, and residual risk;
- results documentation captures verification evidence, traceability, and lessons learned.

The graph structure is important because it allows the ontology to be used for retrieval. For example, if the AI is generating failure modes for an interface, the graph can retrieve interface-related failure questions, interface failure domains, possible causes, detection controls, and evidence questions. This makes the ontology suitable for GraphRAG.

C.14 Use of the Ontology in the Proposed Framework

Within the methodology, the ontology is used together with the parsed model context. The parsed model context is system-specific. It describes parts, actions, ports, flows, connections, requirements, allocations, and state/control information extracted from the SysML model. The ontology is domain-agnostic. It describes the general structure of FMEA reasoning.

The AI component receives both inputs. The parser output grounds the AI in the specific system. The ontology guides the AI in structuring the analysis. This combination supports several activities:

1. **Generation**: the ontology guides the AI to derive failure modes from functions and model context.
2. **Review**: the ontology helps check whether generated rows correctly separate item, function, failure mode, cause, effect, control, and evidence.
3. **Scoring**: the ontology provides concepts for severity, occurrence, detection, RPN, action priority, and residual risk.
4. **Explanation**: the ontology provides a shared vocabulary for explaining why a row was generated.
5. **Revision**: the ontology helps preserve FMEA structure when the user asks for changes.

C.15 Limitations of the Ontology

The ontology has several limitations. First, it is not a complete formal ontology in a description-logic sense. It does not include full formal constraints, inference rules, or machine-verifiable consistency conditions. It is better understood as a structured GraphRAG knowledge artefact.

Second, some node descriptions are written in informal language. This can be useful for internal AI prompting, but thesis-facing or industrial versions should use more neutral wording. Informal phrases should be revised before the ontology is presented as a formal engineering artefact.

Third, the ontology is domain-agnostic. This is useful for reuse, but it also means that it does not contain detailed domain-specific failure physics for any one system. For example, it can guide reasoning about electrical failure modes, but it does not replace detailed electrical reliability data, simulation results, or expert domain knowledge.

Fourth, the ontology depends on governance and maintenance. If new FMEA practices, standards, failure domains, or project-specific concepts are introduced, the ontology must be updated. Otherwise, it may become stale in the same way that ordinary engineering documents can become stale.

Finally, the ontology does not remove the need for human review. It improves the structure and consistency of AI-supported reasoning, but it does not guarantee correctness. Engineering judgement remains necessary for validating generated failure modes, risk scores, recommended actions, and evidence claims.

C.16 OWL Representation

The domain-agnostic FMEA ontology was translated into an OWL-compatible representation using Turtle syntax. Turtle was selected because it is more compact and readable than RDF/XML while still representing OWL classes, individuals, object properties, and annotation properties in a machine-interpretable form. This makes the ontology suitable for documentation, inspection, and later use in semantic tools.

Please not that regardless of which language is used to represent the ontology, it will ultimately has to be turned into a GraphRAG database for AI reasoning.

In the OWL representation, each concept from the ontology is represented as an `owl:NamedIndividual`. The original concept groups, such as *FMEA Step*, *Structure*, *Function*, *Failure Logic*, *Risk Concept*, *Rating*, *Control*, *Action*, *Evidence*, and *AI Question Template*, are represented as OWL classes. Each node is assigned to the class that corresponds to its group. Human-readable information such as labels, titles, descriptions, and AI questions is represented using annotation properties.

The relationships between concepts are represented in two complementary ways. First, each relationship is encoded as a direct OWL object-property assertion. For example, if a function can deviate as a failure mode, this relation is represented as an object property between the corresponding individuals. Second, each relationship is also represented as a reified edge individual. This preserves metadata such as the original relation label and any question prompt attached to the edge. This dual representation keeps the ontology both semantically usable and auditable.

The resulting OWL/Turtle file therefore preserves the main structure of the source ontology:

FMEA standards inform the ontology, the ontology enables an AI FMEA facilitator, FMEA studies contain FMEA records, the seven FMEA steps are connected in sequence, and engineering concepts such as functions, failure modes, causes, effects, controls, evidence, ratings, actions, and residual risk are connected through explicit semantic relations.

Listing C.1 shows the OWL/Turtle representation of the domain-agnostic FMEA ontology.

```
@prefix : <https://example.org/fmea-ontology#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

:FMEADomainAgnosticOntology a owl:Ontology ;
    rdfs:label "FMEA-Domain-Agnostic" ;
    rdfs:comment "Detailed ontology for Failure Mode and Effects Analysis. It is
        designed to help an AI model reason from system context and function to failure
        modes, causes, effects, controls, risk ratings, actions, evidence, and residual
        risk across mechanical, electrical, electronic, software, control, thermal,
        fluidic, chemical, material, human, process, interface, environmental,
        cybersecurity, data/AI, maintenance, and supply-chain failure domains." .

:FMEAConcept a owl:Class ;
    rdfs:label "F M E A Concept" ;
    rdfs:comment "Generic concept used in the FMEA ontology." .

:OntologyEdge a owl:Class ;
    rdfs:label "Ontology Edge" ;
    rdfs:comment "Reified relationship between two ontology concepts." .

:ReasoningRule a owl:Class ;
    rdfs:label "Reasoning Rule" ;
    rdfs:comment "Rule used to guide FMEA reasoning." .

:ReferenceSource a owl:Class ;
    rdfs:label "Reference Source" ;
    rdfs:comment "Standard or guidance source used to inform the ontology." .

:AIFailureModeGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "AI Failure Mode" .

:AIQuestionTemplateGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "AI Question Template" .

:ActionGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Action" .

:AnalysisGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Analysis" .

:ArtifactGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Artifact" .

:CapabilityGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Capability" .

:CauseCategoryGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Cause Category" .

:ChemicalMaterialFailureModeGroup a owl:Class ;
```

```
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Chemical/Material Failure Mode" .

:ContextGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Context" .

:ControlGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Control" .

:ControlMethodGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Control Method" .

:DataFailureModeGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Data Failure Mode" .

:DecisionRuleGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Decision Rule" .

:ElectricalFailureModeGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Electrical Failure Mode" .

:ElectronicFailureModeGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Electronic Failure Mode" .

:ElectronicControlFailureModeGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Electronic/Control Failure Mode" .

:EvidenceGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Evidence" .

:FMEAStepGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "FMEA Step" .

:FMEATypeGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "FMEA Type" .

:FailureDomainGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Failure Domain" .

:FailureLogicGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Failure Logic" .

:FailureModeCategoryGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Failure Mode Category" .

:FunctionGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Function" .

:InterfaceSoftwareFailureModeGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Interface/Software Failure Mode" .
```

```
:KnowledgeGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Knowledge" .

:MechanicalFailureModeGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Mechanical Failure Mode" .

:MechanicalControlFailureModeGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Mechanical/Control Failure Mode" .

:MetricGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Metric" .

:OntologyGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Ontology" .

:RatingGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Rating" .

:RequirementGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Requirement" .

:RiskConceptGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Risk Concept" .

:SoftwareFailureModeGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Software Failure Mode" .

:SoftwareControlFailureModeGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Software/Control Failure Mode" .

:StandardGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Standard" .

:StructureGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Structure" .

:ThermalFailureModeGroup a owl:Class ;
    rdfs:subClassOf :FMEAConcept ;
    rdfs:label "Thermal Failure Mode" .

:hasTitle a owl:AnnotationProperty ;
    rdfs:label "hasTitle" ;
    rdfs:comment "Human-readable title." .

:hasDescription a owl:AnnotationProperty ;
    rdfs:label "hasDescription" ;
    rdfs:comment "Human-readable description." .

:hasGroupName a owl:AnnotationProperty ;
    rdfs:label "hasGroupName" ;
    rdfs:comment "Original group name from the source ontology." .

:hasQuestion a owl:AnnotationProperty ;
    rdfs:label "hasQuestion" ;
    rdfs:comment "AI question associated with a concept." .
```

```
:hasStatement a owl:AnnotationProperty ;
    rdfs:label "hasStatement" ;
    rdfs:comment "Statement of a reasoning rule." .

:hasReferenceUse a owl:AnnotationProperty ;
    rdfs:label "hasReferenceUse" ;
    rdfs:comment "Explanation of how a standard or source is used." .

:hasRelationLabel a owl:AnnotationProperty ;
    rdfs:label "hasRelationLabel" ;
    rdfs:comment "Human-readable label of a relationship." .

:hasQuestionPrompt a owl:AnnotationProperty ;
    rdfs:label "hasQuestionPrompt" ;
    rdfs:comment "Prompt attached to a relationship." .

:hasSource a owl:ObjectProperty ;
    rdfs:label "hasSource" ;
    rdfs:comment "Links a reified ontology edge to its source concept." .

:hasTarget a owl:ObjectProperty ;
    rdfs:label "hasTarget" ;
    rdfs:comment "Links a reified ontology edge to its target concept." .

:adds a owl:ObjectProperty ;
    rdfs:label "adds" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:appliesTo a owl:ObjectProperty ;
    rdfs:label "applies to" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:asks a owl:ObjectProperty ;
    rdfs:label "asks" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:assignedTo a owl:ObjectProperty ;
    rdfs:label "assigned to" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:broadens a owl:ObjectProperty ;
    rdfs:label "broadens" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:canAnalyzeTopEventFor a owl:ObjectProperty ;
    rdfs:label "can analyze top event for" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:canBeAffectedBy a owl:ObjectProperty ;
    rdfs:label "can be affected by" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:canBeClassifiedAs a owl:ObjectProperty ;
    rdfs:label "can be classified as" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:canBeUsedAs a owl:ObjectProperty ;
    rdfs:label "can be used as" ;
```

```
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:canCause a owl:ObjectProperty ;
    rdfs:label "can cause" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:canControl a owl:ObjectProperty ;
    rdfs:label "can control" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:canDetect a owl:ObjectProperty ;
    rdfs:label "can detect" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:canDeviateAs a owl:ObjectProperty ;
    rdfs:label "can deviate as" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:canEvaluate a owl:ObjectProperty ;
    rdfs:label "can evaluate" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:canFailBy a owl:ObjectProperty ;
    rdfs:label "can fail by" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:canLeadTo a owl:ObjectProperty ;
    rdfs:label "can lead to" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:canManage a owl:ObjectProperty ;
    rdfs:label "can manage" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:canMitigate a owl:ObjectProperty ;
    rdfs:label "can mitigate" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:canPrevent a owl:ObjectProperty ;
    rdfs:label "can prevent" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:canReduce a owl:ObjectProperty ;
    rdfs:label "can reduce" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:canResultIn a owl:ObjectProperty ;
    rdfs:label "can result in" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:canTrigger a owl:ObjectProperty ;
    rdfs:label "can trigger" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .
```

```
:canWarnAbout a owl:ObjectProperty ;
    rdfs:label "can warn about" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:captures a owl:ObjectProperty ;
    rdfs:label "captures" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:causes a owl:ObjectProperty ;
    rdfs:label "causes" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:checks a owl:ObjectProperty ;
    rdfs:label "checks" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:clarifies a owl:ObjectProperty ;
    rdfs:label "clarifies" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:conditions a owl:ObjectProperty ;
    rdfs:label "conditions" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:constrains a owl:ObjectProperty ;
    rdfs:label "constrains" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:contains a owl:ObjectProperty ;
    rdfs:label "contains" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:contributesTo a owl:ObjectProperty ;
    rdfs:label "contributes to" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:controls a owl:ObjectProperty ;
    rdfs:label "controls" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:covers a owl:ObjectProperty ;
    rdfs:label "covers" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:defines a owl:ObjectProperty ;
    rdfs:label "defines" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:drives a owl:ObjectProperty ;
    rdfs:label "drives" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:elicits a owl:ObjectProperty ;
```

```
    rdfs:label "elicits" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:emphasizes a owl:ObjectProperty ;
    rdfs:label "emphasizes" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:enables a owl:ObjectProperty ;
    rdfs:label "enables" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:evaluates a owl:ObjectProperty ;
    rdfs:label "evaluates" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:explains a owl:ObjectProperty ;
    rdfs:label "explains" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:feeds a owl:ObjectProperty ;
    rdfs:label "feeds" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:focusesOn a owl:ObjectProperty ;
    rdfs:label "focuses on" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:generates a owl:ObjectProperty ;
    rdfs:label "generates" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:guides a owl:ObjectProperty ;
    rdfs:label "guides" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:has a owl:ObjectProperty ;
    rdfs:label "has" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:hasTypicalModeOrCause a owl:ObjectProperty ;
    rdfs:label "has typical mode or cause" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:identifies a owl:ObjectProperty ;
    rdfs:label "identifies" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:implements a owl:ObjectProperty ;
    rdfs:label "implements" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:improves a owl:ObjectProperty ;
    rdfs:label "improves" ;
    rdfs:domain :FMEAConcept ;
```

```
    rdfs:range :FMEAConcept .

:improvesFuture a owl:ObjectProperty ;
    rdfs:label "improves future" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:includes a owl:ObjectProperty ;
    rdfs:label "includes" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:influences a owl:ObjectProperty ;
    rdfs:label "influences" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:informs a owl:ObjectProperty ;
    rdfs:label "informs" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:isACategoryOf a owl:ObjectProperty ;
    rdfs:label "is a category of" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:isMeasuredBy a owl:ObjectProperty ;
    rdfs:label "is measured by" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:isRecordedIn a owl:ObjectProperty ;
    rdfs:label "is recorded in" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:isVerifiedBy a owl:ObjectProperty ;
    rdfs:label "is verified by" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:judges a owl:ObjectProperty ;
    rdfs:label "judges" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:limits a owl:ObjectProperty ;
    rdfs:label "limits" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:linksTo a owl:ObjectProperty ;
    rdfs:label "links to" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:mayCreate a owl:ObjectProperty ;
    rdfs:label "may create" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:mayReduceSeverityOf a owl:ObjectProperty ;
    rdfs:label "may reduce severity of" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .
```

```
:mayTarget a owl:ObjectProperty ;
    rdfs:label "may target" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:mechanisticallyExplains a owl:ObjectProperty ;
    rdfs:label "mechanistically explains" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:modifiesPrioritizationOf a owl:ObjectProperty ;
    rdfs:label "modifies prioritization of" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:multipliesInto a owl:ObjectProperty ;
    rdfs:label "multiplies into" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:orders a owl:ObjectProperty ;
    rdfs:label "orders" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:performs a owl:ObjectProperty ;
    rdfs:label "performs" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:precedes a owl:ObjectProperty ;
    rdfs:label "precedes" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:prioritizes a owl:ObjectProperty ;
    rdfs:label "prioritizes" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:probes a owl:ObjectProperty ;
    rdfs:label "probes" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:produces a owl:ObjectProperty ;
    rdfs:label "produces" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:propagatesTo a owl:ObjectProperty ;
    rdfs:label "propagates to" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:reduces a owl:ObjectProperty ;
    rdfs:label "reduces" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:requests a owl:ObjectProperty ;
    rdfs:label "requests" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:sets a owl:ObjectProperty ;
    rdfs:label "sets" ;
```

```
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:shouldTarget a owl:ObjectProperty ;
    rdfs:label "should target" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:statesOrOwns a owl:ObjectProperty ;
    rdfs:label "states or owns" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:suggests a owl:ObjectProperty ;
    rdfs:label "suggests" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:supportsReassessmentOf a owl:ObjectProperty ;
    rdfs:label "supports reassessment of" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:surfaces a owl:ObjectProperty ;
    rdfs:label "surfaces" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:traces a owl:ObjectProperty ;
    rdfs:label "traces" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:transfers a owl:ObjectProperty ;
    rdfs:label "transfers" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:usesStep a owl:ObjectProperty ;
    rdfs:label "uses step" ;
    rdfs:domain :FMEAConcept ;
    rdfs:range :FMEAConcept .

:fmea-ontology a owl:NamedIndividual, :OntologyGroup ;
    rdfs:label "FMEA Ontology" ;
    :hasGroupName "Ontology" ;
    :hasTitle "Semantic map for FMEA reasoning" ;
    :hasDescription "Connects system structure, functions, failure modes, causes,
        effects, controls, risk scoring, and AI question prompts." .

:ai-fmea-facilitator a owl:NamedIndividual, :CapabilityGroup ;
    rdfs:label "AI FMEA Facilitator" ;
    :hasGroupName "Capability" ;
    :hasTitle "Guides an AI model to ask function-first failure analysis questions" ;
    :hasDescription "Uses the ontology to check missing context, classify failure types,
         detect weak causal reasoning, and suggest follow-up questions." .

:fmea-study a owl:NamedIndividual, :AnalysisGroup ;
    rdfs:label "FMEA Study" ;
    :hasGroupName "Analysis" ;
    :hasTitle "A bounded FMEA activity for a specific item, process, service, or system"
        ;
    :hasDescription "Defines what is analyzed, why, by whom, under which assumptions,
        and with which scoring criteria." .

:fmea-record a owl:NamedIndividual, :ArtifactGroup ;
    rdfs:label "FMEA Record" ;
```

```
    :hasGroupName "Artifact" ;
    :hasTitle "A single row or case in the FMEA table" ;
    :hasDescription "Links one function to one failure mode, its effects, causes,
        controls, ratings, actions, and residual risk." .

:iec-60812-2018 a owl:NamedIndividual, :StandardGroup ;
    rdfs:label "IEC 60812:2018" ;
    :hasGroupName "Standard" ;
    :hasTitle "International FMEA and FMECA procedure reference" ;
    :hasDescription "Useful as the general method anchor for planning, performing,
        documenting, maintaining FMEA/FMECA, and identifying how items or processes may
        fail to perform functions." ;
    :hasReferenceUse "General FMEA/FMECA method anchor: planning, performing,
        documenting, maintaining, and identifying how items or processes fail to perform
         functions." .

:aiag-vda-fmea-handbook a owl:NamedIndividual, :StandardGroup ;
    rdfs:label "AIAG/VDA FMEA Handbook" ;
    :hasGroupName "Standard" ;
    :hasTitle "Automotive seven-step FMEA structure" ;
    :hasDescription "Useful for structuring FMEA as planning, structure analysis,
        function analysis, failure analysis, risk analysis, optimization, and results
        documentation." ;
    :hasReferenceUse "Seven-step FMEA workflow: planning and preparation, structure
        analysis, function analysis, failure analysis, risk analysis, optimization, and
        results documentation." .

:sae-j1739 a owl:NamedIndividual, :StandardGroup ;
    rdfs:label "SAE J1739" ;
    :hasGroupName "Standard" ;
    :hasTitle "DFMEA, PFMEA, and supplemental FMEA-MSR reference" ;
    :hasDescription "Useful for design, process, and monitoring/system-response FMEA in
        automotive and related engineering contexts." ;
    :hasReferenceUse "Design FMEA, Process FMEA, and supplemental FMEA-MSR reference." .

:iso-14971-risk-thinking a owl:NamedIndividual, :StandardGroup ;
    rdfs:label "ISO 14971 Risk Thinking" ;
    :hasGroupName "Standard" ;
    :hasTitle "Hazard, hazardous situation, harm, and risk-control reasoning" ;
    :hasDescription "Useful when FMEA must connect technical failure modes to
        foreseeable sequences of events, hazards, harms, and risk controls, especially
        in medical or safety-related systems." ;
    :hasReferenceUse "Hazard, foreseeable sequence of events, hazardous situation, harm,
         and risk-control logic for safety-related contexts." .

:planning-preparation a owl:NamedIndividual, :FMEAStepGroup ;
    rdfs:label "Planning and Preparation" ;
    :hasGroupName "FMEA Step" ;
    :hasTitle "Define purpose, scope, team, assumptions, and analysis boundaries" .

:structure-analysis a owl:NamedIndividual, :FMEAStepGroup ;
    rdfs:label "Structure Analysis" ;
    :hasGroupName "FMEA Step" ;
    :hasTitle "Break down the system, process, interfaces, and hierarchy" ;
    :hasDescription "Answers: what parts exist and how are they connected?" .

:function-analysis a owl:NamedIndividual, :FMEAStepGroup ;
    rdfs:label "Function Analysis" ;
    :hasGroupName "FMEA Step" ;
    :hasTitle "Identify intended functions, requirements, operating modes, and
        performance parameters" ;
    :hasDescription "Failure modes should be derived from functions, not guessed
        randomly." .

:failure-analysis a owl:NamedIndividual, :FMEAStepGroup ;
    rdfs:label "Failure Analysis" ;
    :hasGroupName "FMEA Step" ;
```

```
    :hasTitle "Identify failure modes, effects, causes, and mechanisms" ;
    :hasDescription "Connects what can go wrong, why it goes wrong, and what happens
        next." .

:risk-analysis a owl:NamedIndividual, :FMEAStepGroup ;
    rdfs:label "Risk Analysis" ;
    :hasGroupName "FMEA Step" ;
    :hasTitle "Score or rank severity, occurrence, detection, criticality, or action
        priority" .

:optimization a owl:NamedIndividual, :FMEAStepGroup ;
    rdfs:label "Optimization" ;
    :hasGroupName "FMEA Step" ;
    :hasTitle "Select and verify actions that reduce risk" .

:results-documentation a owl:NamedIndividual, :FMEAStepGroup ;
    rdfs:label "Results Documentation" ;
    :hasGroupName "FMEA Step" ;
    :hasTitle "Capture decisions, evidence, residual risk, and lessons learned" ;
    :hasDescription "Makes the analysis reusable, auditable, and traceable." .

:scope a owl:NamedIndividual, :ContextGroup ;
    rdfs:label "Scope" ;
    :hasGroupName "Context" ;
    :hasTitle "Defines what is inside and outside the FMEA" ;
    :hasDescription "Include product/process name, lifecycle phase, customer, use
        context, interfaces, exclusions, and depth of analysis." ;
    :hasQuestion "What exactly are we analyzing?" ;
    :hasQuestion "What is explicitly out of scope?" ;
    :hasQuestion "Why is this boundary chosen?" .

:boundary a owl:NamedIndividual, :ContextGroup ;
    rdfs:label "System Boundary" ;
    :hasGroupName "Context" ;
    :hasTitle "Separates the analyzed item from external systems" ;
    :hasDescription "Prevents blaming external systems without modeling the interface."
        ;
    :hasQuestion "Where does responsibility transfer to another system or team?" ;
    :hasQuestion "Which inputs and outputs cross the boundary?" .

:item a owl:NamedIndividual, :StructureGroup ;
    rdfs:label "Item" ;
    :hasGroupName "Structure" ;
    :hasTitle "The top-level object being analyzed" ;
    :hasDescription "Can be a product, process step, subsystem, software service,
        medical device, robot, manufacturing operation, or organization process." .

:system a owl:NamedIndividual, :StructureGroup ;
    rdfs:label "System" ;
    :hasGroupName "Structure" ;
    :hasTitle "A set of interacting elements with a purpose" ;
    :hasDescription "The top object when FMEA is performed at system level." .

:subsystem a owl:NamedIndividual, :StructureGroup ;
    rdfs:label "Subsystem" ;
    :hasGroupName "Structure" ;
    :hasTitle "A major part of a system" ;
    :hasDescription "Useful for hierarchical effects: local, next-level, system-level,
        user-level." .

:component a owl:NamedIndividual, :StructureGroup ;
    rdfs:label "Component" ;
    :hasGroupName "Structure" ;
    :hasTitle "A physical, software, human, or process element" ;
    :hasDescription "Small enough to have concrete functions and failure modes." .

:interface a owl:NamedIndividual, :StructureGroup ;
```

```
    rdfs:label "Interface" ;
    :hasGroupName "Structure" ;
    :hasTitle "Connection where energy, material, information, force, fluid, heat, or
        control crosses elements" ;
    :hasQuestion "What crosses this interface?" ;
    :hasQuestion "What assumptions does each side make?" ;
    :hasQuestion "What happens if the interface is late, noisy, leaky, blocked, reversed
        , overloaded, or incompatible?" .

:environment a owl:NamedIndividual, :ContextGroup ;
    rdfs:label "Environment" ;
    :hasGroupName "Context" ;
    :hasTitle "External physical, cyber, operational, regulatory, and organizational
        conditions" ;
    :hasDescription "Include temperature, humidity, vibration, EMC, contamination, users
        , network, workload, and maintenance setting." .

:lifecycle_phase a owl:NamedIndividual, :ContextGroup ;
    rdfs:label "Lifecycle Phase" ;
    :hasGroupName "Context" ;
    :hasTitle "Phase where the failure can occur or be detected" ;
    :hasDescription "Examples: concept, design, manufacturing, transport, installation,
        operation, maintenance, update, decommissioning." .

:operating_mode a owl:NamedIndividual, :ContextGroup ;
    rdfs:label "Operating Mode" ;
    :hasGroupName "Context" ;
    :hasTitle "Mode or state in which functions and failures differ" ;
    :hasDescription "Examples: startup, normal operation, degraded mode, emergency stop,
         calibration, cleaning, charging, shutdown." .

:use_scenario a owl:NamedIndividual, :ContextGroup ;
    rdfs:label "Use Scenario" ;
    :hasGroupName "Context" ;
    :hasTitle "A realistic sequence of user, system, and environment interactions" ;
    :hasDescription "Useful for finding context-dependent failures." .

:stakeholder a owl:NamedIndividual, :ContextGroup ;
    rdfs:label "Stakeholder" ;
    :hasGroupName "Context" ;
    :hasTitle "Person or organization affected by the function, failure, risk, or
        decision" ;
    :hasDescription "Examples: user, patient, operator, maintainer, manufacturer,
        regulator, supplier, service team." .

:requirement a owl:NamedIndividual, :ArtifactGroup ;
    rdfs:label "Requirement" ;
    :hasGroupName "Artifact" ;
    :hasTitle "A documented need, constraint, or performance expectation" ;
    :hasDescription "Failure analysis should trace back to requirements when possible."
        ;
    :hasQuestion "Which requirement is violated if this function fails?" ;
    :hasQuestion "Is the requirement measurable or ambiguous?" .

:assumption a owl:NamedIndividual, :ContextGroup ;
    rdfs:label "Assumption" ;
    :hasGroupName "Context" ;
    :hasTitle "A condition believed true but not yet proven" ;
    :hasDescription "Assumptions should be visible because hidden assumptions are
        failure-mode factories." .

:function a owl:NamedIndividual, :FunctionGroup ;
    rdfs:label "Function" ;
    :hasGroupName "Function" ;
    :hasTitle "Intended action of an item under defined conditions" ;
    :hasDescription "Write as verb + object + measurable target when possible, e.g., '
        limit motor current below 5 A'." ;
```

```
    :hasQuestion "What must this item do?" ;
    :hasQuestion "Why does this function matter?" ;
    :hasQuestion "What measurable parameter proves it works?" .

:primary_function a owl:NamedIndividual, :FunctionGroup ;
    rdfs:label "Primary Function" ;
    :hasGroupName "Function" ;
    :hasTitle "Main reason the item exists" ;
    :hasDescription "Failure of a primary function often drives high severity." .

:secondary_function a owl:NamedIndividual, :FunctionGroup ;
    rdfs:label "Secondary Function" ;
    :hasGroupName "Function" ;
    :hasTitle "Supporting function needed for operation, usability, safety, or
        compliance" .

:safety_function a owl:NamedIndividual, :FunctionGroup ;
    rdfs:label "Safety Function" ;
    :hasGroupName "Function" ;
    :hasTitle "Function that prevents or reduces harm" ;
    :hasDescription "Examples: stop motion, isolate energy, limit temperature, alarm
        user, enter safe state." .

:diagnostic_function a owl:NamedIndividual, :FunctionGroup ;
    rdfs:label "Diagnostic Function" ;
    :hasGroupName "Function" ;
    :hasTitle "Function that detects, reports, or isolates faults" ;
    :hasDescription "Important for detection rating and monitoring_response FMEA." .

:performance_parameter a owl:NamedIndividual, :MetricGroup ;
    rdfs:label "Performance Parameter" ;
    :hasGroupName "Metric" ;
    :hasTitle "Measurable variable that defines function quality" ;
    :hasDescription "Examples: torque, voltage, response time, accuracy, temperature,
        flow rate, pressure, current, latency." .

:functional_chain a owl:NamedIndividual, :FunctionGroup ;
    rdfs:label "Functional Chain" ;
    :hasGroupName "Function" ;
    :hasTitle "Ordered set of functions across components or process steps" ;
    :hasDescription "Useful to ask: if upstream output fails, what downstream function
        fails next?" .

:input a owl:NamedIndividual, :StructureGroup ;
    rdfs:label "Input" ;
    :hasGroupName "Structure" ;
    :hasTitle "Energy, material, information, command, or condition entering a function"
        ;
    :hasDescription "Inputs can fail by being absent, wrong, noisy, late, early, too
        high, too low, or incompatible." .

:output a owl:NamedIndividual, :StructureGroup ;
    rdfs:label "Output" ;
    :hasGroupName "Structure" ;
    :hasTitle "Energy, material, information, command, product state, or service result
        produced by a function" ;
    :hasDescription "Outputs become effects or causes for other elements." .

:constraint a owl:NamedIndividual, :RequirementGroup ;
    rdfs:label "Constraint" ;
    :hasGroupName "Requirement" ;
    :hasTitle "A limit on function, design, process, or operation" ;
    :hasDescription "Examples: mass limit, cost limit, regulatory limit, voltage limit,
        sterilization condition." .

:failure_mode a owl:NamedIndividual, :FailureLogicGroup ;
    rdfs:label "Failure Mode" ;
```

```
    :hasGroupName "Failure Logic" ;
    :hasTitle "The specific way a function, item, or process can fail" ;
    :hasDescription "Best phrased as a deviation from intended function: no, partial,
        degraded, intermittent, unintended, wrong, early, late, unstable, or out-of-
        tolerance." ;
    :hasQuestion "How can this function fail?" ;
    :hasQuestion "Is this phrased as a problem, not a cause or effect?" ;
    :hasQuestion "What would an observer see?" .

:failure-effect a owl:NamedIndividual, :FailureLogicGroup ;
    rdfs:label "Failure Effect" ;
    :hasGroupName "Failure Logic" ;
    :hasTitle "Consequence of a failure mode" ;
    :hasDescription "Effects should be described at local, next-higher, system, user,
        safety, business, and regulatory levels where relevant." ;
    :hasQuestion "What happens immediately?" ;
    :hasQuestion "Who notices?" ;
    :hasQuestion "What is the worst credible consequence?" .

:local-effect a owl:NamedIndividual, :FailureLogicGroup ;
    rdfs:label "Local Effect" ;
    :hasGroupName "Failure Logic" ;
    :hasTitle "Effect on the same component or process step" ;
    :hasDescription "Useful for technical diagnosis." .

:next-higher-level-effect a owl:NamedIndividual, :FailureLogicGroup ;
    rdfs:label "Next Higher Level Effect" ;
    :hasGroupName "Failure Logic" ;
    :hasTitle "Effect on subsystem, process output, or downstream function" ;
    :hasDescription "Connects local faults to system behavior." .

:end-user-effect a owl:NamedIndividual, :FailureLogicGroup ;
    rdfs:label "End User Effect" ;
    :hasGroupName "Failure Logic" ;
    :hasTitle "Effect experienced by customer, user, patient, operator, or maintainer" ;
    :hasDescription "Important for severity and communication." .

:failure-cause a owl:NamedIndividual, :FailureLogicGroup ;
    rdfs:label "Failure Cause" ;
    :hasGroupName "Failure Logic" ;
    :hasTitle "Reason the failure mode occurs" ;
    :hasDescription "Should be controllable or testable when possible, not vague like '
        bad design'." ;
    :hasQuestion "Why would this failure mode happen?" ;
    :hasQuestion "Can this cause be prevented, detected, or verified?" .

:root-cause a owl:NamedIndividual, :FailureLogicGroup ;
    rdfs:label "Root Cause" ;
    :hasGroupName "Failure Logic" ;
    :hasTitle "Deeper cause that explains the origin of one or more direct causes" ;
    :hasDescription "Useful for corrective action and lessons learned." .

:failure-mechanism a owl:NamedIndividual, :FailureLogicGroup ;
    rdfs:label "Failure Mechanism" ;
    :hasGroupName "Failure Logic" ;
    :hasTitle "Physical, logical, chemical, human, cyber, or process mechanism causing
        failure" ;
    :hasDescription "Examples: fatigue crack growth, corrosion, memory leak, adhesive
        cure failure, calibration drift." .

:hazard a owl:NamedIndividual, :RiskConceptGroup ;
    rdfs:label "Hazard" ;
    :hasGroupName "Risk Concept" ;
    :hasTitle "Potential source of harm" ;
    :hasDescription "Examples: electrical energy, sharp edge, stored pressure, toxic
        substance, moving actuator, wrong medical advice." .
```

```
:hazardous-situation a owl:NamedIndividual, :RiskConceptGroup ;
    rdfs:label "Hazardous Situation" ;
    :hasGroupName "Risk Concept" ;
    :hasTitle "Circumstance where people, assets, or environment are exposed to a hazard
        " ;
    :hasDescription "Connects technical failures to real-world harm scenarios." .

:harm a owl:NamedIndividual, :RiskConceptGroup ;
    rdfs:label "Harm" ;
    :hasGroupName "Risk Concept" ;
    :hasTitle "Injury, damage, loss, or adverse impact" ;
    :hasDescription "Can include safety, mission, financial, legal, environmental,
        reputational, or user-trust harm." .

:risk a owl:NamedIndividual, :RiskConceptGroup ;
    rdfs:label "Risk Concept" ;
    :hasGroupName "Risk Concept" ;
    :hasTitle "Combination of severity and likelihood, with detectability or
        controllability where the method uses it" .

:severity a owl:NamedIndividual, :RatingGroup ;
    rdfs:label "Severity" ;
    :hasGroupName "Rating" ;
    :hasTitle "Seriousness of the effect if the failure occurs" ;
    :hasDescription "Usually based on worst credible effect, not how easy the fix is." .

:occurrence a owl:NamedIndividual, :RatingGroup ;
    rdfs:label "Occurrence" ;
    :hasGroupName "Rating" ;
    :hasTitle "Estimated likelihood or frequency of the cause or failure mode" ;
    :hasDescription "Should use evidence such as field data, test data, process
        capability, supplier history, or expert judgment." .

:detection-rating a owl:NamedIndividual, :RatingGroup ;
    rdfs:label "Detection Rating" ;
    :hasGroupName "Rating" ;
    :hasTitle "Ability of current controls to detect cause or failure before the effect
        reaches the user" ;
    :hasDescription "High detectability lowers risk priority only if detection is
        realistic and timely." .

:rpn a owl:NamedIndividual, :RatingGroup ;
    rdfs:label "Risk Priority Number" ;
    :hasGroupName "Rating" ;
    :hasTitle "Traditional score calculated from severity, occurrence, and detection" ;
    :hasDescription "Useful but can hide priority differences; do not use blindly." .

:action-priority a owl:NamedIndividual, :RatingGroup ;
    rdfs:label "Action Priority" ;
    :hasGroupName "Rating" ;
    :hasTitle "Priority classification for action based on severity, occurrence, and
        detection" ;
    :hasDescription "Often preferred in AIAG/VDA-style FMEA over relying only on RPN." .

:criticality a owl:NamedIndividual, :RatingGroup ;
    rdfs:label "Criticality" ;
    :hasGroupName "Rating" ;
    :hasTitle "Severity-weighted likelihood or mission/safety importance of a failure
        mode" ;
    :hasDescription "Common in FMECA or reliability-centered analyses." .

:risk-acceptance-criteria a owl:NamedIndividual, :DecisionRuleGroup ;
    rdfs:label "Risk Acceptance Criteria" ;
    :hasGroupName "Decision Rule" ;
    :hasTitle "Defined threshold or rationale for acceptable residual risk" .

:current-control a owl:NamedIndividual, :ControlGroup ;
```

```
    rdfs:label "Current Control" ;
    :hasGroupName "Control" ;
    :hasTitle "Existing prevention, detection, verification, or monitoring activity" ;
    :hasDescription "Controls must be specific and linked to causes or failure modes." .

:prevention-control a owl:NamedIndividual, :ControlGroup ;
    rdfs:label "Prevention Control" ;
    :hasGroupName "Control" ;
    :hasTitle "Control that reduces occurrence by preventing the cause" ;
    :hasDescription "Examples: design margin, derating, poka-yoke, supplier control,
        validated procedure." .

:detection-control a owl:NamedIndividual, :ControlGroup ;
    rdfs:label "Detection Control" ;
    :hasGroupName "Control" ;
    :hasTitle "Control that increases chance of detecting cause, failure mode, or effect
        " ;
    :hasDescription "Examples: inspection, test, diagnostic, alarm, end-of-line
        validation." .

:diagnostic-monitoring a owl:NamedIndividual, :ControlGroup ;
    rdfs:label "Diagnostic Monitoring" ;
    :hasGroupName "Control" ;
    :hasTitle "Runtime detection, isolation, prediction, or response to faults" ;
    :hasDescription "Important for monitored systems, connected products, robots, AI
        services, and safety-critical software." .

:recommended-action a owl:NamedIndividual, :ActionGroup ;
    rdfs:label "Recommended Action" ;
    :hasGroupName "Action" ;
    :hasTitle "Risk-reduction action assigned after analysis" ;
    :hasDescription "Should target severity, occurrence, or detection with clear owner
        and evidence." .

:action-owner a owl:NamedIndividual, :ActionGroup ;
    rdfs:label "Action Owner" ;
    :hasGroupName "Action" ;
    :hasTitle "Person or role responsible for completing the action" .

:action-deadline a owl:NamedIndividual, :ActionGroup ;
    rdfs:label "Action Deadline" ;
    :hasGroupName "Action" ;
    :hasTitle "Due date or decision milestone for the action" ;
    :hasDescription "Connects FMEA to project execution." .

:verification-evidence a owl:NamedIndividual, :EvidenceGroup ;
    rdfs:label "Verification Evidence" ;
    :hasGroupName "Evidence" ;
    :hasTitle "Proof that action or control works" ;
    :hasDescription "Examples: test report, simulation result, inspection data,
        requirement trace, field data, audit record." .

:residual-risk a owl:NamedIndividual, :RiskConceptGroup ;
    rdfs:label "Residual Risk" ;
    :hasGroupName "Risk Concept" ;
    :hasTitle "Risk remaining after controls and actions" ;
    :hasDescription "Must be accepted, reduced further, transferred, or escalated." .

:lessons-learned a owl:NamedIndividual, :KnowledgeGroup ;
    rdfs:label "Lessons Learned" ;
    :hasGroupName "Knowledge" ;
    :hasTitle "Reusable learning from failure, testing, field data, or previous FMEAs" .

:system-fmea a owl:NamedIndividual, :FMEATypeGroup ;
    rdfs:label "System FMEA" ;
    :hasGroupName "FMEA Type" ;
    :hasTitle "Analyzes failures at system architecture and interaction level" ;
```

```
    :hasDescription "Best for early design, interfaces, functions, emergent behavior,
        and system-level hazards." .

:dfmea a owl:NamedIndividual, :FMEATypeGroup ;
    rdfs:label "Design FMEA" ;
    :hasGroupName "FMEA Type" ;
    :hasTitle "Analyzes product or design failure modes before design release" ;
    :hasDescription "Focuses on whether the design itself can meet functions and
        requirements." .

:pfmea a owl:NamedIndividual, :FMEATypeGroup ;
    rdfs:label "Process FMEA" ;
    :hasGroupName "FMEA Type" ;
    :hasTitle "Analyzes manufacturing, assembly, service, or business-process failure
        modes" ;
    :hasDescription "Focuses on process steps, human actions, equipment, materials,
        methods, measurements, and environment." .

:fmea-msr a owl:NamedIndividual, :FMEATypeGroup ;
    rdfs:label "FMEA-MSR" ;
    :hasGroupName "FMEA Type" ;
    :hasTitle "Supplemental FMEA for monitoring and system response" ;
    :hasDescription "Focuses on runtime faults, diagnostic coverage, warning strategy,
        safe state, and controllability." .

:software-fmea a owl:NamedIndividual, :FMEATypeGroup ;
    rdfs:label "Software FMEA" ;
    :hasGroupName "FMEA Type" ;
    :hasTitle "Analyzes software failure modes and their effects" ;
    :hasDescription "Includes logic errors, timing, data, concurrency, interface,
        configuration, update, and exception handling failures." .

:hardware-fmea a owl:NamedIndividual, :FMEATypeGroup ;
    rdfs:label "Hardware FMEA" ;
    :hasGroupName "FMEA Type" ;
    :hasTitle "Analyzes electronic, electrical, mechanical, and physical component
        failures" ;
    :hasDescription "Often linked with reliability data, stress, derating, diagnostics,
        and environmental conditions." .

:fmeca a owl:NamedIndividual, :FMEATypeGroup ;
    rdfs:label "FMECA" ;
    :hasGroupName "FMEA Type" ;
    :hasTitle "FMEA with criticality analysis" ;
    :hasDescription "Adds criticality ranking or quantitative consequence/likelihood
        emphasis." .

:fmeda a owl:NamedIndividual, :FMEATypeGroup ;
    rdfs:label "FMEDA" ;
    :hasGroupName "FMEA Type" ;
    :hasTitle "Failure Modes, Effects, and Diagnostic Analysis" ;
    :hasDescription "Adds diagnostic coverage and failure-rate allocation, often used
        for safety integrity analysis." .

:human-factors-fmea a owl:NamedIndividual, :FMEATypeGroup ;
    rdfs:label "Human Factors FMEA" ;
    :hasGroupName "FMEA Type" ;
    :hasTitle "Analyzes user, operator, maintainer, and organizational interaction
        failures" ;
    :hasDescription "Useful for use error, poor instructions, cognitive load, alarms,
        training, and maintenance." .

:mechanical-failure a owl:NamedIndividual, :FailureDomainGroup ;
    rdfs:label "Mechanical Failure" ;
    :hasGroupName "Failure Domain" ;
    :hasTitle "Failure involving force, motion, geometry, structure, vibration, wear,
        tolerance, fatigue, or contact" ;
```

```
    :hasDescription "Ask about loads, cycles, alignment, lubrication, vibration,
        clearance, stress concentration, and assembly fit." .

:electrical-failure a owl:NamedIndividual, :FailureDomainGroup ;
    rdfs:label "Electrical Failure" ;
    :hasGroupName "Failure Domain" ;
    :hasTitle "Failure involving power, voltage, current, grounding, insulation, wiring,
         or energy distribution" ;
    :hasDescription "Ask about shorts, opens, overload, transients, grounding, connector
         quality, insulation, and supply margins." .

:electronic-failure a owl:NamedIndividual, :FailureDomainGroup ;
    rdfs:label "Electronic Failure" ;
    :hasGroupName "Failure Domain" ;
    :hasTitle "Failure involving electronic components, PCBs, sensors, actuators, ICs,
        solder, or EMC" ;
    :hasDescription "Ask about drift, noise, component aging, ESD, EMI, thermal stress,
        firmware-hardware interaction, and diagnostic coverage." .

:software-failure a owl:NamedIndividual, :FailureDomainGroup ;
    rdfs:label "Software Failure" ;
    :hasGroupName "Failure Domain" ;
    :hasTitle "Failure involving code, algorithms, configuration, state, timing, data,
        APIs, or updates" ;
    :hasDescription "Ask about assumptions, exceptions, race conditions, invalid states,
         rollback, versioning, logging, and test coverage." .

:control-failure a owl:NamedIndividual, :FailureDomainGroup ;
    rdfs:label "Control/System Dynamics Failure" ;
    :hasGroupName "Failure Domain" ;
    :hasTitle "Failure involving feedback loops, stability, estimation, actuation, or
        control logic" ;
    :hasDescription "Ask about sensor delay, actuator saturation, model mismatch,
        unstable loop, calibration, and failover." .

:thermal-failure a owl:NamedIndividual, :FailureDomainGroup ;
    rdfs:label "Thermal Failure" ;
    :hasGroupName "Failure Domain" ;
    :hasTitle "Failure involving heat generation, heat transfer, cooling, thermal
        runaway, or temperature limits" ;
    :hasDescription "Ask about hotspots, cooling path, duty cycle, ambient temperature,
        insulation, and thermal derating." .

:fluidic-failure a owl:NamedIndividual, :FailureDomainGroup ;
    rdfs:label "Fluidic/Pneumatic/Hydraulic Failure" ;
    :hasGroupName "Failure Domain" ;
    :hasTitle "Failure involving liquid, gas, pressure, flow, valves, pumps, seals, or
        contamination" ;
    :hasDescription "Ask about leakage, blockage, cavitation, pressure spikes, filter
        clogging, viscosity, and seal compatibility." .

:chemical-failure a owl:NamedIndividual, :FailureDomainGroup ;
    rdfs:label "Chemical Failure" ;
    :hasGroupName "Failure Domain" ;
    :hasTitle "Failure involving reaction, degradation, corrosion, contamination,
        toxicity, flammability, or material compatibility" ;
    :hasDescription "Ask about chemicals, exposure, cleaning agents, humidity, pH,
        oxidation, and storage conditions." .

:material-failure a owl:NamedIndividual, :FailureDomainGroup ;
    rdfs:label "Material Failure" ;
    :hasGroupName "Failure Domain" ;
    :hasTitle "Failure involving material selection, defects, aging, microstructure,
        creep, fatigue, or environmental degradation" ;
    :hasDescription "Ask about material grade, supplier variation, heat treatment,
        defects, fatigue curves, and aging." .
```

```
:human_error a owl:NamedIndividual, :FailureDomainGroup ;
    rdfs:label "Human Error / Use Error" ;
    :hasGroupName "Failure Domain" ;
    :hasTitle "Failure involving user, operator, maintainer, or team action/inaction" .

:process_failure a owl:NamedIndividual, :FailureDomainGroup ;
    rdfs:label "Process Failure" ;
    :hasGroupName "Failure Domain" ;
    :hasTitle "Failure involving manufacturing, assembly, service, logistics, business
        workflow, or quality process" ;
    :hasDescription "Ask about process step, method, machine, material, measurement,
        manpower, and environment." .

:interface_failure a owl:NamedIndividual, :FailureDomainGroup ;
    rdfs:label "Interface Failure" ;
    :hasGroupName "Failure Domain" ;
    :hasTitle "Failure at the boundary between components, teams, systems, users, or
        organizations" ;
    :hasDescription "Ask about incompatible assumptions, timing, units, protocols,
        tolerances, physical fit, and responsibility handoff." .

:environmental_failure a owl:NamedIndividual, :FailureDomainGroup ;
    rdfs:label "Environmental Failure" ;
    :hasGroupName "Failure Domain" ;
    :hasTitle "Failure caused or triggered by environment" ;
    :hasDescription "Ask about temperature, humidity, dust, water, vibration, shock,
        altitude, salt fog, radiation, EMC, biofouling, and cleaning." .

:cybersecurity_failure a owl:NamedIndividual, :FailureDomainGroup ;
    rdfs:label "Cybersecurity Failure" ;
    :hasGroupName "Failure Domain" ;
    :hasTitle "Failure caused by malicious or unauthorized digital interaction" ;
    :hasDescription "Ask about spoofing, tampering, denial of service, privilege misuse,
         insecure update, secrets, and data exfiltration." .

:data_ai_failure a owl:NamedIndividual, :FailureDomainGroup ;
    rdfs:label "Data / AI Failure" ;
    :hasGroupName "Failure Domain" ;
    :hasTitle "Failure involving data quality, model behavior, AI output, learning
        system drift, or decision support" ;
    :hasDescription "Ask about bias, missing data, hallucination, confidence, monitoring
        , fallback, human override, and auditability." .

:maintenance_failure a owl:NamedIndividual, :FailureDomainGroup ;
    rdfs:label "Maintenance Failure" ;
    :hasGroupName "Failure Domain" ;
    :hasTitle "Failure caused by inadequate, incorrect, delayed, or impossible
        maintenance" ;
    :hasDescription "Ask about maintainability, access, intervals, tooling, instructions
        , spare parts, and condition monitoring." .

:supply_chain_failure a owl:NamedIndividual, :FailureDomainGroup ;
    rdfs:label "Supply Chain Failure" ;
    :hasGroupName "Failure Domain" ;
    :hasTitle "Failure caused by supplier, counterfeit, logistics, inventory, or
        specification mismatch" ;
    :hasDescription "Ask about supplier qualification, change notification, traceability
        , substitute parts, and incoming inspection." .

:loss_of_function a owl:NamedIndividual, :FailureModeCategoryGroup ;
    rdfs:label "Loss of Function" ;
    :hasGroupName "Failure Mode Category" ;
    :hasTitle "Function is completely absent" ;
    :hasDescription "Examples: motor does not start, software service unavailable, valve
         fails to open." .

:partial_function a owl:NamedIndividual, :FailureModeCategoryGroup ;
```

```
    rdfs:label "Partial Function" ;
    :hasGroupName "Failure Mode Category" ;
    :hasTitle "Function occurs but below required level" ;
    :hasDescription "Examples: insufficient torque, weak signal, incomplete weld, low
        flow." .

:degraded_function a owl:NamedIndividual, :FailureModeCategoryGroup ;
    rdfs:label "Degraded Function" ;
    :hasGroupName "Failure Mode Category" ;
    :hasTitle "Function quality worsens over time" ;
    :hasDescription "Examples: sensor drift, wear, battery capacity loss, increasing
        latency." .

:intermittent_function a owl:NamedIndividual, :FailureModeCategoryGroup ;
    rdfs:label "Intermittent Function" ;
    :hasGroupName "Failure Mode Category" ;
    :hasTitle "Function appears and disappears unpredictably" ;
    :hasDescription "Examples: loose connector, race condition, thermal intermittent
        fault." .

:unintended_function a owl:NamedIndividual, :FailureModeCategoryGroup ;
    rdfs:label "Unintended Function" ;
    :hasGroupName "Failure Mode Category" ;
    :hasTitle "Item performs a function when it should not" ;
    :hasDescription "Examples: actuator moves unexpectedly, alarm triggers falsely,
        software sends duplicate command." .

:wrong_function a owl:NamedIndividual, :FailureModeCategoryGroup ;
    rdfs:label "Wrong Function" ;
    :hasGroupName "Failure Mode Category" ;
    :hasTitle "Wrong output, value, action, target, unit, state, or decision" ;
    :hasDescription "Examples: wrong dose, wrong unit conversion, reversed polarity,
        wrong user record." .

:early_function a owl:NamedIndividual, :FailureModeCategoryGroup ;
    rdfs:label "Early Function" ;
    :hasGroupName "Failure Mode Category" ;
    :hasTitle "Function occurs too early" ;
    :hasDescription "Examples: premature release, early trigger, message sent before
        confirmation." .

:late_function a owl:NamedIndividual, :FailureModeCategoryGroup ;
    rdfs:label "Late Function" ;
    :hasGroupName "Failure Mode Category" ;
    :hasTitle "Function occurs too late" .

:unstable_function a owl:NamedIndividual, :FailureModeCategoryGroup ;
    rdfs:label "Unstable / Oscillatory Function" ;
    :hasGroupName "Failure Mode Category" ;
    :hasTitle "Function oscillates, hunts, diverges, or becomes unstable" ;
    :hasDescription "Examples: control loop instability, vibration chatter, repeated
        retry storm." .

:out_of_tolerance a owl:NamedIndividual, :FailureModeCategoryGroup ;
    rdfs:label "Out of Tolerance" ;
    :hasGroupName "Failure Mode Category" ;
    :hasTitle "Output or geometry outside allowed tolerance" ;
    :hasDescription "Examples: dimension out of spec, voltage ripple high, temperature
        outside range." .

:leakage a owl:NamedIndividual, :FailureModeCategoryGroup ;
    rdfs:label "Leakage" ;
    :hasGroupName "Failure Mode Category" ;
    :hasTitle "Unwanted escape of fluid, gas, energy, signal, current, or data" ;
    :hasDescription "Examples: hydraulic leak, current leakage, memory leak,
        confidential data leak." .
```

```
:blockage a owl:NamedIndividual, :FailureModeCategoryGroup ;
    rdfs:label "Blockage / Obstruction" ;
    :hasGroupName "Failure Mode Category" ;
    :hasTitle "Flow, movement, signal, or access is obstructed" ;
    :hasDescription "Examples: clogged filter, blocked nozzle, jammed route, firewall
        blocking required traffic." .

:fracture a owl:NamedIndividual, :MechanicalFailureModeGroup ;
    rdfs:label "Fracture / Breakage" ;
    :hasGroupName "Mechanical Failure Mode" ;
    :hasTitle "Crack, rupture, or break of a part" ;
    :hasDescription "Ask about stress, defects, impact, fatigue, material, notch, and
        overload." .

:fatigue a owl:NamedIndividual, :MechanicalFailureModeGroup ;
    rdfs:label "Fatigue" ;
    :hasGroupName "Mechanical Failure Mode" ;
    :hasTitle "Progressive damage under cyclic loading" ;
    :hasDescription "Ask about load cycles, vibration, stress concentration, surface
        finish, and fatigue limit." .

:wear a owl:NamedIndividual, :MechanicalFailureModeGroup ;
    rdfs:label "Wear" ;
    :hasGroupName "Mechanical Failure Mode" ;
    :hasTitle "Material loss or surface degradation due to contact and motion" ;
    :hasDescription "Ask about lubrication, hardness, particles, alignment, contact
        pressure, and duty cycle." .

:corrosion a owl:NamedIndividual, :ChemicalMaterialFailureModeGroup ;
    rdfs:label "Corrosion" ;
    :hasGroupName "Chemical/Material Failure Mode" ;
    :hasTitle "Material degradation due to chemical or electrochemical reaction" ;
    :hasDescription "Ask about humidity, salt, galvanic couples, coatings, cleaning
        agents, and exposure time." .

:loosening a owl:NamedIndividual, :MechanicalFailureModeGroup ;
    rdfs:label "Loosening" ;
    :hasGroupName "Mechanical Failure Mode" ;
    :hasTitle "Fastener, connector, or fit loses intended retention" ;
    :hasDescription "Ask about torque, vibration, locking method, thermal cycle, and
        assembly procedure." .

:jamming a owl:NamedIndividual, :MechanicalFailureModeGroup ;
    rdfs:label "Jamming / Sticking" ;
    :hasGroupName "Mechanical Failure Mode" ;
    :hasTitle "Moving part cannot move or remains stuck" ;
    :hasDescription "Ask about clearance, debris, lubrication, tolerance stack-up,
        deformation, and ice/dust." .

:overheating a owl:NamedIndividual, :ThermalFailureModeGroup ;
    rdfs:label "Overheating" ;
    :hasGroupName "Thermal Failure Mode" ;
    :hasTitle "Temperature exceeds safe or functional limits" ;
    :hasDescription "Ask about power dissipation, cooling, ambient, duty cycle, thermal
        path, and blocked ventilation." .

:short-circuit a owl:NamedIndividual, :ElectricalFailureModeGroup ;
    rdfs:label "Short Circuit" ;
    :hasGroupName "Electrical Failure Mode" ;
    :hasTitle "Unintended low-resistance electrical path" ;
    :hasDescription "Ask about insulation, moisture, contamination, connector damage,
        solder bridge, and protection." .

:open-circuit a owl:NamedIndividual, :ElectricalFailureModeGroup ;
    rdfs:label "Open Circuit" ;
    :hasGroupName "Electrical Failure Mode" ;
    :hasTitle "Unintended interruption in electrical path" ;
```

```
    :hasDescription "Ask about broken wire, connector disengagement, solder crack, fuse,
        switch, and vibration." .

:insulation-breakdown a owl:NamedIndividual, :ElectricalFailureModeGroup ;
    rdfs:label "Insulation Breakdown" ;
    :hasGroupName "Electrical Failure Mode" ;
    :hasTitle "Loss of dielectric separation" ;
    :hasDescription "Ask about voltage stress, aging, temperature, humidity, creepage,
        clearance, and contamination." .

:emi-emc-susceptibility a owl:NamedIndividual, :ElectronicFailureModeGroup ;
    rdfs:label "EMI/EMC Susceptibility" ;
    :hasGroupName "Electronic Failure Mode" ;
    :hasTitle "Performance affected by electromagnetic interference" ;
    :hasDescription "Ask about shielding, grounding, cable routing, filtering, ESD, and
        immunity testing." .

:sensor-drift a owl:NamedIndividual, :ElectronicControlFailureModeGroup ;
    rdfs:label "Sensor Drift" ;
    :hasGroupName "Electronic/Control Failure Mode" ;
    :hasTitle "Sensor output gradually deviates from true value" ;
    :hasDescription "Ask about calibration, temperature, aging, contamination, and self-
        diagnostics." .

:actuator-stuck a owl:NamedIndividual, :MechanicalControlFailureModeGroup ;
    rdfs:label "Actuator Stuck" ;
    :hasGroupName "Mechanical/Control Failure Mode" ;
    :hasTitle "Actuator remains stuck on, off, open, closed, or at a position" ;
    :hasDescription "Ask about command signal, mechanical blockage, power loss, valve
        stiction, and fail-safe position." .

:power-loss a owl:NamedIndividual, :ElectricalFailureModeGroup ;
    rdfs:label "Power Loss / Brownout" ;
    :hasGroupName "Electrical Failure Mode" ;
    :hasTitle "Power absent, unstable, or below required level" ;
    :hasDescription "Ask about supply margin, battery, wiring, fuse, inrush, load
        transient, and backup power." .

:communication-loss a owl:NamedIndividual, :InterfaceSoftwareFailureModeGroup ;
    rdfs:label "Communication Loss" ;
    :hasGroupName "Interface/Software Failure Mode" ;
    :hasTitle "Required data exchange unavailable, delayed, corrupted, or incompatible"
        ;
    :hasDescription "Ask about protocol, timeout, retry, bandwidth, network, message
        schema, and fallback." .

:software-crash a owl:NamedIndividual, :SoftwareFailureModeGroup ;
    rdfs:label "Software Crash" ;
    :hasGroupName "Software Failure Mode" ;
    :hasTitle "Process, service, task, or controller stops unexpectedly" ;
    :hasDescription "Ask about exceptions, memory, watchdog, resource exhaustion,
        dependency failure, and recovery." .

:timing-race-condition a owl:NamedIndividual, :SoftwareControlFailureModeGroup ;
    rdfs:label "Timing / Race Condition" ;
    :hasGroupName "Software/Control Failure Mode" ;
    :hasTitle "Behavior depends on unsafe timing or event ordering" ;
    :hasDescription "Ask about concurrency, locks, interrupts, asynchronous messages,
        scheduling, and state transitions." .

:data-corruption a owl:NamedIndividual, :DataFailureModeGroup ;
    rdfs:label "Data Corruption" ;
    :hasGroupName "Data Failure Mode" ;
    :hasTitle "Stored or transmitted data becomes wrong, incomplete, stale, duplicated,
        or inconsistent" ;
    :hasDescription "Ask about validation, checksum, transaction, schema, time stamp,
        synchronization, and recovery." .
```

```
:model_misclassification a owl:NamedIndividual, :AIFailureModeGroup ;
    rdfs:label "AI Model Misclassification" ;
    :hasGroupName "AI Failure Mode" ;
    :hasTitle "AI model produces wrong class, recommendation, or interpretation" ;
    :hasDescription "Ask about training data, distribution shift, confidence, edge cases
        , explainability, and human review." .

:requirement_ambiguity a owl:NamedIndividual, :CauseCategoryGroup ;
    rdfs:label "Requirement Ambiguity" ;
    :hasGroupName "Cause Category" ;
    :hasTitle "Requirement is unclear, incomplete, conflicting, unverifiable, or
        interpreted differently" ;
    :hasDescription "Ask: which word can two engineers interpret differently?" .

:design_weakness a owl:NamedIndividual, :CauseCategoryGroup ;
    rdfs:label "Design Weakness" ;
    :hasGroupName "Cause Category" ;
    :hasTitle "Design lacks margin, robustness, compatibility, diagnosability, or safe
        behavior" ;
    :hasDescription "Ask whether the failure is designed_in rather than produced by
        manufacturing." .

:manufacturing_defect a owl:NamedIndividual, :CauseCategoryGroup ;
    rdfs:label "Manufacturing Defect" ;
    :hasGroupName "Cause Category" ;
    :hasTitle "Part or product deviates from design intent during manufacturing" ;
    :hasDescription "Examples: wrong dimension, poor solder, void, burr, incorrect cure,
         contamination." .

:assembly_error a owl:NamedIndividual, :CauseCategoryGroup ;
    rdfs:label "Assembly Error" ;
    :hasGroupName "Cause Category" ;
    :hasTitle "Incorrect assembly, orientation, torque, connection, routing, or
        configuration" ;
    :hasDescription "Ask about poka_yoke, work instructions, fixtures, visual inspection
        , and training." .

:contamination a owl:NamedIndividual, :CauseCategoryGroup ;
    rdfs:label "Contamination" ;
    :hasGroupName "Cause Category" ;
    :hasTitle "Unwanted material affects function" ;
    :hasDescription "Examples: dust, oil, water, particles, microbes, flux residue,
        metal shavings." .

:overload a owl:NamedIndividual, :CauseCategoryGroup ;
    rdfs:label "Overload" ;
    :hasGroupName "Cause Category" ;
    :hasTitle "Load, stress, current, pressure, data traffic, or usage exceeds design
        capability" ;
    :hasDescription "Ask about worst_case assumptions and abnormal but foreseeable use."
         .

:aging_degradation a owl:NamedIndividual, :CauseCategoryGroup ;
    rdfs:label "Aging / Degradation" ;
    :hasGroupName "Cause Category" ;
    :hasTitle "Performance reduces over time due to aging, wear, chemical change, or
        fatigue" ;
    :hasDescription "Ask about lifetime, cycles, storage, maintenance, and degradation
        monitoring." .

:supplier_variation a owl:NamedIndividual, :CauseCategoryGroup ;
    rdfs:label "Supplier Variation" ;
    :hasGroupName "Cause Category" ;
    :hasTitle "Supplier part, material, process, or documentation varies from
        expectation" ;
    :hasDescription "Ask about specification, qualification, substitutions, certificates
```

```
        , and change notification." .

:user-misuse a owl:NamedIndividual, :CauseCategoryGroup ;
    rdfs:label "Use Error / Misuse" ;
    :hasGroupName "Cause Category" ;
    :hasTitle "User action differs from intended use, including foreseeable misuse" ;
    :hasDescription "Ask whether design, labeling, training, or workflow makes the
        misuse likely." .

:inadequate-maintenance a owl:NamedIndividual, :CauseCategoryGroup ;
    rdfs:label "Inadequate Maintenance" ;
    :hasGroupName "Cause Category" ;
    :hasTitle "Maintenance is omitted, late, wrong, undocumented, or impossible under
        real conditions" ;
    :hasDescription "Ask about access, tools, intervals, instructions, and condition-
        based monitoring." .

:external-event a owl:NamedIndividual, :CauseCategoryGroup ;
    rdfs:label "External Event" ;
    :hasGroupName "Cause Category" ;
    :hasTitle "External shock, outage, disaster, regulatory change, cyber event, or
        environmental condition triggers failure" ;
    :hasDescription "Ask what external events are credible, not just convenient." .

:cyberattack a owl:NamedIndividual, :CauseCategoryGroup ;
    rdfs:label "Cyberattack" ;
    :hasGroupName "Cause Category" ;
    :hasTitle "Malicious digital action causes unsafe, unavailable, incorrect, or
        unauthorized behavior" ;
    :hasDescription "Ask about threat actor, attack surface, authentication, logging,
        and isolation." .

:software-defect a owl:NamedIndividual, :CauseCategoryGroup ;
    rdfs:label "Software Defect" ;
    :hasGroupName "Cause Category" ;
    :hasTitle "Bug, configuration error, logic error, dependency problem, or update
        fault causes failure" ;
    :hasDescription "Ask about tests, versioning, rollback, observability, and exception
         handling." .

:design-review a owl:NamedIndividual, :ControlMethodGroup ;
    rdfs:label "Design Review" ;
    :hasGroupName "Control Method" ;
    :hasTitle "Structured review of design intent, interfaces, assumptions, margins, and
         failure logic" ;
    :hasDescription "Good for catching conceptual and interface failures early." .

:simulation-analysis a owl:NamedIndividual, :ControlMethodGroup ;
    rdfs:label "Simulation / Analysis" ;
    :hasGroupName "Control Method" ;
    :hasTitle "Model-based analysis of behavior under nominal, worst-case, and fault
        conditions" ;
    :hasDescription "Examples: dynamics simulation, circuit simulation, thermal model,
        process simulation." .

:tolerance-analysis a owl:NamedIndividual, :ControlMethodGroup ;
    rdfs:label "Tolerance Stack-up Analysis" ;
    :hasGroupName "Control Method" ;
    :hasTitle "Checks whether geometric and process variation can break function" ;
    :hasDescription "Useful for fit, alignment, leakage, jamming, and out-of-tolerance
        failures." .

:finite-element-analysis a owl:NamedIndividual, :ControlMethodGroup ;
    rdfs:label "Finite Element Analysis" ;
    :hasGroupName "Control Method" ;
    :hasTitle "Analyzes stress, strain, vibration, thermal behavior, or structural
        margins" ;
```

```
    :hasDescription "Useful for fracture, fatigue, deformation, and thermal–mechanical
        problems." .

:fault–tree–analysis a owl:NamedIndividual, :ControlMethodGroup ;
    rdfs:label "Fault Tree Analysis" ;
    :hasGroupName "Control Method" ;
    :hasTitle "Top–down logic analysis from undesired event to combinations of causes" ;
    :hasDescription "Complements FMEA by asking how causes combine." .

:test–validation a owl:NamedIndividual, :ControlMethodGroup ;
    rdfs:label "Test and Validation" ;
    :hasGroupName "Control Method" ;
    :hasTitle "Empirical proof under defined conditions that item or process meets needs
        " ;
    :hasDescription "Must be tied to requirements, failure modes, and acceptance
        criteria." .

:inspection a owl:NamedIndividual, :ControlMethodGroup ;
    rdfs:label "Inspection" ;
    :hasGroupName "Control Method" ;
    :hasTitle "Examination, measurement, or audit to detect defects or nonconformance" ;
    :hasDescription "Can be visual, dimensional, electrical, software, document, or
        process inspection." .

:poka–yoke a owl:NamedIndividual, :ControlMethodGroup ;
    rdfs:label "Poka–Yoke / Error Proofing" ;
    :hasGroupName "Control Method" ;
    :hasTitle "Design or process feature that prevents incorrect action or assembly" ;
    :hasDescription "Best when humans are involved and mistakes are predictable." .

:redundancy a owl:NamedIndividual, :ControlMethodGroup ;
    rdfs:label "Redundancy" ;
    :hasGroupName "Control Method" ;
    :hasTitle "Additional element or path that maintains function after a failure" ;
    :hasDescription "Can reduce severity or occurrence of system effect but may add
        complexity." .

:derating a owl:NamedIndividual, :ControlMethodGroup ;
    rdfs:label "Derating" ;
    :hasGroupName "Control Method" ;
    :hasTitle "Operating component below maximum rating to improve reliability" ;
    :hasDescription "Common for electrical, thermal, mechanical, and material stress." .

:safety–margin a owl:NamedIndividual, :ControlMethodGroup ;
    rdfs:label "Safety Margin" ;
    :hasGroupName "Control Method" ;
    :hasTitle "Difference between expected load/stress and capability/limit" .

:fail–safe–state a owl:NamedIndividual, :ControlMethodGroup ;
    rdfs:label "Fail–Safe State" ;
    :hasGroupName "Control Method" ;
    :hasTitle "Defined state that minimizes harm when failure is detected or likely" ;
    :hasDescription "Examples: stop motion, open valve, close valve, isolate power,
        disable output." .

:graceful–degradation a owl:NamedIndividual, :ControlMethodGroup ;
    rdfs:label "Graceful Degradation" ;
    :hasGroupName "Control Method" ;
    :hasTitle "Reduced but controlled functionality after partial failure" ;
    :hasDescription "Useful for resilience and user trust." .

:alarm–alert a owl:NamedIndividual, :ControlMethodGroup ;
    rdfs:label "Alarm / Alert" ;
    :hasGroupName "Control Method" ;
    :hasTitle "Signal that informs user, operator, maintainer, or system about abnormal
        condition" .
```

```
:maintenance-plan a owl:NamedIndividual, :ControlMethodGroup ;
    rdfs:label "Maintenance Plan" ;
    :hasGroupName "Control Method" ;
    :hasTitle "Scheduled or condition-based tasks to preserve function" ;
    :hasDescription "Includes inspection, replacement, calibration, cleaning,
        lubrication, and software updates." .

:training-instruction a owl:NamedIndividual, :ControlMethodGroup ;
    rdfs:label "Training / Instruction" ;
    :hasGroupName "Control Method" ;
    :hasTitle "Human guidance for correct operation, assembly, service, or response" ;
    :hasDescription "Should not be the only control for high-severity risks if design
        control is possible." .

:change-control a owl:NamedIndividual, :ControlMethodGroup ;
    rdfs:label "Change Control" ;
    :hasGroupName "Control Method" ;
    :hasTitle "Process for evaluating and approving changes to design, process, software
        , supplier, or requirement" .

:traceability a owl:NamedIndividual, :ControlMethodGroup ;
    rdfs:label "Traceability" ;
    :hasGroupName "Control Method" ;
    :hasTitle "Links requirements, functions, design elements, tests, risks, actions,
        and evidence" ;
    :hasDescription "Helps AI explain why a failure matters and what evidence closes it.
        " .

:question-context a owl:NamedIndividual, :AIQuestionTemplateGroup ;
    rdfs:label "Context Questions" ;
    :hasGroupName "AI Question Template" ;
    :hasTitle "Questions that define scope, environment, lifecycle, assumptions, and
        stakeholders" ;
    :hasDescription "Use before generating failure modes." .

:question-function a owl:NamedIndividual, :AIQuestionTemplateGroup ;
    rdfs:label "Function Questions" ;
    :hasGroupName "AI Question Template" ;
    :hasTitle "Questions that elicit intended function, measurable outputs, and
        operating modes" ;
    :hasDescription "Use to derive failure modes from what the item must do." .

:question-interface a owl:NamedIndividual, :AIQuestionTemplateGroup ;
    rdfs:label "Interface Questions" ;
    :hasGroupName "AI Question Template" ;
    :hasTitle "Questions about exchanged energy, material, information, force, heat,
        fluid, or control" ;
    :hasDescription "Use to find failures hidden between parts, teams, or systems." .

:question-failure-mode a owl:NamedIndividual, :AIQuestionTemplateGroup ;
    rdfs:label "Failure Mode Questions" ;
    :hasGroupName "AI Question Template" ;
    :hasTitle "Questions that transform each function into possible deviations" ;
    :hasDescription "Use patterns: no, partial, degraded, intermittent, unintended,
        wrong, early, late, unstable, out-of-tolerance." .

:question-effect a owl:NamedIndividual, :AIQuestionTemplateGroup ;
    rdfs:label "Effect Questions" ;
    :hasGroupName "AI Question Template" ;
    :hasTitle "Questions that trace consequences from local effect to end-user or harm"
        ;
    :hasDescription "Use to avoid confusing effects with causes." .

:question-cause a owl:NamedIndividual, :AIQuestionTemplateGroup ;
    rdfs:label "Cause Questions" ;
    :hasGroupName "AI Question Template" ;
    :hasTitle "Questions that find physical, logical, process, human, environmental, and
```

```
        cyber causes" ;
    :hasDescription "Use repeated 'why' carefully until a controllable or testable cause
        appears." .

:question-control a owl:NamedIndividual, :AIQuestionTemplateGroup ;
    rdfs:label "Control Questions" ;
    :hasGroupName "AI Question Template" ;
    :hasTitle "Questions that identify prevention, detection, diagnostic, monitoring,
        and response controls" ;
    :hasDescription "Use to check whether current controls actually target the cause or
        failure mode." .

:question-evidence a owl:NamedIndividual, :AIQuestionTemplateGroup ;
    rdfs:label "Evidence Questions" ;
    :hasGroupName "AI Question Template" ;
    :hasTitle "Questions about data, tests, simulation, inspections, field history, and
        confidence" .

:question-counterfactual a owl:NamedIndividual, :AIQuestionTemplateGroup ;
    rdfs:label "Counterfactual Questions" ;
    :hasGroupName "AI Question Template" ;
    :hasTitle "Questions that ask what must be true for the failure not to occur" ;
    :hasDescription "Useful for finding missing controls and assumptions." .

:question-cross-domain a owl:NamedIndividual, :AIQuestionTemplateGroup ;
    rdfs:label "Cross-Domain Questions" ;
    :hasGroupName "AI Question Template" ;
    :hasTitle "Questions that force review across mechanical, electrical, software,
        human, process, cyber, and environmental domains" .

:question-missing-info a owl:NamedIndividual, :AIQuestionTemplateGroup ;
    rdfs:label "Missing Information Questions" ;
    :hasGroupName "AI Question Template" ;
    :hasTitle "Questions that identify unknowns blocking reliable analysis" ;
    :hasDescription "Use to ask for drawings, requirements, schematics, process maps,
        data, supplier specs, and operating context." .

:question-prioritization a owl:NamedIndividual, :AIQuestionTemplateGroup ;
    rdfs:label "Prioritization Questions" ;
    :hasGroupName "AI Question Template" ;
    :hasTitle "Questions that rank risks and actions" ;
    :hasDescription "Use to decide what to address first based on severity, occurrence,
        detection, criticality, uncertainty, and stakeholder impact." .

:rule-01-function-first a owl:NamedIndividual, :ReasoningRule ;
    rdfs:label "rule-01-function-first" ;
    :hasStatement "Before proposing failure modes, identify the intended function,
        operating mode, environment, input, output, performance parameter, and
        requirement." .

:rule-02-separate-mode-cause-effect a owl:NamedIndividual, :ReasoningRule ;
    rdfs:label "rule-02-separate-mode-cause-effect" ;
    :hasStatement "Do not mix failure mode, cause, and effect. A failure mode is what
        goes wrong with the function; a cause is why it happens; an effect is what
        happens because of it." .

:rule-03-levels-of-effect a owl:NamedIndividual, :ReasoningRule ;
    rdfs:label "rule-03-levels-of-effect" ;
    :hasStatement "Always ask for local effect, next-higher-level effect, system effect,
        end-user effect, and safety/regulatory/business effect when relevant." .

:rule-04-cross-domain-sweep a owl:NamedIndividual, :ReasoningRule ;
    rdfs:label "rule-04-cross-domain-sweep" ;
    :hasStatement "For every important function, sweep across mechanical, electrical,
        electronic, software, control, thermal, fluidic, chemical, material, human,
        process, interface, environmental, cybersecurity, data/AI, maintenance, and
        supply-chain domains." .
```

```
:rule-05-interface-suspicion a owl:NamedIndividual, :ReasoningRule ;
    rdfs:label "rule-05-interface-suspicion" ;
    :hasStatement "Treat interfaces as high-risk zones because mismatched assumptions,
        timing, units, tolerance, protocol, or responsibility boundaries often create
        hidden failures." .

:rule-06-evidence-over-confidence a owl:NamedIndividual, :ReasoningRule ;
    rdfs:label "rule-06-evidence-over-confidence" ;
    :hasStatement "Ratings and claims about controls should cite evidence: test data,
        field data, simulations, inspections, process capability, supplier records, or
        expert rationale." .

:rule-07-current-control-specificity a owl:NamedIndividual, :ReasoningRule ;
    rdfs:label "rule-07-current-control-specificity" ;
    :hasStatement "A current control must state what it controls: cause prevention,
        failure detection, effect mitigation, diagnostic monitoring, or response." .

:rule-08-action-target a owl:NamedIndividual, :ReasoningRule ;
    rdfs:label "rule-08-action-target" ;
    :hasStatement "Every recommended action should explicitly target severity,
        occurrence, detection, diagnostic coverage, uncertainty, or traceability." .

:rule-09-residual-risk a owl:NamedIndividual, :ReasoningRule ;
    rdfs:label "rule-09-residual-risk" ;
    :hasStatement "After actions, reassess severity, occurrence, detection/action
        priority, and record residual risk with acceptance rationale." .

:rule-10-ai-follow-up a owl:NamedIndividual, :ReasoningRule ;
    rdfs:label "rule-10-ai-follow-up" ;
    :hasStatement "If key context is missing, ask targeted questions instead of
        inventing assumptions; if forced to proceed, mark assumptions explicitly." .

:iec-60812-2018 :informs :fmea-ontology .
:edge-001 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :iec-60812-2018 ;
    :hasTarget :fmea-ontology ;
    :hasRelationLabel "informs" .

:aiag-vda-fmea-handbook :informs :fmea-ontology .
:edge-002 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :aiag-vda-fmea-handbook ;
    :hasTarget :fmea-ontology ;
    :hasRelationLabel "informs" .

:sae-j1739 :informs :fmea-ontology .
:edge-003 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :sae-j1739 ;
    :hasTarget :fmea-ontology ;
    :hasRelationLabel "informs" .

:iso-14971-risk-thinking :informs :fmea-ontology .
:edge-004 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :iso-14971-risk-thinking ;
    :hasTarget :fmea-ontology ;
    :hasRelationLabel "informs" .

:fmea-ontology :enables :ai-fmea-facilitator .
:edge-005 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :fmea-ontology ;
    :hasTarget :ai-fmea-facilitator ;
    :hasRelationLabel "enables" .

:ai-fmea-facilitator :guides :fmea-study .
:edge-006 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :ai-fmea-facilitator ;
    :hasTarget :fmea-study ;
```

```
    :hasRelationLabel "guides" .

:fmea-study :contains :fmea-record .
:edge-007 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :fmea-study ;
    :hasTarget :fmea-record ;
    :hasRelationLabel "contains" .

:planning-preparation :precedes :structure-analysis .
:edge-008 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :planning-preparation ;
    :hasTarget :structure-analysis ;
    :hasRelationLabel "precedes" .

:structure-analysis :precedes :function-analysis .
:edge-009 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :structure-analysis ;
    :hasTarget :function-analysis ;
    :hasRelationLabel "precedes" .

:function-analysis :precedes :failure-analysis .
:edge-010 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :function-analysis ;
    :hasTarget :failure-analysis ;
    :hasRelationLabel "precedes" .

:failure-analysis :precedes :risk-analysis .
:edge-011 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :failure-analysis ;
    :hasTarget :risk-analysis ;
    :hasRelationLabel "precedes" .

:risk-analysis :precedes :optimization .
:edge-012 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :risk-analysis ;
    :hasTarget :optimization ;
    :hasRelationLabel "precedes" .

:optimization :precedes :results-documentation .
:edge-013 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :optimization ;
    :hasTarget :results-documentation ;
    :hasRelationLabel "precedes" .

:aiag-vda-fmea-handbook :usesStep :planning-preparation .
:edge-014 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :aiag-vda-fmea-handbook ;
    :hasTarget :planning-preparation ;
    :hasRelationLabel "uses step" .

:aiag-vda-fmea-handbook :usesStep :structure-analysis .
:edge-015 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :aiag-vda-fmea-handbook ;
    :hasTarget :structure-analysis ;
    :hasRelationLabel "uses step" .

:aiag-vda-fmea-handbook :usesStep :function-analysis .
:edge-016 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :aiag-vda-fmea-handbook ;
    :hasTarget :function-analysis ;
    :hasRelationLabel "uses step" .

:aiag-vda-fmea-handbook :usesStep :failure-analysis .
:edge-017 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :aiag-vda-fmea-handbook ;
    :hasTarget :failure-analysis ;
    :hasRelationLabel "uses step" .
```

```
:aiag_vda_fmea_handbook :usesStep :risk_analysis .
:edge_018 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :aiag_vda_fmea_handbook ;
    :hasTarget :risk_analysis ;
    :hasRelationLabel "uses step" .

:aiag_vda_fmea_handbook :usesStep :optimization .
:edge_019 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :aiag_vda_fmea_handbook ;
    :hasTarget :optimization ;
    :hasRelationLabel "uses step" .

:aiag_vda_fmea_handbook :usesStep :results_documentation .
:edge_020 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :aiag_vda_fmea_handbook ;
    :hasTarget :results_documentation ;
    :hasRelationLabel "uses step" .

:planning_preparation :defines :scope .
:edge_021 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :planning_preparation ;
    :hasTarget :scope ;
    :hasRelationLabel "defines" .

:planning_preparation :defines :boundary .
:edge_022 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :planning_preparation ;
    :hasTarget :boundary ;
    :hasRelationLabel "defines" .

:planning_preparation :defines :stakeholder .
:edge_023 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :planning_preparation ;
    :hasTarget :stakeholder ;
    :hasRelationLabel "defines" .

:planning_preparation :defines :lifecycle_phase .
:edge_024 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :planning_preparation ;
    :hasTarget :lifecycle_phase ;
    :hasRelationLabel "defines" .

:planning_preparation :defines :assumption .
:edge_025 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :planning_preparation ;
    :hasTarget :assumption ;
    :hasRelationLabel "defines" .

:planning_preparation :defines :risk_acceptance_criteria .
:edge_026 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :planning_preparation ;
    :hasTarget :risk_acceptance_criteria ;
    :hasRelationLabel "defines" .

:structure_analysis :identifies :item .
:edge_027 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :structure_analysis ;
    :hasTarget :item ;
    :hasRelationLabel "identifies" .

:structure_analysis :identifies :system .
:edge_028 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :structure_analysis ;
    :hasTarget :system ;
    :hasRelationLabel "identifies" .
```

```
:structure–analysis :identifies :subsystem .
:edge–029 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :structure–analysis ;
    :hasTarget :subsystem ;
    :hasRelationLabel "identifies" .

:structure–analysis :identifies :component .
:edge–030 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :structure–analysis ;
    :hasTarget :component ;
    :hasRelationLabel "identifies" .

:structure–analysis :identifies :interface .
:edge–031 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :structure–analysis ;
    :hasTarget :interface ;
    :hasRelationLabel "identifies" .

:structure–analysis :identifies :environment .
:edge–032 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :structure–analysis ;
    :hasTarget :environment ;
    :hasRelationLabel "identifies" .

:function–analysis :identifies :requirement .
:edge–033 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :function–analysis ;
    :hasTarget :requirement ;
    :hasRelationLabel "identifies" .

:function–analysis :identifies :function .
:edge–034 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :function–analysis ;
    :hasTarget :function ;
    :hasRelationLabel "identifies" .

:function–analysis :identifies :primary–function .
:edge–035 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :function–analysis ;
    :hasTarget :primary–function ;
    :hasRelationLabel "identifies" .

:function–analysis :identifies :secondary–function .
:edge–036 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :function–analysis ;
    :hasTarget :secondary–function ;
    :hasRelationLabel "identifies" .

:function–analysis :identifies :safety–function .
:edge–037 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :function–analysis ;
    :hasTarget :safety–function ;
    :hasRelationLabel "identifies" .

:function–analysis :identifies :diagnostic–function .
:edge–038 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :function–analysis ;
    :hasTarget :diagnostic–function ;
    :hasRelationLabel "identifies" .

:function–analysis :identifies :performance–parameter .
:edge–039 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :function–analysis ;
    :hasTarget :performance–parameter ;
    :hasRelationLabel "identifies" .

:function–analysis :identifies :functional–chain .
```

```
:edge_040 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :function_analysis ;
    :hasTarget :functional_chain ;
    :hasRelationLabel "identifies" .

:function_analysis :identifies :input .
:edge_041 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :function_analysis ;
    :hasTarget :input ;
    :hasRelationLabel "identifies" .

:function_analysis :identifies :output .
:edge_042 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :function_analysis ;
    :hasTarget :output ;
    :hasRelationLabel "identifies" .

:function_analysis :identifies :constraint .
:edge_043 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :function_analysis ;
    :hasTarget :constraint ;
    :hasRelationLabel "identifies" .

:function_analysis :identifies :operating_mode .
:edge_044 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :function_analysis ;
    :hasTarget :operating_mode ;
    :hasRelationLabel "identifies" .

:function_analysis :identifies :use_scenario .
:edge_045 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :function_analysis ;
    :hasTarget :use_scenario ;
    :hasRelationLabel "identifies" .

:failure_analysis :identifies :failure_mode .
:edge_046 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :failure_analysis ;
    :hasTarget :failure_mode ;
    :hasRelationLabel "identifies" .

:failure_analysis :identifies :failure_effect .
:edge_047 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :failure_analysis ;
    :hasTarget :failure_effect ;
    :hasRelationLabel "identifies" .

:failure_analysis :identifies :failure_cause .
:edge_048 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :failure_analysis ;
    :hasTarget :failure_cause ;
    :hasRelationLabel "identifies" .

:failure_analysis :identifies :failure_mechanism .
:edge_049 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :failure_analysis ;
    :hasTarget :failure_mechanism ;
    :hasRelationLabel "identifies" .

:failure_analysis :identifies :hazard .
:edge_050 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :failure_analysis ;
    :hasTarget :hazard ;
    :hasRelationLabel "identifies" .

:failure_analysis :identifies :hazardous_situation .
:edge_051 a owl:NamedIndividual, :OntologyEdge ;
```

```
    :hasSource :failure-analysis ;
    :hasTarget :hazardous-situation ;
    :hasRelationLabel "identifies" .

:failure-analysis :identifies :harm .
:edge-052 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :failure-analysis ;
    :hasTarget :harm ;
    :hasRelationLabel "identifies" .

:risk-analysis :evaluates :severity .
:edge-053 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :risk-analysis ;
    :hasTarget :severity ;
    :hasRelationLabel "evaluates" .

:risk-analysis :evaluates :occurrence .
:edge-054 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :risk-analysis ;
    :hasTarget :occurrence ;
    :hasRelationLabel "evaluates" .

:risk-analysis :evaluates :detection-rating .
:edge-055 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :risk-analysis ;
    :hasTarget :detection-rating ;
    :hasRelationLabel "evaluates" .

:risk-analysis :evaluates :rpn .
:edge-056 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :risk-analysis ;
    :hasTarget :rpn ;
    :hasRelationLabel "evaluates" .

:risk-analysis :evaluates :action-priority .
:edge-057 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :risk-analysis ;
    :hasTarget :action-priority ;
    :hasRelationLabel "evaluates" .

:risk-analysis :evaluates :criticality .
:edge-058 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :risk-analysis ;
    :hasTarget :criticality ;
    :hasRelationLabel "evaluates" .

:risk-analysis :evaluates :risk .
:edge-059 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :risk-analysis ;
    :hasTarget :risk ;
    :hasRelationLabel "evaluates" .

:optimization :improves :recommended-action .
:edge-060 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :optimization ;
    :hasTarget :recommended-action ;
    :hasRelationLabel "improves" .

:optimization :improves :prevention-control .
:edge-061 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :optimization ;
    :hasTarget :prevention-control ;
    :hasRelationLabel "improves" .

:optimization :improves :detection-control .
:edge-062 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :optimization ;
```

```
    :hasTarget :detection-control ;
    :hasRelationLabel "improves" .

:optimization :improves :diagnostic-monitoring .
:edge-063 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :optimization ;
    :hasTarget :diagnostic-monitoring ;
    :hasRelationLabel "improves" .

:optimization :improves :residual-risk .
:edge-064 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :optimization ;
    :hasTarget :residual-risk ;
    :hasRelationLabel "improves" .

:results-documentation :captures :verification-evidence .
:edge-065 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :results-documentation ;
    :hasTarget :verification-evidence ;
    :hasRelationLabel "captures" .

:results-documentation :captures :lessons-learned .
:edge-066 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :results-documentation ;
    :hasTarget :lessons-learned ;
    :hasRelationLabel "captures" .

:results-documentation :captures :traceability .
:edge-067 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :results-documentation ;
    :hasTarget :traceability ;
    :hasRelationLabel "captures" .

:scope :sets :boundary .
:edge-068 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :scope ;
    :hasTarget :boundary ;
    :hasRelationLabel "sets" .

:scope :appliesTo :lifecycle-phase .
:edge-069 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :scope ;
    :hasTarget :lifecycle-phase ;
    :hasRelationLabel "applies to" .

:scope :appliesTo :operating-mode .
:edge-070 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :scope ;
    :hasTarget :operating-mode ;
    :hasRelationLabel "applies to" .

:scope :includes :environment .
:edge-071 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :scope ;
    :hasTarget :environment ;
    :hasRelationLabel "includes" .

:system :contains :subsystem .
:edge-072 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :system ;
    :hasTarget :subsystem ;
    :hasRelationLabel "contains" .

:subsystem :contains :component .
:edge-073 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :subsystem ;
    :hasTarget :component ;
```

```
    :hasRelationLabel "contains" .

:component :has :interface .
:edge_074 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :component ;
    :hasTarget :interface ;
    :hasRelationLabel "has" .

:system :has :interface .
:edge_075 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :system ;
    :hasTarget :interface ;
    :hasRelationLabel "has" .

:interface :transfers :input .
:edge_076 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :interface ;
    :hasTarget :input ;
    :hasRelationLabel "transfers" .

:interface :transfers :output .
:edge_077 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :interface ;
    :hasTarget :output ;
    :hasRelationLabel "transfers" .

:environment :influences :operating_mode .
:edge_078 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :environment ;
    :hasTarget :operating_mode ;
    :hasRelationLabel "influences" .

:use_scenario :contains :operating_mode .
:edge_079 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :use_scenario ;
    :hasTarget :operating_mode ;
    :hasRelationLabel "contains" .

:stakeholder :statesOrOwns :requirement .
:edge_080 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :stakeholder ;
    :hasTarget :requirement ;
    :hasRelationLabel "states or owns" .

:requirement :constrains :function .
:edge_081 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :requirement ;
    :hasTarget :function ;
    :hasRelationLabel "constrains" .

:constraint :limits :function .
:edge_082 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :constraint ;
    :hasTarget :function ;
    :hasRelationLabel "limits" .

:item :performs :function .
:edge_083 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :item ;
    :hasTarget :function ;
    :hasRelationLabel "performs" .

:component :performs :function .
:edge_084 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :component ;
    :hasTarget :function ;
    :hasRelationLabel "performs" .
```

```
:system :performs :primary_function .
:edge_085 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :system ;
    :hasTarget :primary_function ;
    :hasRelationLabel "performs" .

:component :performs :secondary_function .
:edge_086 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :component ;
    :hasTarget :secondary_function ;
    :hasRelationLabel "performs" .

:safety_function :controls :hazard .
:edge_087 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :safety_function ;
    :hasTarget :hazard ;
    :hasRelationLabel "controls" .

:diagnostic_function :implements :detection_control .
:edge_088 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :diagnostic_function ;
    :hasTarget :detection_control ;
    :hasRelationLabel "implements" .

:function :isMeasuredBy :performance_parameter .
:edge_089 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :function ;
    :hasTarget :performance_parameter ;
    :hasRelationLabel "is measured by" .

:functional_chain :orders :function .
:edge_090 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :functional_chain ;
    :hasTarget :function ;
    :hasRelationLabel "orders" .

:input :feeds :function .
:edge_091 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :input ;
    :hasTarget :function ;
    :hasRelationLabel "feeds" .

:function :produces :output .
:edge_092 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :function ;
    :hasTarget :output ;
    :hasRelationLabel "produces" .

:operating_mode :conditions :function .
:edge_093 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :operating_mode ;
    :hasTarget :function ;
    :hasRelationLabel "conditions" .

:environment :conditions :function .
:edge_094 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :environment ;
    :hasTarget :function ;
    :hasRelationLabel "conditions" .

:function :canDeviateAs :failure_mode .
:edge_095 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :function ;
    :hasTarget :failure_mode ;
    :hasRelationLabel "can deviate as" ;
    :hasQuestionPrompt "For this function, what does no/partial/degraded/intermittent/
```

```
        unintended/wrong/early/late/unstable/out-of-tolerance look like?" .

:failure-mode :causes :failure-effect .
:edge-096 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :failure-mode ;
    :hasTarget :failure-effect ;
    :hasRelationLabel "causes" .

:failure-effect :canBeClassifiedAs :local-effect .
:edge-097 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :failure-effect ;
    :hasTarget :local-effect ;
    :hasRelationLabel "can be classified as" .

:failure-effect :canBeClassifiedAs :next-higher-level-effect .
:edge-098 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :failure-effect ;
    :hasTarget :next-higher-level-effect ;
    :hasRelationLabel "can be classified as" .

:failure-effect :canBeClassifiedAs :end-user-effect .
:edge-099 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :failure-effect ;
    :hasTarget :end-user-effect ;
    :hasRelationLabel "can be classified as" .

:local-effect :propagatesTo :next-higher-level-effect .
:edge-100 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :local-effect ;
    :hasTarget :next-higher-level-effect ;
    :hasRelationLabel "propagates to" .

:next-higher-level-effect :propagatesTo :end-user-effect .
:edge-101 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :next-higher-level-effect ;
    :hasTarget :end-user-effect ;
    :hasRelationLabel "propagates to" .

:failure-cause :causes :failure-mode .
:edge-102 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :failure-cause ;
    :hasTarget :failure-mode ;
    :hasRelationLabel "causes" .

:root-cause :explains :failure-cause .
:edge-103 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :root-cause ;
    :hasTarget :failure-cause ;
    :hasRelationLabel "explains" .

:failure-mechanism :mechanisticallyExplains :failure-cause .
:edge-104 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :failure-mechanism ;
    :hasTarget :failure-cause ;
    :hasRelationLabel "mechanistically explains" .

:hazard :canLeadTo :hazardous-situation .
:edge-105 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :hazard ;
    :hasTarget :hazardous-situation ;
    :hasRelationLabel "can lead to" .

:hazardous-situation :canResultIn :harm .
:edge-106 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :hazardous-situation ;
    :hasTarget :harm ;
    :hasRelationLabel "can result in" .
```

```
:failure-effect :mayCreate :hazard .
:edge-107 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :failure-effect ;
    :hasTarget :hazard ;
    :hasRelationLabel "may create" .

:harm :drives :severity .
:edge-108 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :harm ;
    :hasTarget :severity ;
    :hasRelationLabel "drives" .

:failure-cause :drives :occurrence .
:edge-109 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :failure-cause ;
    :hasTarget :occurrence ;
    :hasRelationLabel "drives" .

:current-control :influences :detection-rating .
:edge-110 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :current-control ;
    :hasTarget :detection-rating ;
    :hasRelationLabel "influences" .

:prevention-control :reduces :occurrence .
:edge-111 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :prevention-control ;
    :hasTarget :occurrence ;
    :hasRelationLabel "reduces" .

:detection-control :improves :detection-rating .
:edge-112 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :detection-control ;
    :hasTarget :detection-rating ;
    :hasRelationLabel "improves" .

:diagnostic-monitoring :improves :detection-rating .
:edge-113 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :diagnostic-monitoring ;
    :hasTarget :detection-rating ;
    :hasRelationLabel "improves" .

:severity :contributesTo :risk .
:edge-114 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :severity ;
    :hasTarget :risk ;
    :hasRelationLabel "contributes to" .

:occurrence :contributesTo :risk .
:edge-115 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :occurrence ;
    :hasTarget :risk ;
    :hasRelationLabel "contributes to" .

:detection-rating :modifiesPrioritizationOf :risk .
:edge-116 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :detection-rating ;
    :hasTarget :risk ;
    :hasRelationLabel "modifies prioritization of" .

:severity :multipliesInto :rpn .
:edge-117 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :severity ;
    :hasTarget :rpn ;
    :hasRelationLabel "multiplies into" .
```

```
:occurrence :multipliesInto :rpn .
:edge-118 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :occurrence ;
    :hasTarget :rpn ;
    :hasRelationLabel "multiplies into" .

:detection-rating :multipliesInto :rpn .
:edge-119 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :detection-rating ;
    :hasTarget :rpn ;
    :hasRelationLabel "multiplies into" .

:severity :contributesTo :action-priority .
:edge-120 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :severity ;
    :hasTarget :action-priority ;
    :hasRelationLabel "contributes to" .

:occurrence :contributesTo :action-priority .
:edge-121 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :occurrence ;
    :hasTarget :action-priority ;
    :hasRelationLabel "contributes to" .

:detection-rating :contributesTo :action-priority .
:edge-122 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :detection-rating ;
    :hasTarget :action-priority ;
    :hasRelationLabel "contributes to" .

:criticality :prioritizes :risk .
:edge-123 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :criticality ;
    :hasTarget :risk ;
    :hasRelationLabel "prioritizes" .

:risk-acceptance-criteria :judges :residual-risk .
:edge-124 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :risk-acceptance-criteria ;
    :hasTarget :residual-risk ;
    :hasRelationLabel "judges" .

:recommended-action :shouldTarget :failure-cause .
:edge-125 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :recommended-action ;
    :hasTarget :failure-cause ;
    :hasRelationLabel "should target" .

:recommended-action :mayTarget :failure-mode .
:edge-126 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :recommended-action ;
    :hasTarget :failure-mode ;
    :hasRelationLabel "may target" .

:recommended-action :mayReduceSeverityOf :failure-effect .
:edge-127 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :recommended-action ;
    :hasTarget :failure-effect ;
    :hasRelationLabel "may reduce severity of" .

:recommended-action :assignedTo :action-owner .
:edge-128 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :recommended-action ;
    :hasTarget :action-owner ;
    :hasRelationLabel "assigned to" .

:recommended-action :has :action-deadline .
```

```
:edge-129 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :recommended-action ;
    :hasTarget :action-deadline ;
    :hasRelationLabel "has" .

:recommended-action :isVerifiedBy :verification-evidence .
:edge-130 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :recommended-action ;
    :hasTarget :verification-evidence ;
    :hasRelationLabel "is verified by" .

:verification-evidence :supportsReassessmentOf :residual-risk .
:edge-131 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :verification-evidence ;
    :hasTarget :residual-risk ;
    :hasRelationLabel "supports reassessment of" .

:residual-risk :isRecordedIn :results-documentation .
:edge-132 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :residual-risk ;
    :hasTarget :results-documentation ;
    :hasRelationLabel "is recorded in" .

:lessons-learned :improvesFuture :fmea-study .
:edge-133 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :lessons-learned ;
    :hasTarget :fmea-study ;
    :hasRelationLabel "improves future" .

:iec-60812-2018 :covers :fmeca .
:edge-134 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :iec-60812-2018 ;
    :hasTarget :fmeca ;
    :hasRelationLabel "covers" .

:sae-j1739 :covers :dfmea .
:edge-135 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :sae-j1739 ;
    :hasTarget :dfmea ;
    :hasRelationLabel "covers" .

:sae-j1739 :covers :pfmea .
:edge-136 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :sae-j1739 ;
    :hasTarget :pfmea ;
    :hasRelationLabel "covers" .

:sae-j1739 :covers :fmea-msr .
:edge-137 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :sae-j1739 ;
    :hasTarget :fmea-msr ;
    :hasRelationLabel "covers" .

:dfmea :focusesOn :design-weakness .
:edge-138 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :dfmea ;
    :hasTarget :design-weakness ;
    :hasRelationLabel "focuses on" .

:pfmea :focusesOn :process-failure .
:edge-139 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :pfmea ;
    :hasTarget :process-failure ;
    :hasRelationLabel "focuses on" .

:fmea-msr :focusesOn :diagnostic-monitoring .
:edge-140 a owl:NamedIndividual, :OntologyEdge ;
```

```
    :hasSource :fmea-msr ;
    :hasTarget :diagnostic-monitoring ;
    :hasRelationLabel "focuses on" .

:system-fmea :emphasizes :interface-failure .
:edge-141 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :system-fmea ;
    :hasTarget :interface-failure ;
    :hasRelationLabel "emphasizes" .

:software-fmea :focusesOn :software-failure .
:edge-142 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :software-fmea ;
    :hasTarget :software-failure ;
    :hasRelationLabel "focuses on" .

:hardware-fmea :includes :electrical-failure .
:edge-143 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :hardware-fmea ;
    :hasTarget :electrical-failure ;
    :hasRelationLabel "includes" .

:hardware-fmea :includes :mechanical-failure .
:edge-144 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :hardware-fmea ;
    :hasTarget :mechanical-failure ;
    :hasRelationLabel "includes" .

:hardware-fmea :includes :electronic-failure .
:edge-145 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :hardware-fmea ;
    :hasTarget :electronic-failure ;
    :hasRelationLabel "includes" .

:fmeca :adds :criticality .
:edge-146 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :fmeca ;
    :hasTarget :criticality ;
    :hasRelationLabel "adds" .

:fmeda :adds :diagnostic-monitoring .
:edge-147 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :fmeda ;
    :hasTarget :diagnostic-monitoring ;
    :hasRelationLabel "adds" .

:human-factors-fmea :focusesOn :human-error .
:edge-148 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :human-factors-fmea ;
    :hasTarget :human-error ;
    :hasRelationLabel "focuses on" .

:iso-14971-risk-thinking :emphasizes :hazard .
:edge-149 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :iso-14971-risk-thinking ;
    :hasTarget :hazard ;
    :hasRelationLabel "emphasizes" .

:iso-14971-risk-thinking :emphasizes :hazardous-situation .
:edge-150 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :iso-14971-risk-thinking ;
    :hasTarget :hazardous-situation ;
    :hasRelationLabel "emphasizes" .

:iso-14971-risk-thinking :emphasizes :harm .
:edge-151 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :iso-14971-risk-thinking ;
```

```
    :hasTarget :harm ;
    :hasRelationLabel "emphasizes" .

:iso-14971-risk-thinking :emphasizes :risk .
:edge-152 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :iso-14971-risk-thinking ;
    :hasTarget :risk ;
    :hasRelationLabel "emphasizes" .

:failure-mode :canBeClassifiedAs :mechanical-failure .
:edge-153 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :failure-mode ;
    :hasTarget :mechanical-failure ;
    :hasRelationLabel "can be classified as" .

:failure-mode :canBeClassifiedAs :electrical-failure .
:edge-154 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :failure-mode ;
    :hasTarget :electrical-failure ;
    :hasRelationLabel "can be classified as" .

:failure-mode :canBeClassifiedAs :electronic-failure .
:edge-155 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :failure-mode ;
    :hasTarget :electronic-failure ;
    :hasRelationLabel "can be classified as" .

:failure-mode :canBeClassifiedAs :software-failure .
:edge-156 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :failure-mode ;
    :hasTarget :software-failure ;
    :hasRelationLabel "can be classified as" .

:failure-mode :canBeClassifiedAs :control-failure .
:edge-157 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :failure-mode ;
    :hasTarget :control-failure ;
    :hasRelationLabel "can be classified as" .

:failure-mode :canBeClassifiedAs :thermal-failure .
:edge-158 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :failure-mode ;
    :hasTarget :thermal-failure ;
    :hasRelationLabel "can be classified as" .

:failure-mode :canBeClassifiedAs :fluidic-failure .
:edge-159 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :failure-mode ;
    :hasTarget :fluidic-failure ;
    :hasRelationLabel "can be classified as" .

:failure-mode :canBeClassifiedAs :chemical-failure .
:edge-160 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :failure-mode ;
    :hasTarget :chemical-failure ;
    :hasRelationLabel "can be classified as" .

:failure-mode :canBeClassifiedAs :material-failure .
:edge-161 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :failure-mode ;
    :hasTarget :material-failure ;
    :hasRelationLabel "can be classified as" .

:failure-mode :canBeClassifiedAs :human-error .
:edge-162 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :failure-mode ;
    :hasTarget :human-error ;
```

```
    :hasRelationLabel "can be classified as" .

:failure-mode :canBeClassifiedAs :process-failure .
:edge-163 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :failure-mode ;
    :hasTarget :process-failure ;
    :hasRelationLabel "can be classified as" .

:failure-mode :canBeClassifiedAs :interface-failure .
:edge-164 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :failure-mode ;
    :hasTarget :interface-failure ;
    :hasRelationLabel "can be classified as" .

:failure-mode :canBeClassifiedAs :environmental-failure .
:edge-165 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :failure-mode ;
    :hasTarget :environmental-failure ;
    :hasRelationLabel "can be classified as" .

:failure-mode :canBeClassifiedAs :cybersecurity-failure .
:edge-166 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :failure-mode ;
    :hasTarget :cybersecurity-failure ;
    :hasRelationLabel "can be classified as" .

:failure-mode :canBeClassifiedAs :data-ai-failure .
:edge-167 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :failure-mode ;
    :hasTarget :data-ai-failure ;
    :hasRelationLabel "can be classified as" .

:failure-mode :canBeClassifiedAs :maintenance-failure .
:edge-168 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :failure-mode ;
    :hasTarget :maintenance-failure ;
    :hasRelationLabel "can be classified as" .

:failure-mode :canBeClassifiedAs :supply-chain-failure .
:edge-169 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :failure-mode ;
    :hasTarget :supply-chain-failure ;
    :hasRelationLabel "can be classified as" .

:mechanical-failure :hasTypicalModeOrCause :fracture .
:edge-170 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :mechanical-failure ;
    :hasTarget :fracture ;
    :hasRelationLabel "has typical mode or cause" .

:mechanical-failure :hasTypicalModeOrCause :fatigue .
:edge-171 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :mechanical-failure ;
    :hasTarget :fatigue ;
    :hasRelationLabel "has typical mode or cause" .

:mechanical-failure :hasTypicalModeOrCause :wear .
:edge-172 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :mechanical-failure ;
    :hasTarget :wear ;
    :hasRelationLabel "has typical mode or cause" .

:mechanical-failure :hasTypicalModeOrCause :loosening .
:edge-173 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :mechanical-failure ;
    :hasTarget :loosening ;
    :hasRelationLabel "has typical mode or cause" .
```

```
:mechanical_failure :hasTypicalModeOrCause :jamming .
:edge_174 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :mechanical_failure ;
    :hasTarget :jamming ;
    :hasRelationLabel "has typical mode or cause" .

:mechanical_failure :hasTypicalModeOrCause :out_of_tolerance .
:edge_175 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :mechanical_failure ;
    :hasTarget :out_of_tolerance ;
    :hasRelationLabel "has typical mode or cause" .

:mechanical_failure :hasTypicalModeOrCause :blockage .
:edge_176 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :mechanical_failure ;
    :hasTarget :blockage ;
    :hasRelationLabel "has typical mode or cause" .

:mechanical_failure :hasTypicalModeOrCause :leakage .
:edge_177 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :mechanical_failure ;
    :hasTarget :leakage ;
    :hasRelationLabel "has typical mode or cause" .

:electrical_failure :hasTypicalModeOrCause :short_circuit .
:edge_178 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :electrical_failure ;
    :hasTarget :short_circuit ;
    :hasRelationLabel "has typical mode or cause" .

:electrical_failure :hasTypicalModeOrCause :open_circuit .
:edge_179 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :electrical_failure ;
    :hasTarget :open_circuit ;
    :hasRelationLabel "has typical mode or cause" .

:electrical_failure :hasTypicalModeOrCause :insulation_breakdown .
:edge_180 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :electrical_failure ;
    :hasTarget :insulation_breakdown ;
    :hasRelationLabel "has typical mode or cause" .

:electrical_failure :hasTypicalModeOrCause :power_loss .
:edge_181 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :electrical_failure ;
    :hasTarget :power_loss ;
    :hasRelationLabel "has typical mode or cause" .

:electrical_failure :hasTypicalModeOrCause :overheating .
:edge_182 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :electrical_failure ;
    :hasTarget :overheating ;
    :hasRelationLabel "has typical mode or cause" .

:electrical_failure :hasTypicalModeOrCause :leakage .
:edge_183 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :electrical_failure ;
    :hasTarget :leakage ;
    :hasRelationLabel "has typical mode or cause" .

:electronic_failure :hasTypicalModeOrCause :emi_emc_susceptibility .
:edge_184 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :electronic_failure ;
    :hasTarget :emi_emc_susceptibility ;
    :hasRelationLabel "has typical mode or cause" .
```

```
:electronic-failure :hasTypicalModeOrCause :sensor-drift .
:edge-185 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :electronic-failure ;
    :hasTarget :sensor-drift ;
    :hasRelationLabel "has typical mode or cause" .

:electronic-failure :hasTypicalModeOrCause :overheating .
:edge-186 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :electronic-failure ;
    :hasTarget :overheating ;
    :hasRelationLabel "has typical mode or cause" .

:electronic-failure :hasTypicalModeOrCause :short-circuit .
:edge-187 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :electronic-failure ;
    :hasTarget :short-circuit ;
    :hasRelationLabel "has typical mode or cause" .

:electronic-failure :hasTypicalModeOrCause :open-circuit .
:edge-188 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :electronic-failure ;
    :hasTarget :open-circuit ;
    :hasRelationLabel "has typical mode or cause" .

:software-failure :hasTypicalModeOrCause :software-crash .
:edge-189 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :software-failure ;
    :hasTarget :software-crash ;
    :hasRelationLabel "has typical mode or cause" .

:software-failure :hasTypicalModeOrCause :timing-race-condition .
:edge-190 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :software-failure ;
    :hasTarget :timing-race-condition ;
    :hasRelationLabel "has typical mode or cause" .

:software-failure :hasTypicalModeOrCause :data-corruption .
:edge-191 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :software-failure ;
    :hasTarget :data-corruption ;
    :hasRelationLabel "has typical mode or cause" .

:software-failure :hasTypicalModeOrCause :communication-loss .
:edge-192 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :software-failure ;
    :hasTarget :communication-loss ;
    :hasRelationLabel "has typical mode or cause" .

:software-failure :hasTypicalModeOrCause :wrong-function .
:edge-193 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :software-failure ;
    :hasTarget :wrong-function ;
    :hasRelationLabel "has typical mode or cause" .

:software-failure :hasTypicalModeOrCause :late-function .
:edge-194 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :software-failure ;
    :hasTarget :late-function ;
    :hasRelationLabel "has typical mode or cause" .

:control-failure :hasTypicalModeOrCause :sensor-drift .
:edge-195 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :control-failure ;
    :hasTarget :sensor-drift ;
    :hasRelationLabel "has typical mode or cause" .

:control-failure :hasTypicalModeOrCause :actuator-stuck .
```

```
:edge_196 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :control_failure ;
    :hasTarget :actuator_stuck ;
    :hasRelationLabel "has typical mode or cause" .

:control_failure :hasTypicalModeOrCause :unstable_function .
:edge_197 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :control_failure ;
    :hasTarget :unstable_function ;
    :hasRelationLabel "has typical mode or cause" .

:control_failure :hasTypicalModeOrCause :late_function .
:edge_198 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :control_failure ;
    :hasTarget :late_function ;
    :hasRelationLabel "has typical mode or cause" .

:control_failure :hasTypicalModeOrCause :wrong_function .
:edge_199 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :control_failure ;
    :hasTarget :wrong_function ;
    :hasRelationLabel "has typical mode or cause" .

:thermal_failure :hasTypicalModeOrCause :overheating .
:edge_200 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :thermal_failure ;
    :hasTarget :overheating ;
    :hasRelationLabel "has typical mode or cause" .

:thermal_failure :hasTypicalModeOrCause :degraded_function .
:edge_201 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :thermal_failure ;
    :hasTarget :degraded_function ;
    :hasRelationLabel "has typical mode or cause" .

:thermal_failure :hasTypicalModeOrCause :out_of_tolerance .
:edge_202 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :thermal_failure ;
    :hasTarget :out_of_tolerance ;
    :hasRelationLabel "has typical mode or cause" .

:fluidic_failure :hasTypicalModeOrCause :leakage .
:edge_203 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :fluidic_failure ;
    :hasTarget :leakage ;
    :hasRelationLabel "has typical mode or cause" .

:fluidic_failure :hasTypicalModeOrCause :blockage .
:edge_204 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :fluidic_failure ;
    :hasTarget :blockage ;
    :hasRelationLabel "has typical mode or cause" .

:fluidic_failure :hasTypicalModeOrCause :actuator_stuck .
:edge_205 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :fluidic_failure ;
    :hasTarget :actuator_stuck ;
    :hasRelationLabel "has typical mode or cause" .

:fluidic_failure :hasTypicalModeOrCause :partial_function .
:edge_206 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :fluidic_failure ;
    :hasTarget :partial_function ;
    :hasRelationLabel "has typical mode or cause" .

:chemical_failure :hasTypicalModeOrCause :corrosion .
:edge_207 a owl:NamedIndividual, :OntologyEdge ;
```

```
    :hasSource :chemical_failure ;
    :hasTarget :corrosion ;
    :hasRelationLabel "has typical mode or cause" .

:chemical_failure :hasTypicalModeOrCause :contamination .
:edge_208 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :chemical_failure ;
    :hasTarget :contamination ;
    :hasRelationLabel "has typical mode or cause" .

:chemical_failure :hasTypicalModeOrCause :degraded_function .
:edge_209 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :chemical_failure ;
    :hasTarget :degraded_function ;
    :hasRelationLabel "has typical mode or cause" .

:material_failure :hasTypicalModeOrCause :fracture .
:edge_210 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :material_failure ;
    :hasTarget :fracture ;
    :hasRelationLabel "has typical mode or cause" .

:material_failure :hasTypicalModeOrCause :fatigue .
:edge_211 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :material_failure ;
    :hasTarget :fatigue ;
    :hasRelationLabel "has typical mode or cause" .

:material_failure :hasTypicalModeOrCause :wear .
:edge_212 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :material_failure ;
    :hasTarget :wear ;
    :hasRelationLabel "has typical mode or cause" .

:material_failure :hasTypicalModeOrCause :corrosion .
:edge_213 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :material_failure ;
    :hasTarget :corrosion ;
    :hasRelationLabel "has typical mode or cause" .

:material_failure :hasTypicalModeOrCause :aging_degradation .
:edge_214 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :material_failure ;
    :hasTarget :aging_degradation ;
    :hasRelationLabel "has typical mode or cause" .

:human_error :hasTypicalModeOrCause :user_misuse .
:edge_215 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :human_error ;
    :hasTarget :user_misuse ;
    :hasRelationLabel "has typical mode or cause" .

:human_error :hasTypicalModeOrCause :assembly_error .
:edge_216 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :human_error ;
    :hasTarget :assembly_error ;
    :hasRelationLabel "has typical mode or cause" .

:human_error :hasTypicalModeOrCause :inadequate_maintenance .
:edge_217 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :human_error ;
    :hasTarget :inadequate_maintenance ;
    :hasRelationLabel "has typical mode or cause" .

:human_error :hasTypicalModeOrCause :wrong_function .
:edge_218 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :human_error ;
```

```
    :hasTarget :wrong-function ;
    :hasRelationLabel "has typical mode or cause" .

:human-error :hasTypicalModeOrCause :late-function .
:edge-219 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :human-error ;
    :hasTarget :late-function ;
    :hasRelationLabel "has typical mode or cause" .

:process-failure :hasTypicalModeOrCause :manufacturing-defect .
:edge-220 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :process-failure ;
    :hasTarget :manufacturing-defect ;
    :hasRelationLabel "has typical mode or cause" .

:process-failure :hasTypicalModeOrCause :assembly-error .
:edge-221 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :process-failure ;
    :hasTarget :assembly-error ;
    :hasRelationLabel "has typical mode or cause" .

:process-failure :hasTypicalModeOrCause :contamination .
:edge-222 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :process-failure ;
    :hasTarget :contamination ;
    :hasRelationLabel "has typical mode or cause" .

:process-failure :hasTypicalModeOrCause :out-of-tolerance .
:edge-223 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :process-failure ;
    :hasTarget :out-of-tolerance ;
    :hasRelationLabel "has typical mode or cause" .

:process-failure :hasTypicalModeOrCause :supplier-variation .
:edge-224 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :process-failure ;
    :hasTarget :supplier-variation ;
    :hasRelationLabel "has typical mode or cause" .

:interface-failure :hasTypicalModeOrCause :communication-loss .
:edge-225 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :interface-failure ;
    :hasTarget :communication-loss ;
    :hasRelationLabel "has typical mode or cause" .

:interface-failure :hasTypicalModeOrCause :leakage .
:edge-226 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :interface-failure ;
    :hasTarget :leakage ;
    :hasRelationLabel "has typical mode or cause" .

:interface-failure :hasTypicalModeOrCause :blockage .
:edge-227 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :interface-failure ;
    :hasTarget :blockage ;
    :hasRelationLabel "has typical mode or cause" .

:interface-failure :hasTypicalModeOrCause :wrong-function .
:edge-228 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :interface-failure ;
    :hasTarget :wrong-function ;
    :hasRelationLabel "has typical mode or cause" .

:interface-failure :hasTypicalModeOrCause :late-function .
:edge-229 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :interface-failure ;
    :hasTarget :late-function ;
```

```
    :hasRelationLabel "has typical mode or cause" .

:interface-failure :hasTypicalModeOrCause :emi-emc-susceptibility .
:edge-230 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :interface-failure ;
    :hasTarget :emi-emc-susceptibility ;
    :hasRelationLabel "has typical mode or cause" .

:environmental-failure :hasTypicalModeOrCause :overheating .
:edge-231 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :environmental-failure ;
    :hasTarget :overheating ;
    :hasRelationLabel "has typical mode or cause" .

:environmental-failure :hasTypicalModeOrCause :corrosion .
:edge-232 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :environmental-failure ;
    :hasTarget :corrosion ;
    :hasRelationLabel "has typical mode or cause" .

:environmental-failure :hasTypicalModeOrCause :emi-emc-susceptibility .
:edge-233 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :environmental-failure ;
    :hasTarget :emi-emc-susceptibility ;
    :hasRelationLabel "has typical mode or cause" .

:environmental-failure :hasTypicalModeOrCause :contamination .
:edge-234 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :environmental-failure ;
    :hasTarget :contamination ;
    :hasRelationLabel "has typical mode or cause" .

:environmental-failure :hasTypicalModeOrCause :external-event .
:edge-235 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :environmental-failure ;
    :hasTarget :external-event ;
    :hasRelationLabel "has typical mode or cause" .

:cybersecurity-failure :hasTypicalModeOrCause :cyberattack .
:edge-236 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :cybersecurity-failure ;
    :hasTarget :cyberattack ;
    :hasRelationLabel "has typical mode or cause" .

:cybersecurity-failure :hasTypicalModeOrCause :communication-loss .
:edge-237 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :cybersecurity-failure ;
    :hasTarget :communication-loss ;
    :hasRelationLabel "has typical mode or cause" .

:cybersecurity-failure :hasTypicalModeOrCause :data-corruption .
:edge-238 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :cybersecurity-failure ;
    :hasTarget :data-corruption ;
    :hasRelationLabel "has typical mode or cause" .

:cybersecurity-failure :hasTypicalModeOrCause :wrong-function .
:edge-239 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :cybersecurity-failure ;
    :hasTarget :wrong-function ;
    :hasRelationLabel "has typical mode or cause" .

:data-ai-failure :hasTypicalModeOrCause :model-misclassification .
:edge-240 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :data-ai-failure ;
    :hasTarget :model-misclassification ;
    :hasRelationLabel "has typical mode or cause" .
```

```
:data-ai-failure :hasTypicalModeOrCause :data-corruption .
:edge-241 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :data-ai-failure ;
    :hasTarget :data-corruption ;
    :hasRelationLabel "has typical mode or cause" .

:data-ai-failure :hasTypicalModeOrCause :wrong-function .
:edge-242 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :data-ai-failure ;
    :hasTarget :wrong-function ;
    :hasRelationLabel "has typical mode or cause" .

:data-ai-failure :hasTypicalModeOrCause :degraded-function .
:edge-243 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :data-ai-failure ;
    :hasTarget :degraded-function ;
    :hasRelationLabel "has typical mode or cause" .

:maintenance-failure :hasTypicalModeOrCause :inadequate-maintenance .
:edge-244 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :maintenance-failure ;
    :hasTarget :inadequate-maintenance ;
    :hasRelationLabel "has typical mode or cause" .

:maintenance-failure :hasTypicalModeOrCause :wear .
:edge-245 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :maintenance-failure ;
    :hasTarget :wear ;
    :hasRelationLabel "has typical mode or cause" .

:maintenance-failure :hasTypicalModeOrCause :corrosion .
:edge-246 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :maintenance-failure ;
    :hasTarget :corrosion ;
    :hasRelationLabel "has typical mode or cause" .

:maintenance-failure :hasTypicalModeOrCause :sensor-drift .
:edge-247 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :maintenance-failure ;
    :hasTarget :sensor-drift ;
    :hasRelationLabel "has typical mode or cause" .

:maintenance-failure :hasTypicalModeOrCause :aging-degradation .
:edge-248 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :maintenance-failure ;
    :hasTarget :aging-degradation ;
    :hasRelationLabel "has typical mode or cause" .

:supply-chain-failure :hasTypicalModeOrCause :supplier-variation .
:edge-249 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :supply-chain-failure ;
    :hasTarget :supplier-variation ;
    :hasRelationLabel "has typical mode or cause" .

:supply-chain-failure :hasTypicalModeOrCause :manufacturing-defect .
:edge-250 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :supply-chain-failure ;
    :hasTarget :manufacturing-defect ;
    :hasRelationLabel "has typical mode or cause" .

:supply-chain-failure :hasTypicalModeOrCause :material-failure .
:edge-251 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :supply-chain-failure ;
    :hasTarget :material-failure ;
    :hasRelationLabel "has typical mode or cause" .
```

```
:supply-chain-failure :hasTypicalModeOrCause :wrong-function .
:edge-252 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :supply-chain-failure ;
    :hasTarget :wrong-function ;
    :hasRelationLabel "has typical mode or cause" .

:function :canFailBy :loss-of-function .
:edge-253 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :function ;
    :hasTarget :loss-of-function ;
    :hasRelationLabel "can fail by" .

:function :canFailBy :partial-function .
:edge-254 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :function ;
    :hasTarget :partial-function ;
    :hasRelationLabel "can fail by" .

:function :canFailBy :degraded-function .
:edge-255 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :function ;
    :hasTarget :degraded-function ;
    :hasRelationLabel "can fail by" .

:function :canFailBy :intermittent-function .
:edge-256 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :function ;
    :hasTarget :intermittent-function ;
    :hasRelationLabel "can fail by" .

:function :canFailBy :unintended-function .
:edge-257 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :function ;
    :hasTarget :unintended-function ;
    :hasRelationLabel "can fail by" .

:function :canFailBy :wrong-function .
:edge-258 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :function ;
    :hasTarget :wrong-function ;
    :hasRelationLabel "can fail by" .

:function :canFailBy :early-function .
:edge-259 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :function ;
    :hasTarget :early-function ;
    :hasRelationLabel "can fail by" .

:function :canFailBy :late-function .
:edge-260 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :function ;
    :hasTarget :late-function ;
    :hasRelationLabel "can fail by" .

:function :canFailBy :unstable-function .
:edge-261 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :function ;
    :hasTarget :unstable-function ;
    :hasRelationLabel "can fail by" .

:function :canFailBy :out-of-tolerance .
:edge-262 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :function ;
    :hasTarget :out-of-tolerance ;
    :hasRelationLabel "can fail by" .

:input :canBeAffectedBy :leakage .
```

```
:edge-263 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :input ;
    :hasTarget :leakage ;
    :hasRelationLabel "can be affected by" .

:output :canBeAffectedBy :leakage .
:edge-264 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :output ;
    :hasTarget :leakage ;
    :hasRelationLabel "can be affected by" .

:input :canBeAffectedBy :blockage .
:edge-265 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :input ;
    :hasTarget :blockage ;
    :hasRelationLabel "can be affected by" .

:output :canBeAffectedBy :blockage .
:edge-266 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :output ;
    :hasTarget :blockage ;
    :hasRelationLabel "can be affected by" .

:requirement-ambiguity :isACategoryOf :failure-cause .
:edge-267 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :requirement-ambiguity ;
    :hasTarget :failure-cause ;
    :hasRelationLabel "is a category of" .

:design-weakness :isACategoryOf :failure-cause .
:edge-268 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :design-weakness ;
    :hasTarget :failure-cause ;
    :hasRelationLabel "is a category of" .

:manufacturing-defect :isACategoryOf :failure-cause .
:edge-269 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :manufacturing-defect ;
    :hasTarget :failure-cause ;
    :hasRelationLabel "is a category of" .

:assembly-error :isACategoryOf :failure-cause .
:edge-270 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :assembly-error ;
    :hasTarget :failure-cause ;
    :hasRelationLabel "is a category of" .

:contamination :isACategoryOf :failure-cause .
:edge-271 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :contamination ;
    :hasTarget :failure-cause ;
    :hasRelationLabel "is a category of" .

:overload :isACategoryOf :failure-cause .
:edge-272 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :overload ;
    :hasTarget :failure-cause ;
    :hasRelationLabel "is a category of" .

:aging-degradation :isACategoryOf :failure-cause .
:edge-273 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :aging-degradation ;
    :hasTarget :failure-cause ;
    :hasRelationLabel "is a category of" .

:supplier-variation :isACategoryOf :failure-cause .
:edge-274 a owl:NamedIndividual, :OntologyEdge ;
```

```
    :hasSource :supplier-variation ;
    :hasTarget :failure-cause ;
    :hasRelationLabel "is a category of" .

:user-misuse :isACategoryOf :failure-cause .
:edge-275 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :user-misuse ;
    :hasTarget :failure-cause ;
    :hasRelationLabel "is a category of" .

:inadequate-maintenance :isACategoryOf :failure-cause .
:edge-276 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :inadequate-maintenance ;
    :hasTarget :failure-cause ;
    :hasRelationLabel "is a category of" .

:external-event :isACategoryOf :failure-cause .
:edge-277 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :external-event ;
    :hasTarget :failure-cause ;
    :hasRelationLabel "is a category of" .

:cyberattack :isACategoryOf :failure-cause .
:edge-278 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :cyberattack ;
    :hasTarget :failure-cause ;
    :hasRelationLabel "is a category of" .

:software-defect :isACategoryOf :failure-cause .
:edge-279 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :software-defect ;
    :hasTarget :failure-cause ;
    :hasRelationLabel "is a category of" .

:requirement-ambiguity :canCause :wrong-function .
:edge-280 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :requirement-ambiguity ;
    :hasTarget :wrong-function ;
    :hasRelationLabel "can cause" .

:design-weakness :canCause :loss-of-function .
:edge-281 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :design-weakness ;
    :hasTarget :loss-of-function ;
    :hasRelationLabel "can cause" .

:manufacturing-defect :canCause :out-of-tolerance .
:edge-282 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :manufacturing-defect ;
    :hasTarget :out-of-tolerance ;
    :hasRelationLabel "can cause" .

:assembly-error :canCause :loosening .
:edge-283 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :assembly-error ;
    :hasTarget :loosening ;
    :hasRelationLabel "can cause" .

:contamination :canCause :blockage .
:edge-284 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :contamination ;
    :hasTarget :blockage ;
    :hasRelationLabel "can cause" .

:overload :canCause :fracture .
:edge-285 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :overload ;
```

```
    :hasTarget :fracture ;
    :hasRelationLabel "can cause" .

:aging-degradation :canCause :degraded-function .
:edge-286 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :aging-degradation ;
    :hasTarget :degraded-function ;
    :hasRelationLabel "can cause" .

:supplier-variation :canCause :material-failure .
:edge-287 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :supplier-variation ;
    :hasTarget :material-failure ;
    :hasRelationLabel "can cause" .

:user-misuse :canCause :overload .
:edge-288 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :user-misuse ;
    :hasTarget :overload ;
    :hasRelationLabel "can cause" .

:inadequate-maintenance :canCause :wear .
:edge-289 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :inadequate-maintenance ;
    :hasTarget :wear ;
    :hasRelationLabel "can cause" .

:external-event :canTrigger :environmental-failure .
:edge-290 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :external-event ;
    :hasTarget :environmental-failure ;
    :hasRelationLabel "can trigger" .

:cyberattack :canCause :cybersecurity-failure .
:edge-291 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :cyberattack ;
    :hasTarget :cybersecurity-failure ;
    :hasRelationLabel "can cause" .

:software-defect :canCause :software-crash .
:edge-292 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :software-defect ;
    :hasTarget :software-crash ;
    :hasRelationLabel "can cause" .

:design-review :canBeUsedAs :current-control .
:edge-293 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :design-review ;
    :hasTarget :current-control ;
    :hasRelationLabel "can be used as" .

:simulation-analysis :canBeUsedAs :current-control .
:edge-294 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :simulation-analysis ;
    :hasTarget :current-control ;
    :hasRelationLabel "can be used as" .

:tolerance-analysis :canBeUsedAs :current-control .
:edge-295 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :tolerance-analysis ;
    :hasTarget :current-control ;
    :hasRelationLabel "can be used as" .

:finite-element-analysis :canBeUsedAs :current-control .
:edge-296 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :finite-element-analysis ;
    :hasTarget :current-control ;
```

```
    :hasRelationLabel "can be used as" .

:fault-tree-analysis :canBeUsedAs :current-control .
:edge-297 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :fault-tree-analysis ;
    :hasTarget :current-control ;
    :hasRelationLabel "can be used as" .

:test-validation :canBeUsedAs :current-control .
:edge-298 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :test-validation ;
    :hasTarget :current-control ;
    :hasRelationLabel "can be used as" .

:inspection :canBeUsedAs :current-control .
:edge-299 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :inspection ;
    :hasTarget :current-control ;
    :hasRelationLabel "can be used as" .

:poka-yoke :canBeUsedAs :current-control .
:edge-300 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :poka-yoke ;
    :hasTarget :current-control ;
    :hasRelationLabel "can be used as" .

:redundancy :canBeUsedAs :current-control .
:edge-301 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :redundancy ;
    :hasTarget :current-control ;
    :hasRelationLabel "can be used as" .

:derating :canBeUsedAs :current-control .
:edge-302 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :derating ;
    :hasTarget :current-control ;
    :hasRelationLabel "can be used as" .

:safety-margin :canBeUsedAs :current-control .
:edge-303 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :safety-margin ;
    :hasTarget :current-control ;
    :hasRelationLabel "can be used as" .

:fail-safe-state :canBeUsedAs :current-control .
:edge-304 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :fail-safe-state ;
    :hasTarget :current-control ;
    :hasRelationLabel "can be used as" .

:graceful-degradation :canBeUsedAs :current-control .
:edge-305 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :graceful-degradation ;
    :hasTarget :current-control ;
    :hasRelationLabel "can be used as" .

:alarm-alert :canBeUsedAs :current-control .
:edge-306 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :alarm-alert ;
    :hasTarget :current-control ;
    :hasRelationLabel "can be used as" .

:maintenance-plan :canBeUsedAs :current-control .
:edge-307 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :maintenance-plan ;
    :hasTarget :current-control ;
    :hasRelationLabel "can be used as" .
```

```
:training–instruction :canBeUsedAs :current–control .
:edge–308 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :training–instruction ;
    :hasTarget :current–control ;
    :hasRelationLabel "can be used as" .

:change–control :canBeUsedAs :current–control .
:edge–309 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :change–control ;
    :hasTarget :current–control ;
    :hasRelationLabel "can be used as" .

:traceability :canBeUsedAs :current–control .
:edge–310 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :traceability ;
    :hasTarget :current–control ;
    :hasRelationLabel "can be used as" .

:design–review :canDetect :requirement–ambiguity .
:edge–311 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :design–review ;
    :hasTarget :requirement–ambiguity ;
    :hasRelationLabel "can detect" .

:simulation–analysis :canEvaluate :control–failure .
:edge–312 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :simulation–analysis ;
    :hasTarget :control–failure ;
    :hasRelationLabel "can evaluate" .

:tolerance–analysis :canEvaluate :jamming .
:edge–313 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :tolerance–analysis ;
    :hasTarget :jamming ;
    :hasRelationLabel "can evaluate" .

:finite–element–analysis :canEvaluate :fracture .
:edge–314 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :finite–element–analysis ;
    :hasTarget :fracture ;
    :hasRelationLabel "can evaluate" .

:finite–element–analysis :canEvaluate :fatigue .
:edge–315 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :finite–element–analysis ;
    :hasTarget :fatigue ;
    :hasRelationLabel "can evaluate" .

:fault–tree–analysis :canAnalyzeTopEventFor :hazard .
:edge–316 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :fault–tree–analysis ;
    :hasTarget :hazard ;
    :hasRelationLabel "can analyze top event for" .

:test–validation :produces :verification–evidence .
:edge–317 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :test–validation ;
    :hasTarget :verification–evidence ;
    :hasRelationLabel "produces" .

:inspection :canDetect :manufacturing–defect .
:edge–318 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :inspection ;
    :hasTarget :manufacturing–defect ;
    :hasRelationLabel "can detect" .
```

```
:poka-yoke :canPrevent :assembly-error .
:edge-319 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :poka-yoke ;
    :hasTarget :assembly-error ;
    :hasRelationLabel "can prevent" .

:redundancy :canMitigate :loss-of-function .
:edge-320 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :redundancy ;
    :hasTarget :loss-of-function ;
    :hasRelationLabel "can mitigate" .

:derating :canPrevent :overheating .
:edge-321 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :derating ;
    :hasTarget :overheating ;
    :hasRelationLabel "can prevent" .

:safety-margin :canMitigate :overload .
:edge-322 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :safety-margin ;
    :hasTarget :overload ;
    :hasRelationLabel "can mitigate" .

:fail-safe-state :canReduce :harm .
:edge-323 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :fail-safe-state ;
    :hasTarget :harm ;
    :hasRelationLabel "can reduce" .

:graceful-degradation :canReduce :end-user-effect .
:edge-324 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :graceful-degradation ;
    :hasTarget :end-user-effect ;
    :hasRelationLabel "can reduce" .

:alarm-alert :canWarnAbout :hazardous-situation .
:edge-325 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :alarm-alert ;
    :hasTarget :hazardous-situation ;
    :hasRelationLabel "can warn about" .

:maintenance-plan :canManage :aging-degradation .
:edge-326 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :maintenance-plan ;
    :hasTarget :aging-degradation ;
    :hasRelationLabel "can manage" .

:training-instruction :canReduce :user-misuse .
:edge-327 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :training-instruction ;
    :hasTarget :user-misuse ;
    :hasRelationLabel "can reduce" .

:change-control :canControl :supplier-variation .
:edge-328 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :change-control ;
    :hasTarget :supplier-variation ;
    :hasRelationLabel "can control" .

:traceability :linksTo :requirement .
:edge-329 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :traceability ;
    :hasTarget :requirement ;
    :hasRelationLabel "links to" .

:traceability :linksTo :verification-evidence .
```

```
:edge_330 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :traceability ;
    :hasTarget :verification_evidence ;
    :hasRelationLabel "links to" .

:traceability :linksTo :recommended_action .
:edge_331 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :traceability ;
    :hasTarget :recommended_action ;
    :hasRelationLabel "links to" .

:ai_fmea_facilitator :asks :question_context .
:edge_332 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :ai_fmea_facilitator ;
    :hasTarget :question_context ;
    :hasRelationLabel "asks" .

:ai_fmea_facilitator :asks :question_function .
:edge_333 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :ai_fmea_facilitator ;
    :hasTarget :question_function ;
    :hasRelationLabel "asks" .

:ai_fmea_facilitator :asks :question_interface .
:edge_334 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :ai_fmea_facilitator ;
    :hasTarget :question_interface ;
    :hasRelationLabel "asks" .

:ai_fmea_facilitator :asks :question_failure_mode .
:edge_335 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :ai_fmea_facilitator ;
    :hasTarget :question_failure_mode ;
    :hasRelationLabel "asks" .

:ai_fmea_facilitator :asks :question_effect .
:edge_336 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :ai_fmea_facilitator ;
    :hasTarget :question_effect ;
    :hasRelationLabel "asks" .

:ai_fmea_facilitator :asks :question_cause .
:edge_337 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :ai_fmea_facilitator ;
    :hasTarget :question_cause ;
    :hasRelationLabel "asks" .

:ai_fmea_facilitator :asks :question_control .
:edge_338 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :ai_fmea_facilitator ;
    :hasTarget :question_control ;
    :hasRelationLabel "asks" .

:ai_fmea_facilitator :asks :question_evidence .
:edge_339 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :ai_fmea_facilitator ;
    :hasTarget :question_evidence ;
    :hasRelationLabel "asks" .

:ai_fmea_facilitator :asks :question_counterfactual .
:edge_340 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :ai_fmea_facilitator ;
    :hasTarget :question_counterfactual ;
    :hasRelationLabel "asks" .

:ai_fmea_facilitator :asks :question_cross_domain .
:edge_341 a owl:NamedIndividual, :OntologyEdge ;
```

```
    :hasSource :ai-fmea-facilitator ;
    :hasTarget :question-cross-domain ;
    :hasRelationLabel "asks" .

:ai-fmea-facilitator :asks :question-missing-info .
:edge-342 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :ai-fmea-facilitator ;
    :hasTarget :question-missing-info ;
    :hasRelationLabel "asks" .

:ai-fmea-facilitator :asks :question-prioritization .
:edge-343 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :ai-fmea-facilitator ;
    :hasTarget :question-prioritization ;
    :hasRelationLabel "asks" .

:question-context :clarifies :scope .
:edge-344 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :question-context ;
    :hasTarget :scope ;
    :hasRelationLabel "clarifies" ;
    :hasQuestionPrompt "What exactly is the item, boundary, lifecycle phase, use
        scenario, and environment?" .

:question-function :elicits :function .
:edge-345 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :question-function ;
    :hasTarget :function ;
    :hasRelationLabel "elicits" ;
    :hasQuestionPrompt "What must the item do, for whom, under which mode, and how is
        success measured?" .

:question-interface :probes :interface .
:edge-346 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :question-interface ;
    :hasTarget :interface ;
    :hasRelationLabel "probes" ;
    :hasQuestionPrompt "What crosses the interface, in which direction, under which
        timing, tolerance, protocol, or compatibility assumption?" .

:question-failure-mode :generates :failure-mode .
:edge-347 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :question-failure-mode ;
    :hasTarget :failure-mode ;
    :hasRelationLabel "generates" ;
    :hasQuestionPrompt "For each function, how can it be absent, partial, degraded,
        intermittent, unintended, wrong, early, late, unstable, or out-of-tolerance?" .

:question-effect :traces :failure-effect .
:edge-348 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :question-effect ;
    :hasTarget :failure-effect ;
    :hasRelationLabel "traces" ;
    :hasQuestionPrompt "What happens locally, downstream, at system level, to the user,
        and in the worst credible case?" .

:question-cause :elicits :failure-cause .
:edge-349 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :question-cause ;
    :hasTarget :failure-cause ;
    :hasRelationLabel "elicits" ;
    :hasQuestionPrompt "Why would this failure mode occur mechanically, electrically,
        electronically, in software, by process, by user interaction, by environment, by
         supplier, or by cyber event?" .

:question-control :checks :current-control .
:edge-350 a owl:NamedIndividual, :OntologyEdge ;
```

```
    :hasSource :question-control ;
    :hasTarget :current-control ;
    :hasRelationLabel "checks" ;
    :hasQuestionPrompt "What currently prevents the cause, detects the failure, alerts
        the user, isolates the fault, or moves the system to a safe state?" .

:question-evidence :requests :verification-evidence .
:edge-351 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :question-evidence ;
    :hasTarget :verification-evidence ;
    :hasRelationLabel "requests" ;
    :hasQuestionPrompt "What data, test, simulation, inspection, field history, or
        expert source supports the rating and control claim?" .

:question-counterfactual :suggests :prevention-control .
:edge-352 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :question-counterfactual ;
    :hasTarget :prevention-control ;
    :hasRelationLabel "suggests" ;
    :hasQuestionPrompt "What design, process, interface, monitoring, or training change
        would make this failure impossible or less likely?" .

:question-cross-domain :broadens :failure-mode .
:edge-353 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :question-cross-domain ;
    :hasTarget :failure-mode ;
    :hasRelationLabel "broadens" ;
    :hasQuestionPrompt "Could this same effect be caused by mechanical, electrical,
        software, human, process, environmental, cyber, or data/AI failure?" .

:question-missing-info :surfaces :assumption .
:edge-354 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :question-missing-info ;
    :hasTarget :assumption ;
    :hasRelationLabel "surfaces" ;
    :hasQuestionPrompt "What are we assuming because drawings, requirements, schematics,
         logs, test data, supplier specs, or process maps are missing?" .

:question-prioritization :prioritizes :recommended-action .
:edge-355 a owl:NamedIndividual, :OntologyEdge ;
    :hasSource :question-prioritization ;
    :hasTarget :recommended-action ;
    :hasRelationLabel "prioritizes" ;
    :hasQuestionPrompt "Which action is justified first given severity, occurrence,
        detection, uncertainty, stakeholder harm, and feasibility?" .
```

Listing C.1: OWL/Turtle representation of the domain-agnostic FMEA ontology

Appendix D

SysML Model Parser Algorithm

This appendix presents the pseudocode for the SysML model parser used in the framework. The parser transforms a raw textual SysML v2-style model into structured part-level records. These records are then used as bounded system-specific context for AI-supported FMEA reasoning.

Algorithm 2 SysML Model Parser

Require: Raw textual SysML v2-style model M
Ensure: Parsed part records $\mathscr{P}$

$M \leftarrow$ strip code fences, normalise line endings, and remove surrounding whitespace
$M_{masked} \leftarrow$ mask line comments and block comments while preserving character positions
$\mathscr{B}_{part} \leftarrow$ parse all `part` and `ref part` declarations in M_{masked}
$\mathscr{B}_{action} \leftarrow$ parse all `action` declarations in M_{masked}
$\mathscr{B}_{req} \leftarrow$ parse all `requirement` declarations in M_{masked}
for all declaration b in $\mathscr{B}_{part} \cup \mathscr{B}_{action} \cup \mathscr{B}_{req}$ **do**
    if b has an opening brace **then**
        Find the matching closing brace using brace-depth counting
        Store declaration name, start index, end index, header, body, and full text
    else
        Store declaration as a single-line declaration
    end if
end for
for all part $p \in \mathscr{B}_{part}$ **do**
    Assign the smallest containing part block as `parent_part`
    Assign all directly contained parts as children of p
end for
for all part $p \in \mathscr{B}_{part}$ **do**
    if p has no child parts **then**
        Set $\texttt{layer}(p) \leftarrow 0$
    else
        Set $\texttt{layer}(p) \leftarrow 1 + \max(\texttt{layer}(c))$ for all children c of p
    end if
end for
$\mathscr{S} \leftarrow$ extract global relation statements from M_{masked} ▷ `connect`, `bind`, `flow`, `perform`, `allocate`, `transition`
$\mathscr{P} \leftarrow \emptyset$
for all part $p \in \mathscr{B}_{part}$ **do**
    $D_p \leftarrow$ body of p with nested child-part blocks removed
    $F_p \leftarrow$ extract local features from D_p ▷ ports, attributes, performed actions, bindings, connections, and states
    $N_p \leftarrow$ build related-name set for p ▷ part name, path, ports, attributes, actions, and child names
    $\mathscr{A}_p \leftarrow$ select actions from $\mathscr{B}_{action}$ that match performed actions in F_p
    $\mathscr{S}_p \leftarrow \emptyset$
    for all statement $s \in \mathscr{S}$ **do**
        if s is not inside an unrelated part and s mentions an element in N_p **then**
            Add s to $\mathscr{S}_p$
        end if
    end for
    $\mathscr{R}_p \leftarrow$ select requirements from $\mathscr{B}_{req}$ referenced by allocations or related statements in $\mathscr{S}_p$
    $C_p \leftarrow$ assemble labelled context sections from $\mathscr{A}_p$, F_p, $\mathscr{S}_p$, $\mathscr{R}_p$, and state/control statements
    Add record with fields `part_name`, `parent_part`, `layer`, `part_chunk`, and `context_text` to $\mathscr{P}$
end for
return $\mathscr{P}$

Appendix E

FMEA Risk Scoring Prompt

This appendix presents the prompt used for risk scoring in the generated FMEA table. The scoring prompt operationalises the risk-rating step by instructing the AI to evaluate each failure mode using Severity (S), Occurrence (O), and Detection (D) scores on a scale from 1 to 10. Severity captures the seriousness of the failure effect, Occurrence captures the expected likelihood of the failure, and Detection captures how difficult the failure is to detect before its effect occurs. Since a higher Detection value represents weaker detectability, failures with poor detection mechanisms receive higher risk scores. The Risk Priority Number is calculated as $RPN = S \times O \times D$, after which the row is assigned a low, medium, or high risk class according to predefined thresholds and severity override rules. The prompt also requires a recommended action, scoring reasons, and explicit assumptions when information is missing. In this way, the scoring step remains structured, explainable, and suitable for human review rather than being treated as an automatic final judgement.

```
You are an expert FMEA risk scoring assistant.

Your task is to score each failure mode using Severity (S), Occurrence (O), and
    Detection (D).

You will receive a failure mode. Each record may include:

- part_name
- failure_mode
- effect

Your job is to assign:
- Severity, S
- Occurrence, O
- Detection, D
- RPN = S x O x D
- risk_class
- scoring reasons

Return only valid JSON.
Do not return Markdown.
Do not wrap the JSON in code fences.

======================================================================
SCORING RULES
======================================================================

Severity (S) means: how serious is the effect if the failure happens?

Use this scale:
```

```
10 = safety hazard, fire, injury, regulatory violation, or catastrophic system
    failure
9 = complete system failure with serious consequences
8 = system cannot perform its main function
7 = major performance loss
6 = degraded main function; user strongly affected
5 = partial function loss; user notices the problem
4 = minor degradation of performance
3 = small inconvenience
2 = barely noticeable effect
1 = no meaningful effect

Occurrence (O) means: how likely is the failure to happen?

Use this scale:

10 = very frequent / expected often
9 = repeated failures likely
8 = high likelihood
7 = moderately high likelihood
6 = occasional
5 = possible during product life
4 = low but realistic
3 = unlikely
2 = very unlikely
1 = remote / almost impossible

Increase Occurrence when the failure involves:
- wear
- corrosion
- mechanical movement
- user interaction
- electrical contact
- exposed ports
- environmental exposure
- complex control/software behavior

Decrease Occurrence when the part is:
- passive
- protected
- simple
- redundant
- unlikely to be stressed

Detection (D) means: how likely the failure is to be detected before it causes
    the effect.

Important: higher Detection score is worse.

Use this scale:

10 = no detection method
9 = detection very unlikely
8 = weak detection; usually detected only after failure
7 = detection depends on user noticing
6 = manual inspection only
5 = basic test can detect it
4 = functional test likely detects it
3 = specific verification test detects it
2 = automatic diagnostic or strong inspection detects it
1 = almost certain prevention or detection

If no detection control is specified, use D between 7 and 10.
If only user observation is available, use D between 7 and 8.
If testing or inspection is specified, use D between 4 and 6.
If automatic diagnostics or monitoring exist, use D between 1 and 3.
```

```
======================================================================
RISK CLASS RULES
======================================================================

Calculate:

RPN = S x O x D

Risk class:

High if RPN >= 200
Medium if RPN >= 80 and RPN < 200
Low if RPN < 80

Severity override:

If S >= 9, risk_class must be at least Medium.
If S = 10 and O >= 3, risk_class should usually be High.
If S >= 9 and D >= 7, risk_class should usually be High.

======================================================================
RESIDUAL RISK RULES
======================================================================

Also recommend one action to control or mitigate risk.

======================================================================
OUTPUT FORMAT
======================================================================

Return a JSON array.

Each output object must follow this schema:

[
  {
    "part_name": "",
    "failure_mode": "",
    "effect": "",
    "severity": 1,
    "severity_reason": "",
    "occurrence": 1,
    "occurrence_reason": "",
    "detection": 1,
    "detection_reason": "",
    "rpn": 1,
    "risk_class": "Low",
    "recommended_action": "",
    "assumptions": []
  }
]

Be consistent, explainable, and conservative.
If information is missing, make a reasonable engineering assumption and include
    it in "assumptions".
```

Listing E.1: Prompt used for FMEA risk scoring

Appendix F

Generated Quadcopter FMEA Table

This appendix presents the complete generated and scored FMEA output for the quadcopter case study. The table is placed in the appendix because it is too large for the main results chapter. The table records the generated item, failure mode, effect, severity, occurrence, detection, Risk Priority Number (RPN), risk class, and recommended action.

The table should be interpreted as the generated output of the framework, not as a fully validated industrial FMEA. Several repeated or near-duplicate rows remain in the generated output, especially for `propellerGuard`. These repetitions are discussed in Chapter 5 as evidence that the global review stage requires stronger duplicate consolidation and score harmonisation.

Table F.1: Generated and scored FMEA rows for the quadcopter case study

Item	Failure mode	Effect	S/O/D/RPN	Risk	Recommended action
propellerGuard	structural damage	Reduced ability to protect users and bystanders from rotating propellers, increasing injury risk.	9/5/7/315	High	Implement routine detailed inspections and use more impact-resistant materials to improve durability.
propellerGuard	breakage	Loss of protection from rotating propellers, significantly increasing risk of injury to users and bystanders.	10/4/8/320	High	Design guard for higher impact resistance and develop fail-safe design or secondary containment; increase inspection frequency.
propellerGuard	detachment	Propeller guard detaches during flight, exposing rotating propellers and increasing risk of injury or damage.	10/5/8/400	High	Implement improved fastening design with redundant attachment points and conduct routine vibration and attachment integrity inspections.
propellerGuard	loose fitting	Propeller guard shifts during flight due to loose fitting, potentially contacting rotating propellers and increasing risk of injury.	9/6/7/378	High	Use locking mechanisms or quality control checks during assembly to ensure tight fit; increase maintenance frequency to check fittings.
propellerGuard	interference with propeller rotation	May cause propeller damage, loss of thrust, unstable flight conditions, and increased risk of crash.	9/4/8/288	High	Implement pre-flight functional check procedures and improve design to minimize possibility of interference; consider adding sensors or indicators to detect guard obstruction before flight.
propellerGuard	material degradation (e.g. UV, fatigue)	Gradual weakening of the propeller guard material due to environmental exposure, leading to structural failure under stress and loss of its protective function.	7/6/7/294	High	Implement regular scheduled inspections and replace the propeller guard proactively based on usage and environmental exposure. Use UV-resistant materials to improve durability.
arm	No relevant failure modes found	No specific effects identified due to lack of data.	2/3/8/48	Low	Perform detailed analysis to identify specific failure modes and implement detection methods to improve risk awareness.
centerBody	No relevant failure modes found	No effects identified due to lack of data.	1/1/10/10	Low	Perform detailed analysis or testing to identify potential failure modes and effects for this part to enable proper risk assessment.
landingGear	No failure modes found in ontology	No data available on effects of failure modes for landingGear.	5/3/8/120	Medium	Perform detailed FMEA with available component and operational data to identify failure modes and implement appropriate detection methods and preventive maintenance.
payloadMount	No relevant failure modes found	No failure modes have been identified for the payloadMount part in the current ontology context.	1/1/10/10	Low	Perform detailed design analysis or testing to identify potential failure modes for payloadMount to enable proper risk management.
arm	No relevant failure modes found	No specific effects identified due to lack of data	1/1/10/10	Low	Gather detailed failure mode and effect information to enable proper risk assessment and mitigation.
centerBody	No relevant failure modes found	No effects identified due to lack of data	1/1/10/10	Low	Perform detailed analysis to identify potential failure modes and effects for accurate risk assessment.

Continued on next page

Table F.1: Generated and scored FMEA rows for the quadcopter case study (continued)

Item	Failure mode	Effect	S/O/D/RPN	Risk	Recommended action
landingGear	No failure modes found in ontology	No data available on effects of failure modes for landingGear	9/3/8/216	High	Implement regular and thorough inspection and maintenance routines, add condition monitoring sensors to detect wear or malfunction early.
propellerGuard	structural damage	Reduced or lost ability to protect from rotating propellers, increasing injury risk to users and bystanders.	9/5/8/360	High	Implement regular maintenance inspection protocols and use more durable materials or design reinforcements to reduce structural damage likelihood.
propellerGuard	breakage	Reduced or lost ability to protect from rotating propellers, increasing injury risk to users and bystanders.	9/4/8/288	High	Increase guard robustness, implement periodic visual inspections, and consider incorporating fracture indicators to alert users.
propellerGuard	detachment or loose fitting	Propeller guard may detach or shift during flight, increasing risk of contact with rotating propellers and causing injury.	9/4/7/252	High	Implement a positive locking mechanism with visual and tactile indicators and enforce mandatory pre-flight inspection checklist for secure fitting of the propeller guard.
propellerGuard	interference with propeller rotation	Causes propeller damage, loss of thrust, unstable flight, and increases crash risk.	9/5/7/315	High	Implement a design review to ensure propellerGuard cannot interfere with propeller rotation under any operating condition and include physical inspection procedures during maintenance.
propellerGuard	material degradation (e.g. UV exposure, fatigue)	Weakening of guard structure over time leading to failure under stress and loss of protective function.	7/6/8/336	High	Implement routine scheduled inspections focusing on material integrity and UV damage, use UV-resistant materials, and consider design changes that provide increased durability or redundancy in the guard structure.
propellerGuard	structural damage	Loss or reduction of protection from rotating propellers, increasing injury risk to users and bystanders.	9/5/8/360	High	Implement regular scheduled inspections and a robust damage detection protocol, possibly including automated sensors or indicators of guard integrity.
propellerGuard	breakage	Complete loss of physical barrier from rotating propellers, significantly increasing injury risk to users and bystanders.	10/4/9/360	High	Design for improved material strength, incorporate breakage indicators, and enforce mandatory inspections before use.
propellerGuard	detachment	Guard may detach during flight, risking contact with rotating propellers and causing injury.	10/5/8/400	High	Implement improved mechanical securing methods and routine inspection procedures before flight to prevent detachment.
propellerGuard	loose fitting	Guard may shift during flight due to loose fitting, risking contact with rotating propellers and causing injury.	9/6/7/378	High	Enhance fastening mechanism robustness and establish mandatory pre-flight fitting checks by the user.
propellerGuard	interference with propeller rotation	Causes propeller damage, loss of thrust, unstable flight, and increased risk of crash.	9/4/8/288	High	Implement regular pre-flight inspection protocols and improvement of propellerGuard design to prevent interference.

Continued on next page

Table F.1: Generated and scored FMEA rows for the quadcopter case study (continued)

Item	Failure mode	Effect	S/O/D/RPN	Risk	Recommended action
`propellerGuard`	material degradation (e.g., UV exposure, fatigue)	Weakening over time leading to structural failure and loss of protective function.	7/6/8/336	High	Use UV-resistant materials and implement scheduled inspection and replacement program to prevent structural failure.